Deployment-Oriented Neural Network State-Estimation Methods for Battery Management Systems:

Ageing Estimation, Robustness Benchmarking, Embedded Optimization, and Hardware Integration

submitted by
M. Sc.
Florian Rzepka

Chair of Electrical Energy Storage Technology
Institute of Energy and Automation Technology
Faculty IV, Technische Universität Berlin
in partial fulfillment of the requirements for the academic degree of

Doctor of Engineering
Dr.-Ing.

Doctoral thesis

Doctoral Committee:

Chair: tbd
Reviewer: tbd
Reviewer: tbd

Date of the scientific defense: tbd

Berlin, 2026

Preface

This research work took shape during my time at the Chair of Electrical Energy Storage Technology at Technische Universität Berlin. Its questions emerged gradually from the research environment there and from my own interest in how artificial intelligence can support battery-state estimation and battery management under real operating conditions rather than in idealized settings. This direction was reinforced by my involvement in the MG-FARM project, an international research collaboration on smart, stand-alone micro-grids for the electrification of remote agricultural regions. Working alongside university and industry partners in Morocco, France, and Algeria, I was able to see at first hand how decentralized energy systems and their battery storage are deployed and operated in places where grid access, connectivity, and maintenance are limited. Taken together, these influences shaped the central question of this thesis: how battery-state estimation can be made not only accurate, but also robust against imperfect measurements and executable on the modest embedded hardware that such systems actually rely on.

I am grateful for the support I received in many forms throughout this work. My sincere thanks go above all to my supervisor, Prof. Dr.-Ing. Julia Kowal, and to the Chair of Electrical Energy Storage Technology at Technische Universität Berlin, for the guidance and the freedom to pursue this topic in my own way. I thank my colleagues at the chair for countless discussions and for a productive working atmosphere, and the partners of the MG-FARM consortium across Morocco, France, and Algeria for their experience and a genuinely international perspective on decentralized energy. I also thank my family and friends for their steady support throughout this time. I am especially grateful to my partner Hannah, whose positive spirit and steady encouragement helped me regain confidence and continue through the difficult phases of this work.

Abstract

Reliable estimates of state of charge (SOC) and state of health (SOH) are central to the safe and efficient operation of lithium-ion battery systems. At the same time, many neural battery estimators are still evaluated mainly as offline models under clean laboratory signals, whereas practical battery management systems (BMS) impose additional requirements through disturbed measurements, limited embedded resources, firmware constraints, and the physical measurement chain. This research therefore develops a deployment-oriented perspective on artificial intelligence for BMS applications. Predictive accuracy is treated as one requirement among others, and is assessed together with robustness, embedded feasibility, and hardware integration.

The work is organized around four connected studies grounded in NMC and LFP ageing datasets and a custom laboratory BMS platform. First, a neural ageing-estimation study shows that a compact multilayer perceptron can estimate SOH with competitive accuracy when cumulative-current information and lag-sequence history are used to represent ageing-relevant temporal context. Second, a measurement-only robustness benchmark compares direct-measurement, hybrid, equivalent-circuit, and data-driven estimator classes under current bias, noise, missing samples, irregular sampling, burst dropout, voltage spikes, ADC quantization, and initialization mismatch. The benchmark demonstrates that clean-condition MAE alone is not sufficient for practical model selection: estimators that are accurate under nominal signals can fail under bias or disturbed sampling, while other classes recover faster or remain more stable under specific faults. Third, an embedded-AI study transfers stateful recurrent SOC and SOH estimators to an STM32 microcontroller through structured pruning and INT8 weight quantization, measuring accuracy, flash, RAM, and latency and deriving a clearly identified constant-power energy proxy. Fourth, the custom BMS platform provides the hardware and firmware context in which estimator exchangeability, sensing, and

embedded validation can be investigated reproducibly.

The central conclusion is that AI-enabled battery-state estimation should not be judged by a single accuracy number or by one nominal test. The goal is not to identify one universally best estimator, but to understand how different estimator classes behave across the requirement dimensions relevant to practical BMS deployment. For this purpose, accuracy, disturbance robustness, recovery behaviour, compressibility, runtime cost, and hardware integration must be evaluated together.

Table of Contents

Abbreviations

2RC	Two-resistor-capacitor equivalent circuit model
ADC	Analog-to-digital converter
AFE	Analog front end
AI	Artificial Intelligence
BMS	Battery Management System
CC-CV	Constant-current constant-voltage charging
CV	Constant voltage
DD	Data-driven estimator
DM	Direct-measurement estimator
DoD	Depth of discharge
DWT	Data Watchpoint and Trace
ECM	Equivalent Circuit Model
EFC	Equivalent Full Cycles
EMI	Electromagnetic interference
FP16	16-bit floating-point format
FP32	32-bit floating-point format
GELU	Gaussian error linear unit
GPU	Graphics processing unit

GRU	Gated recurrent unit
HDM	Hybrid direct-measurement model
HECM	Hybrid equivalent-circuit model
HPPC	Hybrid pulse-power characterization
IC	Integrated circuit
INT8	8-bit signed integer representation
KPI	Key performance indicator
LFP	Lithium iron phosphate
LSTM	Long Short-Term Memory
MAE	Mean absolute error
MCU	Microcontroller Unit
MG-FARM	Smart micro-grid research project for remote agricultural regions
MLP	Multilayer Perceptron
MSE	Mean squared error
NMC	Lithium nickel manganese cobalt oxide
NPU	Neural processing unit
NTC	Negative temperature coefficient thermistor
OCV	Open-circuit voltage
ONNX	Open Neural Network Exchange
P95	95th percentile
PV	Photovoltaic
RAM	Random-access memory
RC	Resistor-capacitor

ReLU	Rectified linear unit
RGB	Red-green-blue color representation
RMSE	Root mean squared error
SRAM	Static random-access memory
STM32	STMicroelectronics 32-bit microcontroller family
STM32H7	STM32 high-performance microcontroller family based on Arm Cortex-M7
STM32H753ZI	STM32H7 microcontroller target used for the embedded benchmark
STM32N6	STM32 microcontroller family with integrated neural accelerator
SOC	State of Charge
SOC-GRU	SOC estimator based on a GRU recurrent core
SOH	State of Health
SOH-LSTM	SOH estimator based on an LSTM recurrent core
TU	Technische Universität
UART	Universal asynchronous receiver-transmitter
`.bss`	Linker section for zero-initialized or uninitialized static and global data
`.data`	Linker section for initialized static and global data copied from flash to RAM

Symbols

α	Smoothing factor of the causal SOH post-filter
ΔI	Current quantization step
ΔMAE	Difference in mean absolute error relative to the baseline case
Δt	Sampling interval
ΔT	Temperature quantization step
ΔU	Voltage quantization step
ε	Small numerical constant used in normalization
η	Coulombic efficiency factor in the equivalent-circuit observer
σ_I	Standard deviation of injected current noise
σ_T	Standard deviation of injected temperature noise
σ_U	Standard deviation of injected voltage noise
τ_1, τ_2	Time constants of the two RC branches
$\mathbf{A}$	State-transition matrix of the equivalent-circuit observer
$\mathbf{b}$	Input vector of the equivalent-circuit observer
$\mathbf{c}_t$	LSTM cell state at sample t
$\mathbf{f}_t$	LSTM forget-gate activation
$\tilde{\mathbf{c}}_t$	LSTM candidate cell state at sample t
$\mathbf{h}_t$	Recurrent hidden state at sample t

$\tilde{\mathbf{h}}_t$	GRU candidate hidden state at sample t
$\mathbf{i}_t$	LSTM input-gate activation
$\mathbf{o}_t$	LSTM output-gate activation
$\mathbf{r}_t$	GRU reset-gate activation
$\mathbf{x}$	State vector or model input vector, depending on context
$\mathbf{z}_t$	GRU update-gate activation
b	Neural-network bias vector
$C(t)$	Capacity measured at time t
C_b	SOH-scaled capacity used by the equivalent-circuit observer
C_{app}	Apparent capacity used in SOC reference reconstruction
C_{chg}	Charge C-rate in the LFP ageing design
C_{dis}	Discharge C-rate in the LFP ageing design
C_{eff}	Effective capacity obtained from nominal capacity and estimated SOH
C_k	Measured discharge capacity during check-up k
C_{N}	Nominal capacity of the NMC cell
C_{nom}	Nominal cell capacity
E	Energy per inference used in the embedded utility score
F	Flash-memory footprint used in the embedded utility score
H	Recurrent hidden-state size
I	Current
I_k	Current sample at discrete time index k
$\lvert I\rvert_{\max}$	Maximum absolute current magnitude used to scale current-bias scenarios

k	Discrete time index
L_{SOC}	SOC training sequence length
m_k	Median of feature channel k used for robust normalization
n	Number of evaluated samples
P	Covariance matrix of the observer correction step
q	INT8 quantized weight code
Q	Cumulative charge or charge throughput
Q_c	Online cumulative charge state
r_k	Interquartile range of feature channel k used for robust normalization
R	Total RAM usage in the embedded utility score
R_0, R_1, R_2	Ohmic and polarization resistances of the equivalent-circuit model
R^2	Coefficient of determination
s	Quantization scale factor
s_h	Structured-pruning importance score of hidden channel h
SOC	State of charge
SOH	State of health
t	Time
T	Temperature
U	Voltage or composite utility score, depending on context
U_1, U_2	Polarization voltages of the two RC branches
W	Neural-network weight matrix
w	Individual neural-network weight
$\hat{w}$	Reconstructed floating-point weight after INT8 quantization

$x_{t,k}$	Feature value of channel k at sample t
y_i	Measured target value of sample i
$\hat{y}_i$	Predicted target value of sample i
$\mathrm{d}I/\mathrm{d}t$	Current derivative
$\mathrm{d}U/\mathrm{d}t$	Voltage derivative
DoD	Depth of discharge
EFC	Equivalent full cycles
MAE	Mean absolute error
MSE	Mean squared error
OCV	Open-circuit voltage
P95	95th percentile of the absolute error
RMSE	Root mean squared error

List of Figures

List of Tables

Chapter 1

Introduction and Motivation

Lithium-ion batteries are increasingly used in systems that must operate reliably for long periods with limited maintenance access. This includes stationary storage in decentralized energy systems, for example remote microgrids in regions where grid connection can be limited or unavailable. The same requirement appears in mobile and autonomous applications that have to continue operating when communication, remote control, or service access is restricted. In all of these cases, the battery-management system (BMS) needs reliable information about internal battery states that cannot be measured directly. The state of charge (SOC) affects usable energy and power limits, while the state of health (SOH) affects lifetime assessment, maintenance planning, and safe operating margins.

Artificial-intelligence-based estimation methods are attractive because battery behaviour is nonlinear and depends on ageing, load history, temperature, and operating point. The problem is not simply to obtain a low error on a clean laboratory test set. A method that is useful in a BMS must also handle imperfect measurements, causal online execution, memory and timing limits, and the practical route from a trained model to firmware. This research therefore examines battery-state estimation as a connected development and deployment problem rather than only as an offline modelling task.

1.1 Research Aim and Questions

The aim of this work is to develop and evaluate AI-based battery-state estimation methods along the path from data representation to embedded execution. The work is guided by three research questions:

1. How can simple, low-complexity models capture the complex and time-dependent ageing behaviour of lithium-ion batteries without resorting to large or elaborate architectures?

2. How do battery-state estimators behave under realistic operating and measurement conditions, and which factors determine their robustness in practice?

3. How can such estimators be transferred to embedded BMS hardware and optimized to run within its memory, latency, and energy limits?

Each question targets a different stage of this path. The first concerns model design. Ageing develops slowly and depends on prior operation, so a compact estimator must receive sufficient information about this history. The second concerns evaluation. An estimator that performs well on clean data must also be confronted with disturbed and incomplete signals before its operational reliability can be judged. The third concerns realization. A model is useful in a BMS only if it remains accurate while meeting the resource and timing budget of the target controller.

1.2 Contribution, Scope, and Publication Basis

The work makes three main technical contributions. First, it studies how lag-sequence construction and cumulative-current history can turn a memoryless multilayer perceptron (MLP) into a time-aware SOH estimator without increasing the architectural complexity unnecessarily. Second, it develops a measurement-only robustness benchmark that compares direct-measurement, hybrid model-based, and data-driven SOC estimators under one common disturbance protocol. Third, it transfers recurrent SOC and SOH models to STM32 microcontroller execution and evaluates the interaction between model architecture, compression, and measured embedded performance.

The ageing-estimation contribution presented in Chapter 4 is based on the published study by Rzepka et al.[1] The robustness benchmark in Chapter 5 forms the basis of a manuscript submitted to the *Journal of Energy Storage*.[2] The embedded-AI study in Chapter 6 is available as an SSRN preprint.[3] The chapter texts are integrated and expanded for the monographic research argument rather than reproduced as independent articles.

These contributions are supported by an experimental basis consisting of two ageing datasets and a custom BMS platform. The hardware platform is not treated as a separate algorithmic contribution in the same sense as the three estimator studies. It provides the measurement, firmware, and validation environment required to connect the methodological results to practical BMS implementation.

1.3 Battery-Management Requirement Framework

A BMS supervises a lithium-ion cell or pack and keeps it within safe and efficient operating limits by monitoring voltage, current, and temperature and by controlling charging, balancing, protection, and communication. SOC and SOH are central internal quantities in this control chain. They influence usable energy, power limitation, lifetime-aware operation, and maintenance decisions, but they cannot be obtained directly from one sensor channel. They must be inferred from the measurement stream and from knowledge about the cell or pack.[4, 5, 6]

For a BMS, state estimation is therefore not only an algorithmic accuracy problem. The estimator output has to be reliable enough for operating decisions, the implementation has to fit the controller and sensing chain, and the method has to be transferable into maintainable firmware. Figure 1.1 organizes these constraints into three connected dimensions.

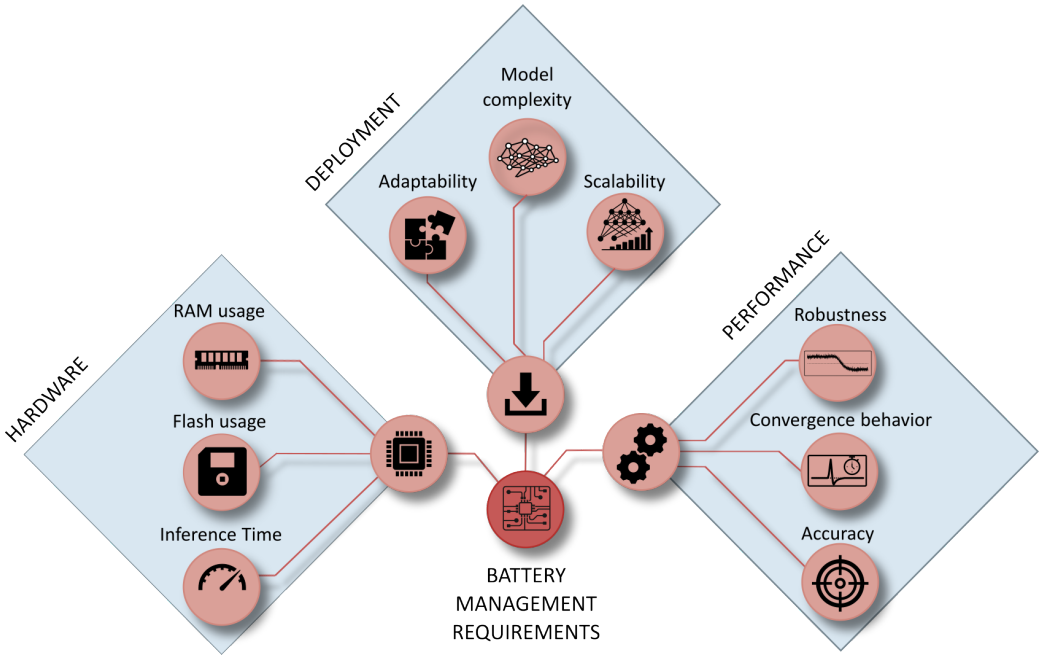


Figure 1.1: Battery-management requirement framework used throughout this work. The three connected dimensions cover estimator performance, the hardware resource envelope, and the engineering path to deployment.

1.3.1 Performance requirements

Performance requirements describe whether the estimated SOC or SOH signal is suitable for BMS decisions during operation. In this work, performance is not reduced to average error under clean measurements. It also includes robustness against disturbed input signals and convergence or recovery after an inconsistent initial state. This distinction is important because bias, noise, missing samples, timing irregularities, and communication interruptions can change the estimator ranking even when the clean-data error is low.[7, 8, 9]

1.3.2 Hardware requirements

Hardware requirements define the resource envelope in which state estimation can run on the BMS controller. Flash memory limits the amount of code, parameters, and lookup data that can be stored. RAM usage limits the size of recurrent states, intermediate buffers, and runtime data. Inference time determines whether the estimator can meet the required update rate with sufficient timing margin. Energy per inference becomes relevant when the controller has to operate under strict power budgets.[10, 11] The measurement chain belongs to the same requirement area because sensor resolution, noise, sampling timing, and communication behaviour define the input information available to the estimator.

1.3.3 Deployment requirements

Deployment requirements describe the engineering path from a trained or identified method to a reusable BMS implementation. Relevant aspects are model complexity, firmware transfer, calibration effort, adaptability to changed cells or operating conditions, and scalability to related estimation tasks or target platforms. A method is therefore not deployment-ready only because it runs once on a controller. It must also be reproducible, maintainable, and modifiable without rebuilding the full BMS software around it. Compression methods such as pruning and quantization can support deployment, but their effect depends on the architecture and must be verified on the target implementation.[12, 13]

1.4 Structure of the Research and Development Path

Chapter 2 introduces the battery, BMS, state-estimation, machine-learning, and embedded-AI foundations required by the later studies. Chapter 3 describes the two datasets, the construction of the reference quantities, the embedded targets, and the Custom BMS Platform. Chapters 4 to 6 contain the three technical contributions. Chapter 7 combines the general discussion, overall conclusion, remaining gaps, and outlook. The requirement framework in Figure 1.1 is revisited in this final synthesis rather than repeated as a separate introductory chapter.

Table 1.1: Structure of the research, function of the contribution chapters, and associated publication basis.

Chapter	Role	Primary dimension	Publication basis
3	Experimental and hardware basis	Data, labels, and validation context	Shared basis for all three contribution studies
4	Ageing estimation for stationary storage	Representation design and SOH estimation	Published journal article[1]
5	Robustness benchmark for battery-state estimation	Disturbed-operation performance	Manuscript submitted to the *Journal of Energy Storage*[2]
6	Embedded AI for SOC and SOH estimation	Deployment and compression	SSRN preprint[3]

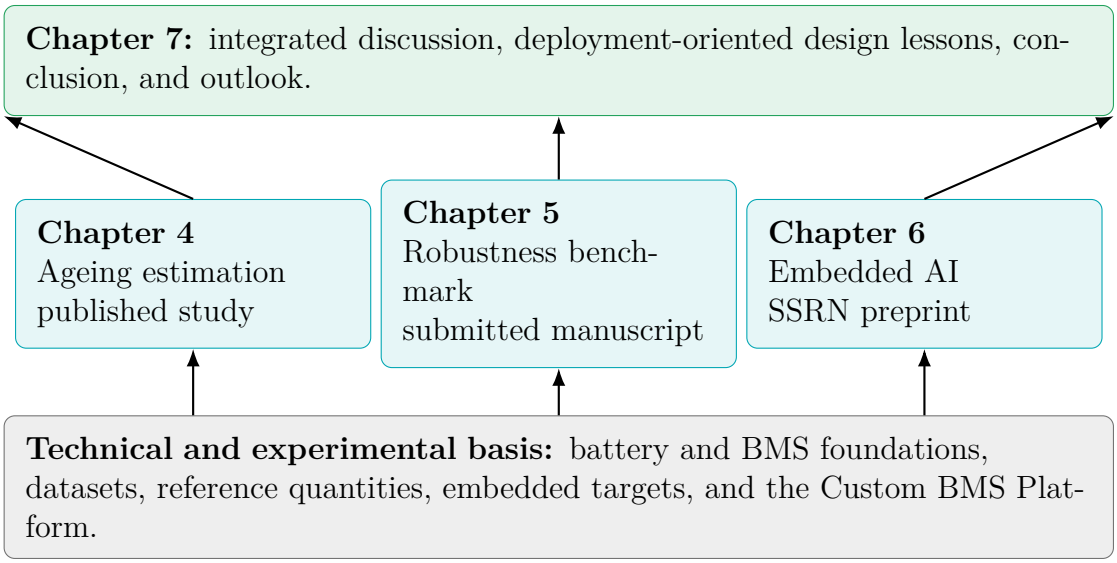


Figure 1.2: Roadmap from the shared basis to the three contribution studies and their integrated synthesis. Chapters 4 to 6 correspond to a published study, a submitted manuscript, and an SSRN preprint.[1, 2, 3]

Chapter 2

State of the Art and Technical Foundations

2.1 Lithium-Ion Cell Fundamentals

A lithium-ion cell stores energy through reversible movement of lithium ions between a negative and a positive electrode. During charging, lithium ions leave the positive electrode and are inserted into the negative electrode, which is commonly graphite. During discharge, the direction is reversed and electrons flow through the external circuit. A porous separator prevents direct electrical contact between the electrodes, while the electrolyte provides ionic transport. The resulting terminal voltage is not determined by SOC alone. It combines the equilibrium open-circuit voltage with ohmic, charge-transfer, diffusion, temperature, and hysteresis effects.[14, 4]

A reduced representation of this relation is

$$U_{\mathrm{t}}(t) = U_{\mathrm{OCV}}(z(t), T(t)) - \eta_{\mathrm{ohm}}(t) - \eta_{\mathrm{dyn}}(t), \tag{2.1}$$

where z denotes SOC, U_{OCV} is the equilibrium voltage, and the two overvoltage terms collect the immediate resistive drop and slower electrochemical dynamics. This relation explains why a measured voltage cannot generally be converted into an unambiguous SOC during load. Current integration provides temporal continuity, while voltage-based information can correct drift when the cell model and operating point are sufficiently observable.

Battery loading is commonly described by the C-rate and depth of discharge (DOD). A current of 1C would transfer the nominal capacity in one hour under idealized conditions. DOD describes the fraction of capacity removed within a cycle. Equivalent full cycles (EFC) aggregate absolute charge throughput into full-capacity equivalents and therefore provide a common loading coordinate when the

individual cycles have different depths. Temperature, C-rate, mean SOC, and DOD affect both the immediate voltage response and long-term degradation.

2.1.1 NMC and LFP cell chemistry

The experiments in this work use cells with nickel-manganese-cobalt oxide (NMC) and lithium iron phosphate (LFP) positive electrodes. Both are lithium-ion chemistries, but their voltage characteristics and typical application trade-offs differ. NMC cells combine comparatively high specific energy with a voltage curve that varies noticeably over much of the SOC range. LFP cells are valued for thermal stability and cycle life, but their open-circuit-voltage curve contains a broad, comparatively flat plateau. On this plateau, a voltage difference may correspond to a large SOC interval. Voltage-only SOC estimation is therefore weakly observable in this region, and reliable estimation places greater emphasis on current integration, dynamic voltage response, temperature, and history.[14, 4, 15]

Table 2.1: Qualitative differences between the NMC and LFP cell chemistries used in this work. The properties are typical tendencies rather than universal specifications for every commercial cell.

Aspect	NMC	LFP
Positive-electrode family	Layered transition-metal oxide	Olivine phosphate
Typical application strength	Higher specific energy and compact storage	Thermal stability and long cycle life
Voltage characteristic	More pronounced SOC-dependent slope over broad regions	Broad and comparatively flat voltage plateau
Estimation consequence	Voltage correction can be informative, but remains temperature- and model-dependent	Voltage alone provides limited SOC information on the plateau, increasing the importance of current history and dynamic features

2.1.2 Ageing, capacity, and state of health

Lithium-ion ageing has calendar and cyclic components. Calendar ageing develops with time and depends strongly on temperature and storage SOC. Cyclic ageing is additionally shaped by charge throughput, current rate, DOD, and operating boundaries. The observable consequences include loss of usable capacity and an increase in internal resistance. These quantities do not necessarily change in a perfectly smooth or monotonic way from one diagnostic test to the next. Relaxation and reversible recovery, small differences in conditioning and temperature, and uncertainty in current integration and cut-off detection can cause a later capacity test to return a slightly higher value even though the long-term degradation trend remains downward.[16, 17, 18]

SOH is therefore a derived quantity whose exact meaning depends on the selected reference. Capacity-based SOH compares the currently measured usable capacity with a nominal or initial reference capacity. Resistance-based SOH instead tracks the increase in internal resistance. This work uses capacity-based SOH because the datasets contain periodic controlled capacity tests. The precise label construction for the NMC and LFP campaigns is specified in Chapter 3.

2.2 Battery Management Systems and Battery States

Practical BMS implementations combine measurement, protection, control, communication, and state estimation. A cell-monitoring front end records cell voltages and often controls passive balancing. Current and temperature sensors provide pack-level and thermal information. A microcontroller executes protection logic, communication, diagnostics, and higher-level estimation algorithms. Isolation, contactor or relay control, and external interfaces complete the system.[19, 20, 21, 22, 23]

Within this architecture, SOC describes the currently available charge relative to a defined usable capacity. SOH describes degradation relative to a capacity or resistance reference. State of power describes the short-term charging or discharging capability under voltage, current, and temperature limits. These states cannot be read from an individual sensor. They must be reconstructed from measured voltage, current, temperature, time, and a representation of the cell dynamics.[4, 5, 6]

The main estimator families are direct-measurement methods, model-based observers, data-driven models, and combinations of these approaches. Figure 2.1 summarizes this relationship. Coulomb counting is the principal direct method. Equivalent-circuit or electrochemical models provide the physical basis for observers. Neural networks learn a nonlinear mapping from measured and engineered variables to the target state. Hybrid estimators combine physical propagation with learned capacity, parameter, or state correction.[24, 25, 26, 27]

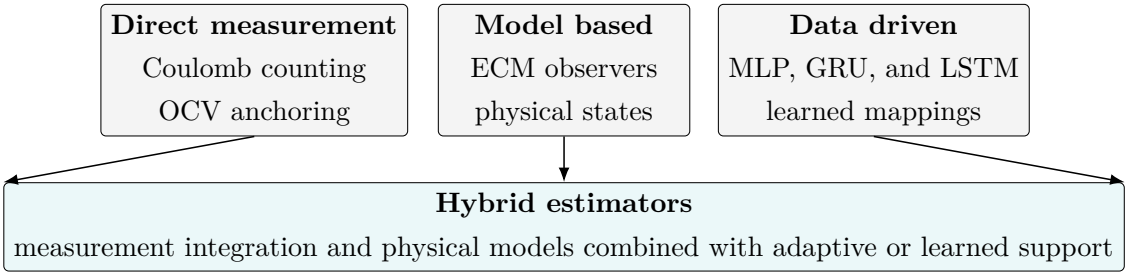


Figure 2.1: Main families of battery-state estimators. Hybrid methods combine elements of direct measurement, physical model structure, and data-driven adaptation.

2.3 Model-Based Estimation and Kalman Filtering

Equivalent-circuit models (ECMs) represent the terminal behaviour of a cell through an open-circuit-voltage source, an ohmic resistance, and one or more resistor-capacitor branches. The SOC and polarization voltages form the model state. Measured current is the input, while the predicted terminal voltage is compared with the measured voltage. This structure is computationally efficient and is therefore widely used in practical BMS observers.[4, 28, 29]

A nonlinear discrete state-space model can be written as

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k) + \mathbf{w}_k, \tag{2.2}$$

$$\mathbf{y}_k = h(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{v}_k, \tag{2.3}$$

where $\mathbf{x}_k$ contains the hidden battery states, $\mathbf{u}_k$ contains measured inputs, and $\mathbf{y}_k$ contains the predicted measurements. The process-noise term $\mathbf{w}_k$ represents uncertainty in the state model, while $\mathbf{v}_k$ represents measurement uncertainty.

The extended Kalman filter (EKF) alternates between model-based prediction and measurement-based correction. Using the local Jacobians $\mathbf{F}_k$ and $\mathbf{H}_k$, the

principal steps are

$$\hat{\mathbf{x}}_k^- = f(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k), \tag{2.4}$$

$$\mathbf{P}_k^- = \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^\mathsf{T} + \mathbf{Q}_k, \tag{2.5}$$

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}_k^\mathsf{T} \left(\mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^\mathsf{T} + \mathbf{R}_k \right)^{-1}, \tag{2.6}$$

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + \mathbf{K}_k \left(\mathbf{y}_k - h(\hat{\mathbf{x}}_k^-, \mathbf{u}_k) \right). \tag{2.7}$$

The covariance matrix $\mathbf{P}_k$ describes state uncertainty, while $\mathbf{Q}_k$ and $\mathbf{R}_k$ tune the assumed process and measurement uncertainty. The Kalman gain $\mathbf{K}_k$ determines how strongly the voltage residual corrects the predicted state. The unscented Kalman filter avoids explicit local linearization by propagating selected sigma points through the nonlinear functions. Both approaches depend on a sufficiently accurate cell model and suitable parameterization over SOC, temperature, and ageing.[30, 25, 26]

Kalman-filter-based estimators remain an important application benchmark because they combine causal execution, voltage correction, and physical interpretation. Their limitations are equally important. Model mismatch, inaccurate open-circuit-voltage relations, poorly identified resistance-capacitance parameters, and inappropriate covariance tuning can introduce systematic errors. LFP cells are particularly challenging when the voltage residual is small on the flat OCV plateau. Neural estimators exchange this explicit physical structure for a learned nonlinear mapping and therefore require a different form of validation.

2.4 Machine Learning and Neural-Network Foundations

2.4.1 Supervised regression, features, and labels

The studies in this work formulate SOC and SOH estimation as supervised regression. Each training example contains an input vector or sequence $\mathbf{x}$ and a target label y. Direct sensor values such as voltage, current, and temperature are measurements. Features are the variables presented to the model and can contain both measurements and causally derived quantities such as cumulative charge, EFC, derivatives, or lagged values. Labels are reference SOC or SOH values used to adapt and evaluate the model. They can rely on controlled capacity tests or other

dataset-side information that is unavailable to an online estimator.

The model parameters $\boldsymbol{\theta}$ are adjusted to minimize a training loss. For a scalar target, the mean-squared-error loss is

$$\mathcal{L}_{\mathrm{MSE}}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{n=1}^{N}\left(y_n - \widehat{y}_{\boldsymbol{\theta}}(\mathbf{x}_n)\right)^2. \tag{2.8}$$

Separate validation data are used for model selection and overfitting control. A final test set must remain independent of both weight fitting and hyperparameter decisions. For time-dependent battery data, splitting complete cells or complete trajectories is generally more informative than randomly separating neighbouring samples because adjacent windows are strongly correlated.[27, 31]

2.4.2 Artificial neurons and multilayer perceptrons

An artificial neuron maps an input vector $\mathbf{x}$ to one scalar output in two steps. It first forms the affine pre-activation z_j and then applies the activation function f,

$$\begin{aligned} z_j &= \sum_{i=1}^{n_{\mathrm{in}}} w_{ji}x_i + b_j, \\ y_j &= f(z_j). \end{aligned} \tag{2.9}$$

The weights w_{ji} determine the contribution of input x_i, while the bias b_j shifts the pre-activation. Without a nonlinear activation function, several stacked layers would still be equivalent to one affine mapping. Nonlinear functions therefore enable the network to represent curved and interacting relationships between the input variables.

A multilayer perceptron stacks these neuron operations into fully connected layers. Every neuron in one layer receives the activations of all neurons in the preceding layer. Let $n_{\ell-1}$ and n_ℓ denote the input and output dimensions of layer ℓ, and set $\mathbf{a}^{(0)} = \mathbf{x}$. The complete layer mapping is

$$\begin{aligned} \mathbf{z}^{(\ell)} &= \mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \\ \mathbf{a}^{(\ell)} &= f^{(\ell)}\left(\mathbf{z}^{(\ell)}\right), \qquad \ell = 1, \ldots, L, \\ \widehat{\mathbf{y}} &= \mathbf{a}^{(L)}. \end{aligned} \tag{2.10}$$

Here, $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ contains the trainable connection weights and $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$ contains one bias per neuron. The scalar entry $w_{ji}^{(\ell)} = [\mathbf{W}^{(\ell)}]_{ji}$ is the connection

from neuron i in layer $\ell - 1$ to neuron j in layer ℓ. The corresponding bias $b_j^{(\ell)}$ belongs to the target neuron and is added to its complete weighted sum, not to an individual connection. The hidden activation vectors $\mathbf{a}^{(\ell)}$ form increasingly task-specific representations before the final layer produces $\widehat{\mathbf{y}}$. Rectified linear units are frequently used in the hidden layers,

$$f_{\mathrm{ReLU}}(z) = \max(0, z), \tag{2.11}$$

because they are inexpensive to evaluate and preserve a useful gradient for positive activations. A regression output can remain linear when the target is not explicitly bounded. A sigmoid output is suitable when the prediction must remain between zero and one.

For a dense layer with n_{in} inputs and n_{out} neurons, the number of trainable weights and biases is

$$N_{\mathrm{param}}^{(\ell)} = n_{\mathrm{in}} n_{\mathrm{out}} + n_{\mathrm{out}}. \tag{2.12}$$

Increasing width or depth can improve the representational capacity, but it also increases parameter storage, arithmetic effort, and the risk of fitting dataset-specific details. Validation data, regularisation, and a suitable architecture search are therefore required to balance approximation capability and generalisation.

Figure 2.2 summarizes this feedforward structure and links the scalar neuron calculation in Equation (2.9) to the layer-wise mapping in Equation (2.10). Each layer transforms the complete activation vector of its predecessor. The grey connections denote full connectivity, while the highlighted edge from input x_i to hidden neuron j identifies the scalar matrix entry $w_{ji}^{(1)} = [\mathbf{W}^{(1)}]_{ji}$. Information propagates only from the input toward the output.

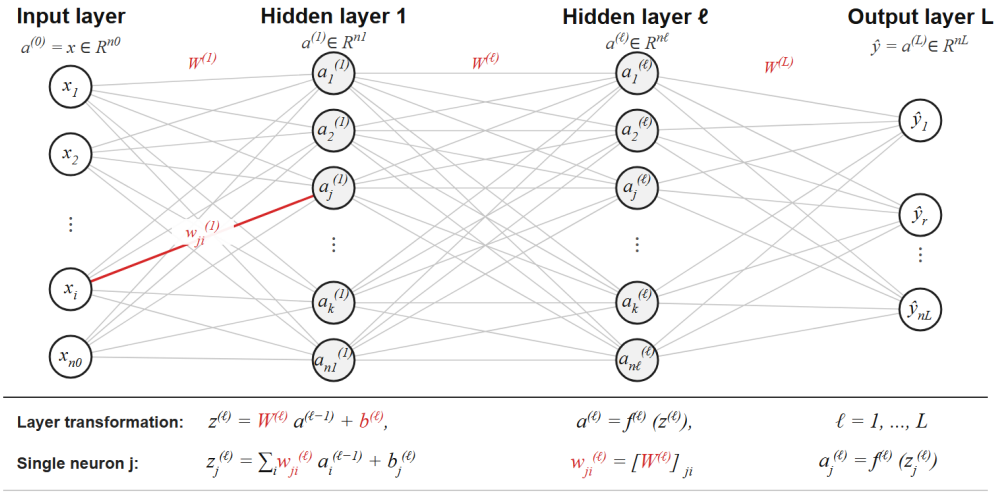


Figure 2.2: General structure and layer-wise mathematical mapping of a fully connected multilayer perceptron. The input vector $\mathbf{x} = \mathbf{a}^{(0)}$ is propagated through hidden representations $\mathbf{a}^{(\ell)}$ to the output $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$. The highlighted connection from neuron i to neuron j represents the scalar entry $w_{ji}^{(1)}$ of the matrix $\mathbf{W}^{(1)}$. The bias is added separately at the target neuron.

An MLP applies the same static mapping to every input vector and has no internal temporal state. Time dependence must therefore be supplied explicitly, for example through cumulative quantities or a lag sequence of previous measurements. This gives the designer direct control over the represented history and allows efficient parallel training. The limitation is that the selected history length and temporal aggregation are fixed before training. This is the strategy used for the ageing estimator in Chapter 4.

2.4.3 Recurrent neural networks

A basic recurrent neural network carries a hidden state $\mathbf{h}_t$ from one sample to the next,

$$\mathbf{h}_t = f_{\mathrm{RNN}}(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)\,. \tag{2.13}$$

Here, f_{RNN} denotes the recurrent activation function. The hidden state is a learned representation of the preceding sequence. It is not a directly interpretable physical battery state, although it can encode information related to load history, dynamic voltage response, temperature, and ageing. During training, errors are propagated backwards through the unfolded sequence. Repeated multiplication by recurrent

weights can cause gradients to vanish or grow rapidly, which makes long dependencies difficult to learn with a basic recurrent network. Gated architectures provide controlled paths for retaining, updating, and exposing information.[32, 33]

Long short-term memory

The long short-term memory (LSTM) cell carries two recurrent vectors. The cell state $\mathbf{c}_t$ forms the internal memory path, while the hidden state $\mathbf{h}_t$ is the exposed representation passed to the next time step and to subsequent network layers. Four affine transformations calculate a forget gate $\mathbf{f}_t$, an input gate $\mathbf{i}_t$, a candidate cell state $\tilde{\mathbf{c}}_t$, and an output gate $\mathbf{o}_t$:

$$\begin{aligned}
\mathbf{f}_t &= \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f)\,, \\
\mathbf{i}_t &= \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i)\,, \\
\tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_g \mathbf{x}_t + \mathbf{U}_g \mathbf{h}_{t-1} + \mathbf{b}_g)\,, \\
\mathbf{o}_t &= \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o)\,, \\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \\
\mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t).
\end{aligned} \tag{2.14}$$

Here, $\odot$ denotes element-wise multiplication. The sigmoid gates take values between zero and one. The forget gate scales the retained part of $\mathbf{c}_{t-1}$, the input gate scales the candidate information written into the memory, and the output gate determines how much of the updated cell state becomes visible in $\mathbf{h}_t$. The candidate $\tilde{\mathbf{c}}_t$ contains the new signed content proposed for storage. The additive cell-state update provides a more direct path through time than the repeated nonlinear replacement in a basic recurrent network. It supports long-term information transport, although the practically retained horizon still depends on the learned gate activations, training data, state dimension, and sequence handling.

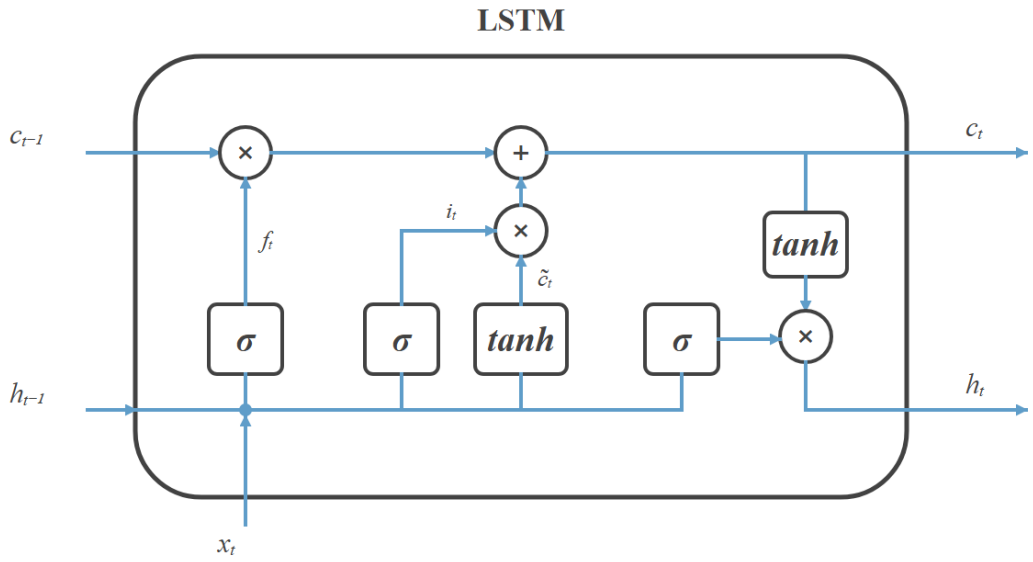


Figure 2.3: Detailed data flow through an LSTM cell corresponding to Equation (2.14). The forget gate controls the retained cell state, the input gate controls the candidate write operation, and the output gate controls the exposed hidden state. Blue paths indicate the directed state and gate signals, $\odot$ denotes element-wise multiplication, and the addition node forms the updated cell state.

Figure 2.3 separates the recurrent cell from any application-specific input preparation or output network. At each time step, the cell receives the current feature vector and both states from the preceding step. The updated hidden state can be passed to a regression head, while the hidden and cell states are carried into the next update. A unidirectional LSTM can therefore operate causally in an online BMS. A bidirectional network would additionally require future samples and is not suitable for the real-time estimators considered here.

For input dimension D and hidden dimension H, each of the four transformations contains an input matrix of size $H \times D$ and a recurrent matrix of size $H \times H$. With one combined bias vector, the parameter count is

$$N_{\text{param}}^{\text{LSTM}} = 4H(D + H + 1). \tag{2.15}$$

Software libraries normally stack the four input matrices and the four recurrent matrices for efficient evaluation. In the PyTorch gate order, for example, $\mathbf{W}_{ih} = [\mathbf{W}_i; \mathbf{W}_f; \mathbf{W}_g; \mathbf{W}_o]$ and $\mathbf{W}_{hh} = [\mathbf{U}_i; \mathbf{U}_f; \mathbf{U}_g; \mathbf{U}_o]$. PyTorch also stores separate input-side and recurrent-side bias vectors. This changes the stored parameter count

to $4H(D+H+2)$, although the two bias contributions can be combined in the mathematical formulation above. The quadratic recurrent term explains why increasing the hidden dimension rapidly increases memory and arithmetic cost. In contrast to an MLP with a fixed lag vector, an LSTM learns how information is retained internally and can process sequences of varying duration. This flexibility requires explicit state management during training, validation, streaming evaluation, and embedded deployment. Resetting or misaligning either recurrent state changes the estimator trajectory even when the instantaneous input remains unchanged.

Gated recurrent unit

The gated recurrent unit (GRU) combines memory and exposed output in a single hidden state $\mathbf{h}_t$. An update gate $\mathbf{z}_t$ controls the interpolation between the previous state and a new candidate, while a reset gate $\mathbf{r}_t$ controls how strongly the previous state contributes when that candidate is formed:

$$\begin{aligned}
\mathbf{z}_t &= \sigma(\mathbf{W}_z\mathbf{x}_t + \mathbf{U}_z\mathbf{h}_{t-1} + \mathbf{b}_z)\,, \\
\mathbf{r}_t &= \sigma(\mathbf{W}_r\mathbf{x}_t + \mathbf{U}_r\mathbf{h}_{t-1} + \mathbf{b}_r)\,, \\
\tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}_h\mathbf{x}_t + \mathbf{U}_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h)\,, \\
\mathbf{h}_t &= (1-\mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t.
\end{aligned} \tag{2.16}$$

In this convention, an update-gate value close to one favours the candidate state, whereas a value close to zero retains the previous hidden state. The reset gate acts only on the history used to construct $\tilde{\mathbf{h}}_t$. A small reset value allows the candidate calculation to disregard selected components of the previous state. The symbol $\mathbf{z}_t$ follows conventional GRU notation for the update gate and is unrelated to the dense-layer pre-activation $\mathbf{z}^{(\ell)}$ used in Equation (2.10).

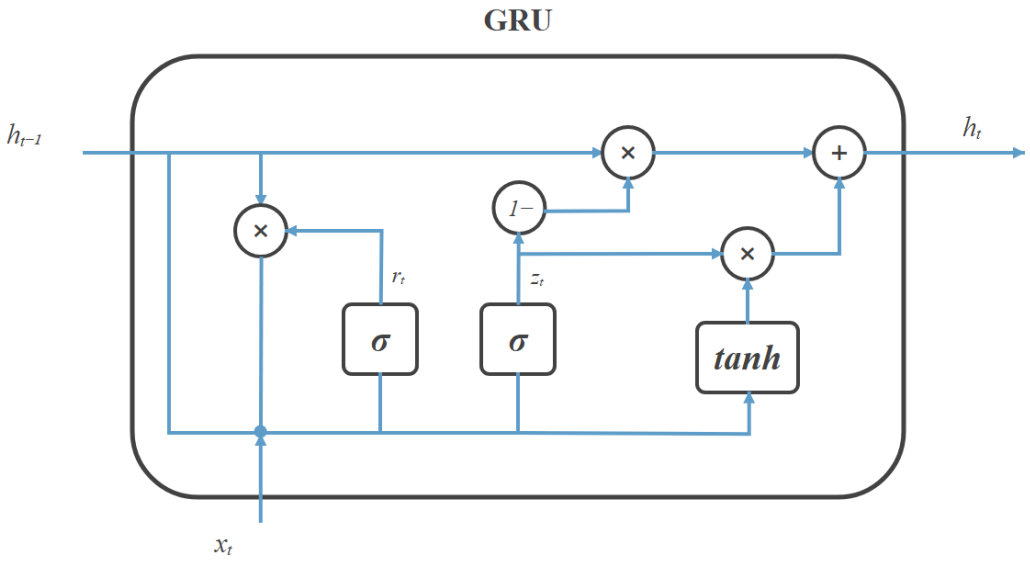


Figure 2.4: Detailed data flow through a GRU cell corresponding to Equation (2.16). The reset gate controls the historical contribution to the candidate state. In the convention used here, $\mathbf{z}_t$ weights the newly computed candidate and $1 - \mathbf{z}_t$ weights the previous hidden state. Unlike the LSTM, the GRU propagates only one recurrent state.

Equation (2.16) and Figure 2.4 use the candidate-weight convention and the reset-before formulation. The update gate therefore weights $\tilde{\mathbf{h}}_t$ directly, and $\mathbf{r}_t$ is applied to $\mathbf{h}_{t-1}$ before the recurrent matrix multiplication. An equally common notation lets the update gate weight the previous state instead. With the complementary gate $\mathbf{z}'_t = 1 - \mathbf{z}_t$, the same update can be written as

$$\mathbf{h}_t = (1 - \mathbf{z}'_t) \odot \tilde{\mathbf{h}}_t + \mathbf{z}'_t \odot \mathbf{h}_{t-1}. \tag{2.17}$$

Implementations may also apply the reset gate either before or after the recurrent affine transformation. These alternatives leave the tensor dimensions unchanged, but the equations, trained parameters, and recurrent states must be interpreted using the same convention.

For input dimension D and hidden dimension H, the update gate, reset gate, and candidate each require one input and one recurrent transformation. With one combined bias per transformation, the parameter count is

$$N_{\text{param}}^{\text{GRU}} = 3H(D + H + 1). \tag{2.18}$$

An implementation that stores separate input-side and recurrent-side biases instead contains $3H(D + H + 2)$ parameters. For identical D and H, a GRU consequently uses approximately three quarters of the recurrent parameters and affine operations of an LSTM. This does not imply a universal accuracy advantage. The LSTM offers a distinct cell-state path and independent forget, write, and output control, whereas the GRU uses a more compact interpolation in one state. The suitable architecture depends on the required memory behaviour, available data, optimization, and deployment constraints.

2.4.4 Comparison with Kalman-filter-based estimation

Neural and Kalman-filter-based estimators solve related state-estimation problems with different assumptions. Table 2.2 summarizes the principal trade-offs. The comparison does not imply that one family is universally superior. A Kalman filter is attractive when a sufficiently accurate model and parameterization are available. A neural network is attractive when a representative dataset can capture nonlinear dependencies that are difficult to express explicitly.

Reported model-based SOH approaches reach low nominal errors when identification and operating conditions are well controlled, while measurement-driven and neural methods avoid some explicit model assumptions but depend more strongly on feature validity and dataset coverage.[30, 34, 35, 36, 37] Recent hybrid and sequence-based methods increasingly combine physical constraints, filtering, and learned correction.[38, 39, 40, 41, 42]

Table 2.2: General strengths and limitations of Kalman-filter-based and neural battery-state estimators. Hybrid methods can combine several of these properties.

Aspect	Kalman-filter-based estimator	Neural estimator
Main prior knowledge	State-space model, parameters, OCV relation, and noise assumptions	Representative measurements, engineered features, and reference labels
Online correction	Explicit residual between predicted and measured voltage	Learned response to the current feature and recurrent history
Interpretability	States and correction equations are physically structured	Internal representation is less directly interpretable
Adaptation challenge	Parameters and covariance tuning must remain valid over temperature and ageing	Training coverage must include the relevant cells and operating conditions
Typical strength	Causal physical propagation with transparent uncertainty handling	Flexible nonlinear approximation and multi-signal fusion
Typical weakness	Model mismatch and weak observability can degrade correction	Domain shift, label quality, and unseen disturbances can degrade generalization
Embedded cost	Usually compact, but depends on model order and matrix operations	Strongly architecture-dependent and may require compression

2.5 Embedded AI and Model Compression

Embedded deployment requires more than transferring a trained model from a workstation framework to a constrained controller. Architecture, numerical representation, storage layout, recurrent-state handling, preprocessing, and execution kernels jointly determine whether the implementation preserves the intended model behaviour while meeting flash, RAM, and timing limits.[43, 10, 11]

This section introduces two complementary families of model compression. The pruning subsection distinguishes removal granularity and structural dependencies, explains criteria and ranking scopes, and discusses when pruning can be integrated into training or applied to an existing model. The quantization subsection describes which quantities can cross a precision boundary, how floating-point ranges are mapped to integer grids, and how range selection, granularity, calibration, arithmetic, and hardware support affect the resulting implementation. These foundations provide the terminology for the application-specific compression methods examined later in Chapter 6.

2.5.1 Pruning

Pruning reduces a neural network by removing parameters or complete model components that contribute comparatively little to its mapping. The underlying assumption is that trained networks commonly contain redundancy and that not every parameter is equally important for the final prediction. Its objective is therefore to identify a smaller subnetwork that retains an acceptable level of predictive accuracy. The design of a pruning procedure requires several separate decisions. These concern the structure to be removed, the criterion used to rank candidates, the scope of the ranking, the point in the training process at which pruning occurs, and the way in which the remaining network is adapted.[44, 45, 46]

Let the parameters $\boldsymbol{\theta}$ be partitioned into K candidate groups $\mathcal{G}_1, \ldots, \mathcal{G}_K$. A binary variable m_k determines whether group $\mathcal{G}_k$ remains active. The general constrained problem can be expressed as

$$\underset{\boldsymbol{\theta}, \boldsymbol{m}}{\text{minimize}} \quad \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{m} \mid \mathcal{D}) \quad \text{subject to} \quad m_k \in \{0, 1\}, \qquad \sum_{k=1}^{K} m_k \leq K', \tag{2.19}$$

where $\mathcal{L}$ is the task loss on data $\mathcal{D}$ and K' is the permitted number of retained groups. The pruning fraction is $p = 1 - K'/K$, while $1 - p$ is the retained density.

This formulation does not prescribe how the discrete selection is solved. Practical methods replace the combinatorial search by ranking rules, continuous mask approximations, regularization, or repeated removal and retraining.

Parameter count, stored model size, arithmetic effort, and measured inference time remain distinct quantities. A high pruning fraction does not automatically produce an equivalent hardware speed-up because the practical benefit depends on the removed structure, its representation, the execution kernels, and memory traffic. This distinction is especially important for embedded targets, where a mathematically sparse model may still be executed as a dense model.

Pruning structures and granularity

The most direct classification concerns the granularity of the removed object. Unstructured pruning removes individual connections. For a weight matrix W, this operation can be represented by a binary mask M,

$$\widetilde{W}_{\text{unstructured}} = M \odot W, \qquad M_{ij} \in \{0, 1\}, \tag{2.20}$$

where a zero entry in M suppresses the corresponding weight. This fine granularity offers many possible pruning patterns and can produce high numerical sparsity while preserving important connections. However, the dimensions of $\widetilde{W}_{\text{unstructured}}$ remain identical to those of W. Standard dense kernels still execute the zero-valued entries. A reduction in storage or arithmetic therefore requires a sparse representation, index metadata, and execution kernels that can exploit the resulting irregular access pattern. At moderate sparsity, this overhead can offset part of the theoretical benefit.[12, 44, 45]

Structured pruning removes complete groups such as neurons, channels, filters, attention heads, or recurrent states. If $\mathcal{I}_{\text{out}}$ and $\mathcal{I}_{\text{in}}$ denote the retained output and input indices, the resulting matrix is

$$\widetilde{W}_{\text{structured}} = W[\mathcal{I}_{\text{out}}, \mathcal{I}_{\text{in}}]. \tag{2.21}$$

The tensor dimensions are reduced and the remaining values form a smaller dense matrix. Conventional dense kernels can therefore benefit directly, and the reduction can extend to intermediate activations and recurrent states. The coarser granularity provides fewer possible pruning choices than element-wise removal and can cause a larger accuracy loss if an important group is removed. Connected layers must

also be modified consistently because removing one output component changes the corresponding input dimension of the following operation.[47, 48, 46]

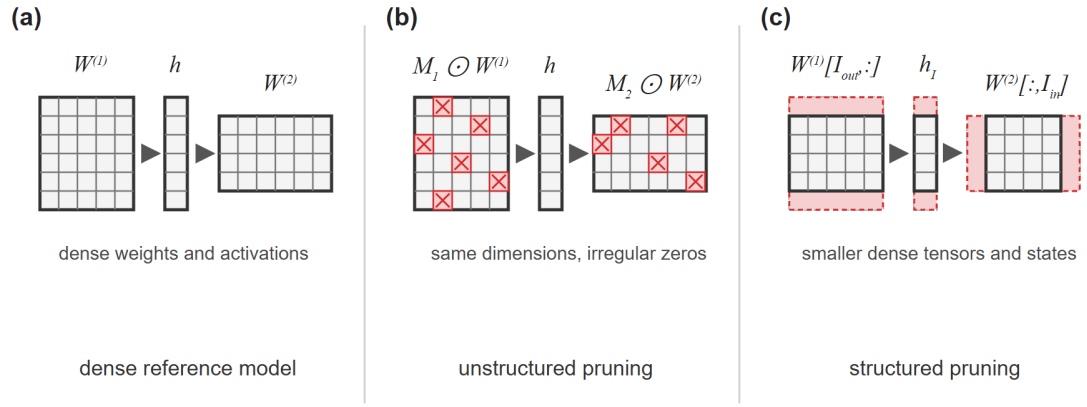


Figure 2.5: Comparison of pruning granularities. Panel (a) shows two dense weight matrices connected through an intermediate representation. In panel (b), unstructured pruning suppresses individual weights while retaining the original tensor dimensions. Panel (c) illustrates structured pruning, where complete output structures and the corresponding inputs of the following layer are removed. The resulting tensors and intermediate representation are smaller but remain dense.

Table 2.3 summarizes the practical trade-off. Unstructured pruning provides greater flexibility in selecting individual connections, while structured pruning more directly translates the removal decision into conventional embedded memory and compute reductions. Neither strategy guarantees a proportional reduction in measured latency because the realized gain still depends on storage format, kernel implementation, memory traffic, and the target processor.

Table 2.3: Practical advantages and limitations of unstructured and structured pruning.

Aspect	Unstructured pruning	Structured pruning
Removed object	Individual weights or connections	Complete neurons, channels, filters, or recurrent states
Granularity	Fine and highly flexible	Coarser and constrained by the network architecture
Resulting tensors	Original dimensions with irregular zero entries	Reduced dimensions with dense retained values
Main advantage	High numerical sparsity can be reached without removing complete functional groups	Parameter memory, intermediate states, and operation counts can be reduced using standard dense representations
Main limitation	Practical acceleration requires sparse storage and suitable sparse kernels	Coupled layers must be changed consistently and coarse removal can affect accuracy more strongly
Embedded suitability	Hardware- and sparsity-format-dependent	Generally easier to map to predictable dense execution

Semi-structured pruning occupies an intermediate position by enforcing small regular sparsity patterns, such as retaining a fixed number of weights within each block. A common example is an N:M pattern in which exactly N of every M weights remain active. It sacrifices some of the freedom of unstructured pruning to obtain predictable local patterns. Practical acceleration still requires a kernel and target processor that support the selected pattern.[45, 49]

Architectural dependencies. Structured pruning changes tensor dimensions and therefore cannot always be applied independently to every layer or path. In a sequential network, removing outputs of layer i requires removal of the corresponding inputs of layer $i + 1$. Parallel and residual paths impose an additional constraint. For

$$\boldsymbol{y} = f_A(\boldsymbol{x}) + f_B(\boldsymbol{x}), \tag{2.22}$$

the two outputs must have compatible dimensions and coordinate semantics. If the branches are independently reduced to $\mathbb{R}^{H'_A}$ and $\mathbb{R}^{H'_B}$ with $H'_A \neq H'_B$, the addition is not defined. Even if both branches retain the same number of components, independently selected channel identities may no longer correspond. Safe structured pruning therefore coordinates retained index sets across dependent paths or inserts an explicit projection that restores compatibility.

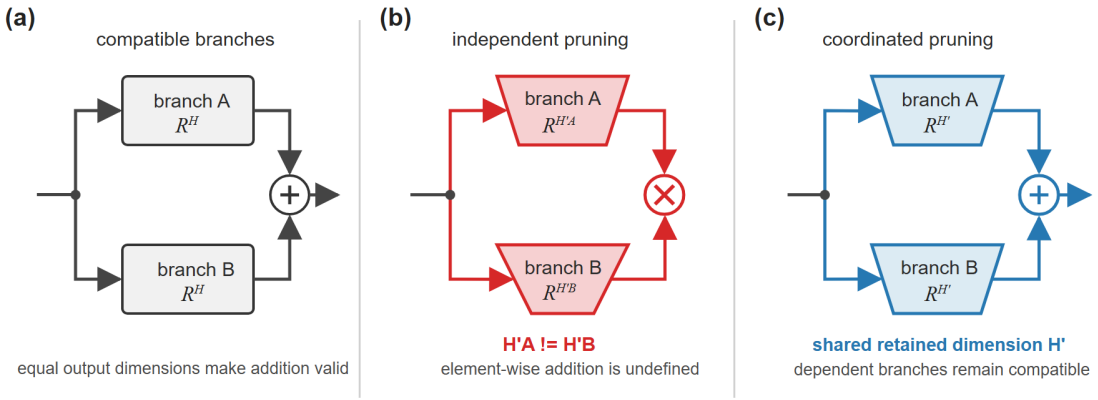


Figure 2.6: Dependency constraints in structured pruning. Panel (a) shows two compatible branches whose outputs can be added. Panel (b) illustrates an invalid result when the branches are independently reduced to different output dimensions. Panel (c) uses a shared retained structure so that both branches remain compatible. Similar consistency constraints apply between successive layers and between the coupled gates of recurrent networks.

Figure 2.6 illustrates why a structured pruning decision is not solely a local ranking problem. The removable unit is defined by the computation graph. For recurrent architectures, one hidden state participates in several gate matrices, the recurrent state vector, and the input of a subsequent output head. Treating these occurrences as one group preserves a valid recurrent model after removal. This architectural coupling is revisited for the LSTM estimator in Section 6.4.

Pruning criteria and ranking scope

After the pruning structure has been defined, an importance score is required for every candidate group. The score estimates the expected effect of removing the candidate. No inexpensive score can prove global optimality because parameters interact and the loss changes after each removal. Criteria therefore trade computational cost against task-specific information.

Magnitude criteria. The simplest criterion assumes that parameters with small magnitudes contribute less to the network mapping. For an individual weight, $s_{ij} = |w_{ij}|$. For a structured group $\mathcal{G}_k$, the same idea can be extended through a grouped norm,

$$s_k^{\text{mag}} = \|\boldsymbol{\theta}_{\mathcal{G}_k}\|_q = \left(\sum_{i \in \mathcal{G}_k} |\theta_i|^q \right)^{1/q}, \qquad q \in \{1, 2\}. \tag{2.23}$$

The L_1 norm sums absolute magnitudes, while the L_2 norm gives greater relative influence to large entries. Magnitude criteria require neither labels nor additional forward or backward passes after training. Their limitation is that parameter scale is not always directly comparable across layers or operations. A small parameter can still be influential when it multiplies a large activation or interacts with other weights.[12, 50, 45]

Loss-sensitivity criteria. Gradient-based criteria use the task loss to approximate the consequence of removal. Setting all parameters of group $\mathcal{G}_k$ to zero changes them by $\Delta\theta_i = -\theta_i$. A first-order Taylor approximation gives

$$\Delta\mathcal{L}_k \approx -\sum_{i \in \mathcal{G}_k} \frac{\partial \mathcal{L}}{\partial \theta_i} \theta_i, \qquad s_k^{\text{Taylor}} = \left| \sum_{i \in \mathcal{G}_k} \frac{\partial \mathcal{L}}{\partial \theta_i} \theta_i \right|. \tag{2.24}$$

This score combines the parameter value with current loss sensitivity and therefore requires representative data and a backward pass. Second-order methods additionally use curvature. For an individual parameter near a stationary point, where the first-order gradient is small, a diagonal approximation gives

$$\Delta\mathcal{L}_i \approx \frac{1}{2} H_{ii} \theta_i^2, \tag{2.25}$$

where H_{ii} is a diagonal entry of the loss Hessian. This Hessian-based approximation was used in early second-order pruning research to rank parameters by their estimated contribution to the loss. Curvature can provide richer information, but computing or approximating it is substantially more expensive than a grouped weight norm.[51, 52]

Activation and learned-mask criteria. Activation-based criteria measure how strongly a neuron, channel, or recurrent state responds over representative sam-

ples. Reconstruction criteria instead select structures whose removal changes an intermediate or final output as little as possible. Both approaches include information about the data distribution, but their result depends on the calibration set. A further class introduces trainable gates or masks and optimizes them jointly with the network. Sparsity penalties then encourage gates or complete parameter groups to approach zero. Examples include L_0-motivated gate regularization and group penalties for structured sparsity.[53, 47, 48]

Local and global ranking. The ranking scope determines how scores are converted into a retained set. Local pruning assigns a target K'_ℓ to each layer ℓ and ranks only within that layer,

$$\mathcal{R}_\ell = \mathrm{TopK}_{K'_\ell}\left(\{s_{\ell,k}\}_{k=1}^{K_\ell}\right). \tag{2.26}$$

It provides direct control over the width of each layer and prevents an entire sensitive or small layer from disappearing. However, the layer-wise pruning fractions must be selected in advance and may retain weak groups in one layer while discarding stronger groups in another.

Global pruning combines all candidates in one ranking,

$$\mathcal{R} = \mathrm{TopK}_{K'}\left(\bigcup_\ell \{s_{\ell,k}\}_{k=1}^{K_\ell}\right). \tag{2.27}$$

This allows the remaining capacity to be distributed according to the scores instead of fixed layer quotas. It can also over-prune a layer when raw scores are not comparable across layers. Normalization, minimum retained widths, and dependency constraints are therefore common safeguards. Both local and global selection are ranking heuristics rather than proofs that the resulting subnetwork is optimal.[44, 45]

Pruning schedules and optimization strategies

The classic pruning workflow trains a dense model, removes low-ranked parameters or groups, and fine-tunes the retained network,

$$\boldsymbol{\theta}_0 \xrightarrow{\text{train}} \boldsymbol{\theta}^* \xrightarrow{\text{score and prune}} (\boldsymbol{m}, \boldsymbol{\theta}^*) \xrightarrow{\text{fine-tune}} \widehat{\boldsymbol{\theta}}_{\boldsymbol{m}}. \tag{2.28}$$

The first training phase supplies task-adapted parameters from which importance is estimated. Pruning produces an immediate perturbation of the learned mapping.

Fine-tuning then adapts the surviving parameters to the reduced model capacity. This train–prune–fine-tune framework remains a strong practical baseline because its stages are interpretable and can be applied to an existing trained model.[12]

One-shot pruning performs the removal once at the target pruning fraction. It is computationally economical and reproducible, but a large single change may be difficult to recover from. Iterative pruning removes smaller fractions over several rounds and alternates them with optimization. The gradual process updates the ranking after the network has adapted, but requires more training and introduces additional choices for the number of rounds and the pruning schedule. Recovery can use conventional fine-tuning with a reduced learning rate, weight rewinding to an earlier training state, or learning-rate rewinding while retaining the final surviving weights.[54, 55]

Pruning is not restricted to a fully trained network. The principal strategy families can be distinguished by when and how the sparse structure is selected:

Pruning at initialization seeks an informative mask before ordinary weight training. SNIP, for example, ranks connections by their loss sensitivity at initialization and removes them in a single step.[56] A related hypothesis is that a randomly initialized dense network may already contain a sparse subnetwork that can be trained to comparable accuracy when its original initialization is retained.[54] These approaches address the cost of first training a dense model, but they solve a different problem from compressing a specific trained estimator whose behaviour is already known.

Dynamic sparse training begins with a sparse topology and changes the connectivity during optimization. Sparse evolutionary training removes low-magnitude connections and introduces new ones while maintaining a sparse parameter budget.[57] RigL combines magnitude-based removal with gradient information for regrowth and keeps the training cost tied to the selected density.[58] Such methods can reduce the cost of training itself. Their practical benefit depends on efficient sparse forward and backward kernels, which are not automatically available on all training platforms.

Learned-mask and penalty-based methods incorporate structure selection into the optimization objective. A generic regularized form is

$$\underset{\boldsymbol{\theta},\,\boldsymbol{\alpha}}{\text{minimize}} \quad \mathcal{L}(\boldsymbol{\theta} \odot g(\boldsymbol{\alpha})) + \lambda \sum_{k=1}^{K} \Omega(\alpha_k)\,, \tag{2.29}$$

Table 2.4: Principal strategies for obtaining a pruned or sparse neural network. The categories describe different optimization procedures and do not imply identical deployment representations.

Strategy	Structure selected	Main principle	Principal trade-off
Post-training pruning	After dense training	Rank trained parameters, remove candidates, and fine-tune or rewind	Reuses a trained model, but dense pretraining cost remains
Pruning at initialization	Before ordinary training	Estimate connection sensitivity or search for a trainable initial subnetwork	Avoids dense training of the final mask, but reliable early importance estimation is difficult
Gradual pruning during training	Across a prescribed schedule	Increase sparsity while weights continue adapting	Smoother transition, but schedule and final density become training hyperparameters
Dynamic sparse training	Throughout sparse training	Maintain a sparse budget while repeatedly removing and regrowing connections	Can reduce dense training cost, but requires sparse training support and topology updates
Learned masks or gates	Jointly with the weights	Optimize continuous gates and sparsity penalties, then discretize the result	Data-driven selection, but adds optimization variables and regularization choices
Penalty-based structured learning	During training	Apply group penalties so complete structures approach zero	Couples structure discovery with training, but penalty strength controls the accuracy–sparsity balance

where $g(\boldsymbol{\alpha})$ represents differentiable gates and Ω encourages sparsity. Group penalties can be applied to complete rows, columns, channels, or recurrent states so that deployment-relevant structures vanish together.[53, 47] Compared with a fixed post-training criterion, these methods allow importance and weights to co-adapt. They also introduce an additional regularization path and can require thresholding or model reconstruction after optimization.

The pruning fraction remains a design parameter rather than a universal property of a network. Increasing it strengthens the potential resource reduction but raises the risk of removing information that cannot be recovered. The selected model must therefore be assessed in terms of predictive performance and deployment-relevant quantities such as parameter count, persistent storage, intermediate-state memory, operation count, and measured inference time.

Software frameworks and deployment implications

Software libraries support different portions of the pruning workflow. Table 2.5 lists representative options. Their APIs simplify mask creation, scheduling, or comparison, but a pruning mask alone does not guarantee a physically smaller exported network.

PyTorch provides reusable pruning parametrizations and ranking utilities, while TensorFlow Model Optimization provides scheduled magnitude pruning and conversion paths for deployment.[59, 60, 49] Neural Network Intelligence exposes multiple pruning algorithms and separates mask generation from graph-level model speed-up.[61] ShrinkBench focuses on reproducible evaluation across pruning methods and models.[44]

For structured pruning, explicit model surgery is often required even when a framework can identify candidate groups. The retained rows and columns must be copied into newly dimensioned layers, coupled paths must share compatible indices, and downstream layers must be adapted. This produces a compact dense model that can use conventional kernels without storing masks or sparse indices. The specific one-shot, magnitude-based LSTM procedure used in this work and its custom PyTorch reconstruction are described in Section 6.4.

Table 2.5: Representative pruning support in common software ecosystems. Capabilities refer to the documented pruning workflow and must be matched to the target operator and deployment backend.

Framework	Main pruning support	Useful role	Deployment consideration
PyTorch pruning utilities	Mask-based local and global pruning with unstructured and structured criteria	Rapid implementation of standard criteria and custom pruning methods	Masks preserve the original module dimensions until explicit model reconstruction or sparse execution is applied
TensorFlow Model Optimization Toolkit	Magnitude pruning with schedules, Keras integration, model stripping, and supported structured sparsity patterns	Training-time pruning and export into the TensorFlow Lite ecosystem	Compression and acceleration depend on the exported representation and backend support
Neural Network Intelligence	Collection of pruners, evaluators, masks, and model-speedup utilities	Comparative experimentation and dependency-aware compression workflows	Supported operators and graph transformations must be checked for the chosen architecture
ShrinkBench	Standardized pruning experiments, metrics, and reproducible baselines	Method comparison and research evaluation	It is primarily an evaluation framework rather than an embedded inference runtime

2.5.2 Quantization

Quantization reduces the numerical precision used to store or process selected quantities in a neural network. Instead of retaining every value in a high-precision floating-point format, a quantizer maps it to one of a finite number of representable levels. This can reduce persistent model storage, memory traffic, intermediate-state memory, and arithmetic cost. The realized benefit depends on which tensors are quantized and whether the target processor provides kernels that operate directly on the selected low-precision format. Quantization is therefore both a numerical approximation problem and a model–hardware interface problem.[62, 63]

Quantizable quantities and precision boundaries

A neural layer combines stored parameters and dynamic tensors. For a dense neuron, the unquantized calculation is

$$z_j = \sum_{i=1}^{m} w_{ji} x_i + b_j, \qquad y_j = f(z_j), \tag{2.30}$$

where w_{ji} denotes a weight, b_j a bias, x_i an input activation, z_j the pre-activation, and y_j the output activation. This is the same neuron mapping introduced in Equation (2.9). Its quantities do not have to share one numerical format. The following precision boundaries are commonly distinguished:

- **Weights** are static after training and often account for most of the persistent model storage. Weight-only quantization can therefore reduce flash or file size without changing the representation of dynamic activations.
- **Biases** are stored parameters but are commonly retained at higher precision. In integer-only kernels they can be represented as higher-width integers matched to the product of the weight and activation scales.
- **Input and output activations** are generated at runtime. Their quantization can reduce intermediate memory and enables low-precision matrix operations, but their ranges depend on the input data and may require calibration or dynamic scale estimation.
- **Accumulators** collect many products. They normally use a wider type, such as INT32 for INT8 operands, to avoid overflow before the result is requantized.

- **Recurrent states** are dynamic activations that are fed back into the next update. Quantizing them changes not only one layer output but also the numerical state from which later predictions are computed.

The nonlinear function f is an operation rather than a stored tensor. Its input and output may be quantized, while the operation itself may be evaluated through integer arithmetic, a lookup table, a polynomial approximation, or a floating-point routine. Consequently, the label "quantized network" is incomplete unless the precision of weights, biases, activations, states, accumulators, and arithmetic is stated explicitly.

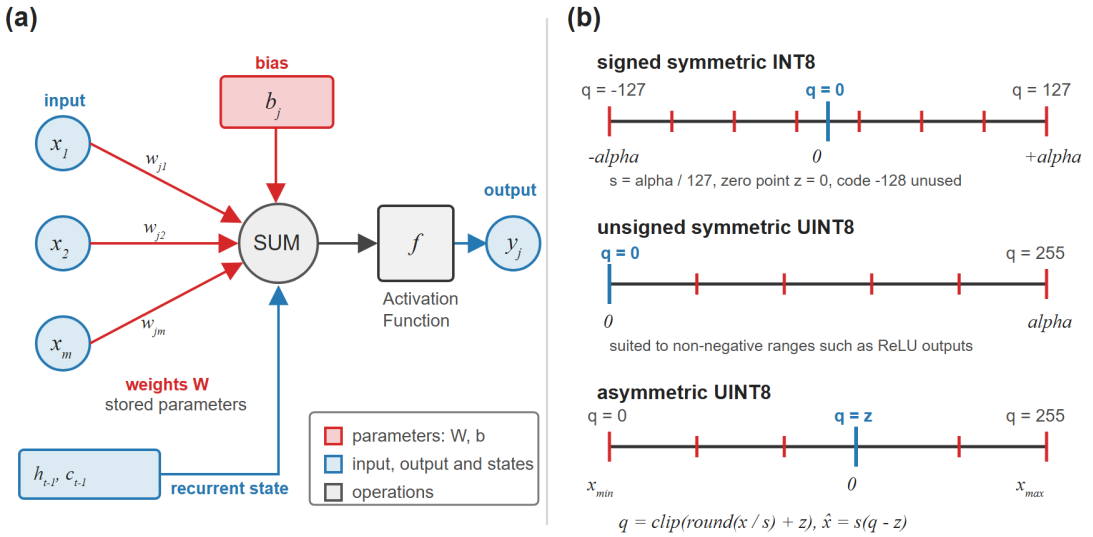


Figure 2.7: Quantizable quantities and representative uniform 8-bit grids. Panel (a) distinguishes stored weights and biases, dynamic inputs, outputs and recurrent states, and the summation and activation operations of a neural computation. Panel (b) compares signed symmetric INT8, unsigned symmetric UINT8 for non-negative values, and asymmetric UINT8 with a non-zero zero point. The signed symmetric example uses the exactly balanced code range -127 to 127 and leaves the INT8 code -128 unused.

Figure 2.7 illustrates that quantization can be applied at several different locations. If tensor k contains N_k values represented by b_k bits, the ideal raw storage requirement is

$$M_{\text{raw}} = \sum_{k=1}^{K} \frac{N_k b_k}{8} + M_{\text{meta}}, \tag{2.31}$$

where M_{meta} contains scales, zero points, tensor descriptors, alignment, and any

retained higher-precision parameters. Converting one FP32 tensor to INT8 ideally reduces the raw storage of that tensor by a factor of four. It does not imply a fourfold reduction of the complete model if other parameters, metadata, or code remain unchanged.

Uniform affine quantization

Uniform affine quantization places representable values on an evenly spaced grid. For a real value x, bit width b, scale $s > 0$, zero point z, and integer limits $q_{\min}$ and $q_{\max}$, quantization and reconstruction are defined as

$$q = \operatorname{clip}\left(\operatorname{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right), \qquad \widehat{x} = s(q - z). \tag{2.32}$$

The scale defines the distance between neighbouring reconstruction levels. The integer zero point ensures that real zero can be represented exactly, which is important for zero padding, inactive activations, and sparse values.[13, 62]

For an asymmetric quantizer that maps a real interval $[x_{\min}, x_{\max}]$ to the complete integer range, the parameters can be selected as

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \tag{2.33}$$

$$z = \operatorname{clip}\left(\operatorname{round}\left(q_{\min} - \frac{x_{\min}}{s}\right), q_{\min}, q_{\max}\right). \tag{2.34}$$

The offset permits efficient use of the available codes when the value range is not centred on zero. The correction terms associated with a non-zero zero point can, however, increase kernel complexity.

Symmetric quantization sets $z = 0$ and chooses equal positive and negative limits. For signed b-bit quantization,

$$q_{\max} = 2^{b-1} - 1, \qquad q_{\min} = -q_{\max}, \qquad s = \frac{\alpha}{q_{\max}}, \tag{2.35}$$

where $\alpha = \max_i |x_i|$ if the complete observed range is retained. At $b = 8$, this produces the symmetric set $\{-127, \ldots, 0, \ldots, 127\}$. The underlying signed INT8 type also contains -128, but leaving this code unused gives equal-magnitude positive and negative limits. This convention is not mandatory. Other implementations use the full two's-complement interval and account for its one-code asymmetry.

Unsigned symmetric quantization is useful when the represented quantity is

known to be non-negative. For UINT8, $q_{\min} = 0$, $q_{\max} = 255$, and $z = 0$. A ReLU output is a common example. Signed symmetric quantization is more natural for weights that are approximately centred around zero. Asymmetric quantization is useful when the range includes zero but is shifted or strongly unbalanced.

Rounding, clipping, and range selection

Quantization introduces two principal numerical errors. Values inside the selected interval are rounded to the nearest grid point. With nearest-integer rounding and no clipping,

$$|x - \widehat{x}| \leq \frac{s}{2}. \tag{2.36}$$

Values outside the interval are clipped to its lower or upper boundary and can exceed this bound. Expanding the interval reduces clipping but increases s, which makes the grid coarser and increases rounding error. Narrowing the interval has the opposite effect. Range selection therefore balances clipping of rare extremes against resolution in the densely populated part of the distribution.[62, 64]

The simplest choice uses the observed minimum and maximum. It avoids clipping of the calibration values but is sensitive to outliers. Percentile-based calibration deliberately excludes a small tail of the distribution. Error-based methods select lower and upper thresholds by minimizing a local reconstruction objective such as

$$\underset{\ell,u}{\text{minimize}} \quad \frac{1}{N} \sum_{i=1}^{N} \left(x_i - \widehat{x}_i(\ell, u)\right)^2 . \tag{2.37}$$

Entropy-based criteria compare the original and quantized distributions. These local objectives do not necessarily predict the final task error because sensitivity differs between layers. More advanced post-training methods therefore include bias correction, cross-layer equalization, reconstruction losses, or learned rounding decisions.[65, 62]

Weights are fixed after training, so their ranges can be determined directly from the stored tensors. Activation ranges depend on the input data. Static activation quantization collects representative statistics before deployment and stores fixed quantization parameters. Dynamic quantization calculates activation scales during inference. Dynamic ranges can adapt to each input or time window but add runtime operations and may complicate deterministic timing.

Quantization granularity

The scale and zero point can be shared by different portions of a tensor. Granularity controls the compromise between representation accuracy, metadata, and kernel complexity:

Table 2.6: Principal quantization granularities and their implementation consequences. The term channel includes a matrix output row when each row produces one output component.

Granularity	Shared parameters	Main advantage	Main consequence
Per-tensor	One scale and zero point for a complete tensor	Minimal metadata and simple kernels	Channels with small ranges may use only a small part of the grid
Per-channel or per-row	One parameter set for every output channel or matrix row	Adapts to different output ranges and often reduces weight error	Requires multiple scales and channel-aware reconstruction or requantization
Per-group or per-block	One parameter set for a defined group of values	Intermediate compromise between local adaptation and metadata	Group shape must match packing and kernel support
Mixed precision	Different bit widths or formats for selected tensors and operators	Protects sensitive quantities while compressing less sensitive ones	Requires a deployment backend that supports every selected combination

Per-tensor quantization is straightforward because one scale can be factored out of a complete matrix operation. Per-channel weight quantization uses a separate scale for every output component. It improves the representation when rows have different magnitudes and is compatible with many integer kernels because each accumulator output can be rescaled independently. Per-channel activation quantization is more difficult because input-channel scales occur inside the summation. Per-group methods provide finer adaptation but increase the number of parameters and can require specialized packing.[66, 62]

Integer arithmetic and requantization

A reduced storage type does not by itself define the arithmetic path. Consider quantized weights and input activations with parameters (s_w, z_w) and (s_x, z_x). An integer matrix operation forms

$$A_j^{\text{int}} = q_{b,j} + \sum_i \left(q_{w,ji} - z_w\right)\left(q_{x,i} - z_x\right), \tag{2.38}$$

where A_j^{int} is commonly accumulated in INT32. The corresponding real-valued pre-activation is approximately

$$a_j \approx s_w s_x A_j^{\text{int}}. \tag{2.39}$$

If the next activation uses parameters (s_y, z_y), the accumulator is requantized according to

$$q_{y,j} = \text{clip}\left(\text{round}\left(\frac{s_w s_x}{s_y} A_j^{\text{int}}\right) + z_y, q_{y,\min}, q_{y,\max}\right). \tag{2.40}$$

The bias can be stored as a higher-width integer with scale $s_b = s_w s_x$. With per-channel weight scales, s_w and s_b become output-dependent. Integer-only inference keeps the multiplication, accumulation, bias addition, and requantization in integer-compatible forms. Quantization-aware model export must preserve these scale relationships.[13]

Lower bit widths can reduce data movement and permit more parallel arithmetic operations, but these gains require suitable instructions and kernels.[67] A weight-only model that converts each integer weight back to floating point inside the innermost loop reduces persistent storage but may add conversion and scaling operations. Likewise, quantize and dequantize boundaries between unsupported operators can remove an expected speed advantage. Runtime, RAM, and energy improvements must therefore be measured on the selected backend.

Post-training and training-aware strategies

Quantization approaches can be organized by the point at which the numerical constraints enter model development:

Post-training quantization starts from a trained floating-point model. Weight-only PTQ requires no activation data. Static full-integer PTQ normally requires a representative calibration set to estimate activation ranges. Dynamic PTQ evalu-

Table 2.7: Principal quantization strategies and the information required to construct the deployed model. PTQ denotes post-training quantization and QAT denotes quantization-aware training.

Strategy	Quantized quantities	Main procedure	Principal trade-off
Weight-only PTQ	Stored weights, with other quantities retained at higher precision	Quantize a trained model directly from its weight tensors	Simple and effective for storage, but does not provide complete integer arithmetic
Static PTQ	Weights and selected activations	Use representative calibration data to fix activation ranges before deployment	Low inference overhead, but quality depends on representative ranges
Dynamic PTQ	Weights and activations with runtime activation scales	Determine dynamic tensor ranges during inference	Adapts to changing inputs, but adds scale-estimation overhead
Quantization-aware training	Simulated low-precision weights and activations during training or fine-tuning	Insert fake quantizers so the model adapts to the numerical grid	Can preserve accuracy at lower bit widths, but requires training data and additional optimization
Quantized training	Selected forward, backward, gradient, or optimizer quantities	Perform parts of the training computation itself at low precision	Can reduce training cost, but is distinct from inference-only compression and requires stable low-precision optimization

ates selected activation parameters at runtime. PTQ is attractive when the original training pipeline is expensive or unavailable, but aggressive bit-width reduction can exceed the error that the fixed parameters can tolerate.[64, 65]

Quantization-aware training inserts simulated quantizers into the forward path. A fake quantizer applies quantization followed by reconstruction while retaining floating-point tensors for gradient-based optimization,

$$x_{\mathrm{fq}} = s\left[\mathrm{clip}\Big(\mathrm{round}\Big(\frac{x}{s}\Big) + z, q_{\min}, q_{\max}\Big) - z\right]. \tag{2.41}$$

The derivative of the rounding operation is zero or undefined, so QAT commonly uses a straight-through estimator that approximates the gradient through the quantizer. The weights can then adapt to quantization noise, clipping thresholds, and learned step sizes. This generally increases development effort but can be important for low-bit weights and activations.[13, 68, 62]

Numerical formats and recurrent networks

Integer quantization is only one form of reduced precision. FP16 and bfloat16 retain floating-point exponents and are often used where floating-point accelerators are available. INT8 uses a fixed affine interpretation supplied by scales and zero points. Four-bit, ternary, and binary representations can compress more strongly but provide fewer levels and place greater demands on training methods and specialized kernels. Power-of-two quantization restricts s to 2^k so that rescaling can be implemented through shifts, at the cost of a less flexible grid. Mixed-precision schemes assign different formats or bit widths to tensors according to sensitivity and backend support.[63, 62]

Recurrent networks require particular care because the hidden and cell states connect successive time steps. Weight quantization introduces a fixed perturbation to every recurrent update. Activation or state quantization additionally feeds a quantized dynamic state back into the next update. This does not imply that error must grow without bound because gates and nonlinearities can attenuate perturbations. It does mean that a single-step comparison is insufficient to establish long-horizon equivalence. Continuous replay without state resets is required to detect systematic drift, state-dependent error, or isolated deviations.

Weight-only recurrent quantization is a conservative precision boundary. It targets persistent matrix storage while preserving activation functions and recurrent states at higher precision. Full-integer recurrent inference can reduce arithmetic

and state memory further, but requires compatible implementations of gate accumulation, sigmoid and hyperbolic-tangent functions, cell-state updates, and requantization. The best choice therefore depends on whether flash, RAM, latency, or energy is the active constraint.

Software support and deployment

Software frameworks cover different parts of the quantization workflow. Their high-level options are useful for model preparation, but the exported numerical graph and the target kernels remain decisive:

Table 2.8: Representative software support for quantization. Exact operator coverage, data types, and acceleration depend on the selected release and deployment backend.

Framework	Main support	Useful role	Deployment consideration
PyTorch and torchao	Weight-only, static, dynamic, mixed-precision, and QAT workflows	Model-side experimentation and fake-quantized training	Efficient execution depends on exported operators and available low-precision kernels
TensorFlow Model Optimization and Lite	PTQ, representative calibration, QAT, and integer-model conversion	End-to-end conversion into the TensorFlow Lite ecosystem	Full-integer acceleration requires supported operators and a representative calibration set
ONNX Runtime	Static and dynamic PTQ, QDQ and quantized operators, calibration, and debugging	Framework-neutral conversion and tensor-level comparison	Execution-provider support determines the useful formats and operators
STM32Cube.AI and custom C	Vendor conversion or explicit parameter export into microcontroller firmware	Integration with the final embedded application and direct measurement	Unsupported recurrent paths can require custom kernels and explicit scale handling

Current PyTorch tooling distinguishes weight-only, dynamic, static, and QAT workflows, while TensorFlow Lite provides post-training and training-aware con-

version paths.[69, 70] ONNX Runtime represents quantized graphs through quantized operators or explicit quantize–dequantize nodes and provides static and dynamic calibration workflows.[71] Vendor tools can shorten microcontroller integration when the architecture and operators are supported. A custom implementation provides a transparent precision boundary but places responsibility for scaling, accumulation width, state handling, and validation on the developer.

The application in Chapter 6 instantiates only one part of this design space. It uses symmetric per-row INT8 post-training quantization for the recurrent weight matrices and retains biases, activations, recurrent states, the MLP head, and arithmetic in FP32. The exact transformation and its manual C export are described in Section 6.5. This separation keeps the present subsection general and leaves the application-specific numerical boundary with the experimental method.

2.5.3 Model-hardware co-design

Neural SOC and SOH estimators have been demonstrated on microcontroller-class hardware and dedicated BMS-oriented edge devices.[72, 73, 74, 75] Newer controller families increasingly integrate neural-processing accelerators, but compact recurrent estimators can also be executed on general-purpose Arm Cortex-M cores. The important design question is not whether an accelerator exists, but whether the complete estimator path meets the application requirements with reproducible numerical behaviour.

2.6 Research Gap and Contribution Scope

The state of research is uneven across the three requirement dimensions introduced in Figure 1.1. Practical BMS measurement and protection hardware is mature. Model-based and data-driven estimators can both reach low errors in controlled studies. Embedded demonstrations show that neural estimators can run on microcontrollers. These results are often reported separately, however, which makes it difficult to connect model accuracy, disturbed-operation behaviour, and measured embedded execution.

The first gap concerns representation. A complex history-dependent battery state does not necessarily require a large architecture if the input representation makes the relevant history accessible. The second gap concerns evaluation. Clean-condition error alone does not reveal how direct, hybrid, model-based, and data-

driven estimators react to the same sensor and timing disturbances. The third gap concerns deployment. Parameter compression has to be traced through model export, stateful streaming execution, linked memory, and measured runtime before its practical benefit is known.

The three contribution chapters address these gaps in sequence. Chapter 4 studies temporal representation for a compact MLP. Chapter 5 compares estimator families under a shared measurement-only disturbance protocol. Chapter 6 connects structured pruning and weight quantization to continuous recurrent inference and measured microcontroller behaviour.

Chapter 3

Experimental Basis and Hardware Ecosystem

The experimental basis of this work combines laboratory battery datasets, embedded execution targets, and the Custom BMS Platform for battery management. This combined setup makes it possible to study battery-related artificial intelligence not only as an offline modelling problem, but also as an embedded systems problem shaped by sensing, communication, firmware partitioning, and hardware constraints. The resulting experimental ecosystem forms the common basis for the ageing-estimation work, the robustness benchmark, and the embedded implementation studies.

Part of this experimental work was conducted within MG-FARM, which develops smart renewable-energy microgrids for the electrification of agricultural farms.[76, 77, 78] The project was funded through the LEAP-RE Joint Call 2021 for collaborative African–European research and innovation on renewable energy.[79] The overarching LEAP-RE programme received European Union Horizon 2020 funding under Grant Agreement No. 963530.[80] The Technische Universität Berlin subproject was funded by the German Federal Ministry of Education and Research under grant No. 03SF0684A.[81, 1]

3.1 Data Sources and Ageing Campaigns

Two battery datasets with different cell chemistries, ageing protocols, and application contexts are used. The first dataset uses NMC cells and supports the ageing-estimation study in Chapter 4. The second uses LFP cells and forms the experimental basis for both the robustness benchmark in Chapter 5 and the embedded SOC/SOH study in Chapter 6. Across both datasets, the directly measured quantities are voltage, current, temperature, and time. Dataset-side processing derives

the SOC and SOH labels used for training and evaluation. The contribution chapters then construct their own causal features from these measurements. Table 3.1 compares the two campaigns.

Table 3.1: Key properties of the two battery datasets used in this work. The NMC dataset supports the SOH ageing-estimation study in Chapter 4, while the LFP dataset provides the basis for Chapters 5 and 6.

Property	NMC Dataset	LFP Dataset
Cell type	INR18650 NMC	JGCFR18650 LFP
Nominal capacity	2,6 A h	1,8 A h
Nominal voltage	3,7 V	3,2 V
Number of cells	14	15
Campaign type	Cyclic ageing	DoE cyclic ageing
Varied parameters	$\mathrm{SOC}_{\mathrm{mean}}$, DOD, C-rate, temperature	C_{chg}, C_{dis}, DOD
Temperature	35 °C / 45 °C	25 °C (constant)
Data source	TU Berlin	TU Berlin / MG-FARM[82]
Used in chapters	Chapter 4	Chapters 5 and 6

3.1.1 NMC Dataset for Stationary-Storage Ageing Estimation

The first dataset consists of cyclic ageing tests on Li-ion INR18650 NMC cells carried out at Technische Universität Berlin. The cells have a nominal capacity of 2,6 A h, a nominal voltage of 3,7 V, a maximum charging voltage of 4,2 V, and a discharge cut-off at 2,75 V. In total, 14 cells are investigated across seven ageing scenarios, with two cells per scenario. The scenarios systematically vary four operating parameters: mean state of charge, depth of discharge, discharge C-rate, and operating temperature. This parametric design is intended to reproduce representative operating windows of stationary-storage systems rather than highly dynamic mobile profiles. Charging follows a CC-CV procedure, while discharging uses constant current. Periodic capacity tests inserted at defined intervals track the SOH progression over time and supply the degradation labels used for model training.

Table 3.2: Ageing scenarios of the NMC stationary-storage dataset and the systematic variation of mean SOC, DOD, discharge C-rate, and operating temperature.

Scenario	Cell No.	$\mathrm{SOC}_{\mathrm{mean}}$	DOD	$\mathrm{C\text{-}Rate}_{\mathrm{dis}}$	T (°C)
1	1 and 2	50%	100%	1C	35
2	3 and 4	20%	30%	1C	35
3	5 and 6	50%	30%	1C	35
4	7 and 8	80%	30%	1C	35
5	9 and 10	50%	100%	2C	35
6	11 and 12	50%	100%	1C	45
7	13 and 14	50%	100%	2C	45

The sampling intervals depend on the respective process step, so the measurements are resampled to 1 s before feature engineering. Scenario 2 contains premature test termination and irregular measurements for cells 3 and 4. These data remain part of the training pool because they represent the type of incomplete and non-ideal records that can also occur in stationary-storage operation. Complete cells are nevertheless withheld for independent testing, as described in Chapter 4.

The state of health is defined as the ratio of the current cell capacity to the nominal capacity,[17, 14]

$$\mathrm{SOH}(t) = \frac{C(t)}{C_{\mathrm{N}}}, \tag{3.1}$$

where $C(t)$ is determined from periodic capacity tests in which the cell is fully charged and then discharged under controlled conditions. Because capacity tests exist only at discrete time points, the SOH label is linearly interpolated onto the continuous measurement timeline. Figure 3.1 shows the resulting SOH progression for all 14 cells over both equivalent full cycles and absolute time in days. Temperature has a pronounced impact on degradation, whereas the interactions between the remaining parameters are less obvious from direct visual inspection alone.

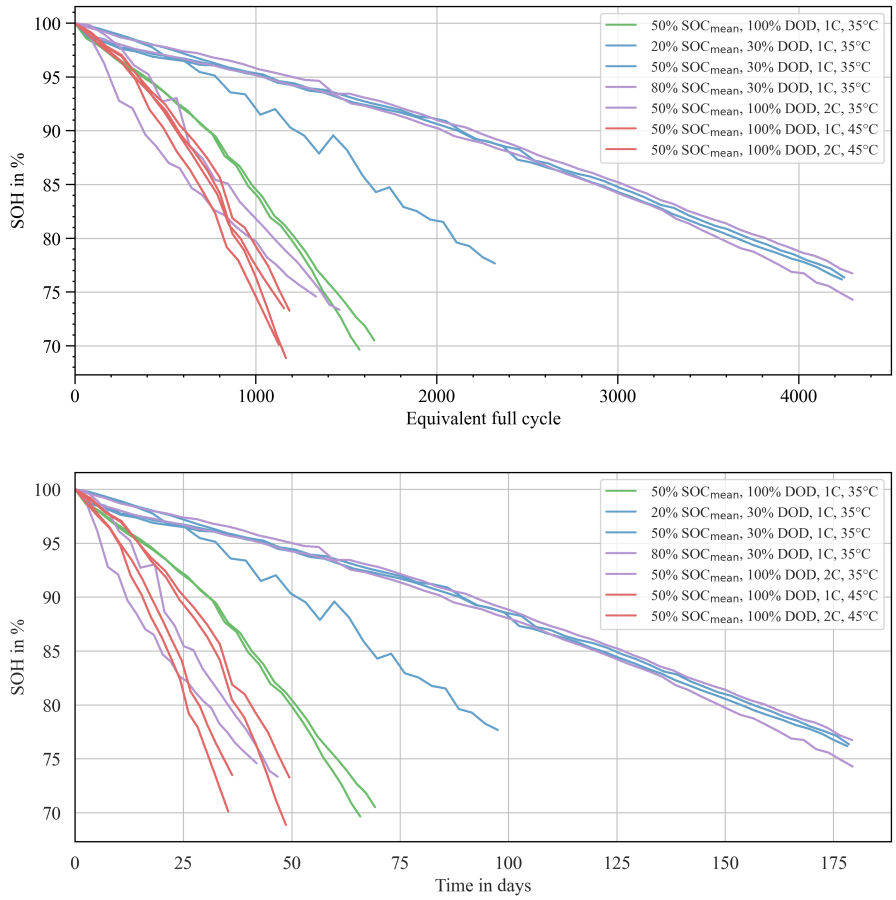


Figure 3.1: SOH progression of the 14 investigated NMC cells: top over equivalent full cycles and bottom over absolute time in days. Colors denote the seven ageing scenarios, with two cells per scenario.

This dataset is used exclusively in Chapter 4. The present section defines the cells, test scenarios, and SOH labels, while the contribution chapter concentrates on the cumulative-current and lag-sequence feature engineering.

3.1.2 LFP Dataset for Robustness Benchmarking and Embedded Deployment

The second dataset is the published LFP DoE Cycle Ageing Dataset recorded at Technische Universität Berlin within the MG-FARM project.[82] It uses commercially available JGCFR18650-1800mAh-3.2V LFP cells with a nominal capacity of 1,8 A h and a nominal voltage of 3,2 V. The ageing campaign follows a three-factor design-of-experiments scheme conducted at a constant ambient temperature of 25 °C. The three varied factors are the charge rate C_{chg} from 0.3 C to 1.0 C, the

discharge rate C_{dis} from 0.5 C to 3.0 C, and the depth of discharge DoD from 38% to 71%. A mixed full and fractional factorial plan assigns 15 test cells across eight operating points. Between ageing loops, dedicated check-up phases with capacity tests, open-circuit-voltage sweeps, and hybrid pulse-power characterisation are inserted. Stationary-storage load profiles derived from residential PV home storage are additionally applied so that the data contain both tightly controlled diagnostics and application-oriented dynamic behaviour. Raw measurements are available at 1 Hz and include voltage, current, terminal temperature, and auxiliary channels.

A design of experiments is used here to structure the ageing campaign so that several operating influences can be varied systematically within a manageable number of cells.[83] Instead of changing only one operating variable at a time, the DoE plan combines charge rate, discharge rate, and depth of discharge into defined operating points. This makes it possible to observe not only the isolated influence of one factor, but also differences that arise from their combined loading conditions. For the later SOC and SOH studies, this is important because the estimators are not trained and evaluated on one narrow cycling regime. They see cells that have aged under different current rates and cycling depths, while the check-up phases still provide comparable reference information across the campaign. The DoE structure therefore gives the dataset both experimental control and operational diversity, which is useful for robustness benchmarking and embedded estimator validation.

Figure 3.2 visualizes the three-factor operating-point design. Figure 3.3 shows a representative segment of the resulting trajectory structure, combining cyclic ageing blocks, check-up phases, and stationary-storage load-profile segments. Figure 3.4 shows the SOH spread across cells and operating points over the full campaign duration.

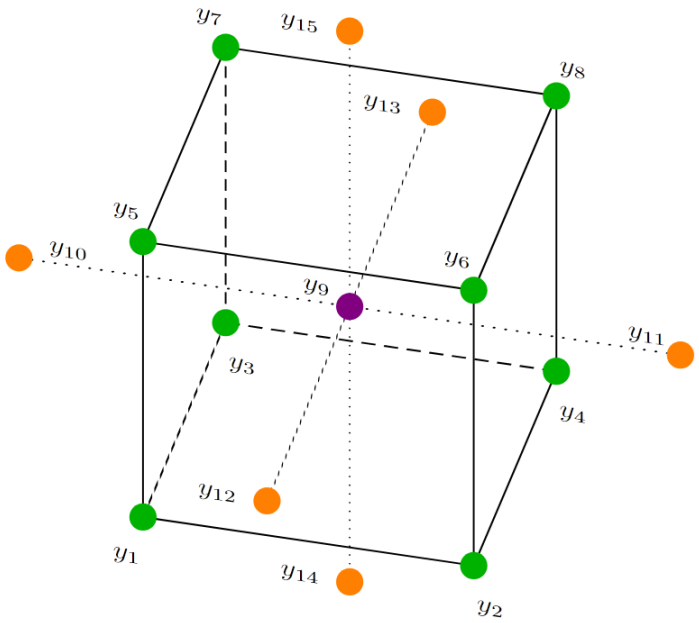


Figure 3.2: Design-of-experiments cube of the LFP ageing dataset. The three axes correspond to charge C-rate, discharge C-rate, and depth of discharge. The highlighted points define the factorial test matrix assigned to the 15 ageing cells.

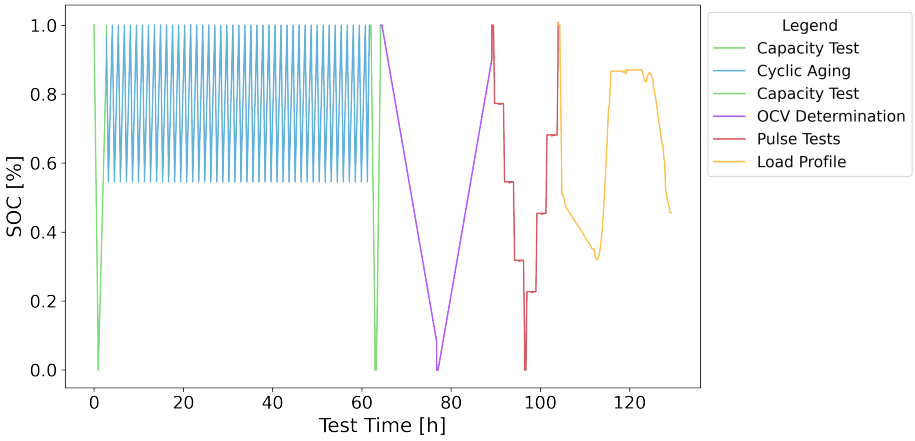


Figure 3.3: Representative SOC trajectory illustrating the experimental structure of the LFP campaign. The sequence combines cyclic ageing operation, diagnostic check-up blocks, and stationary-storage load-profile segments, capturing both laboratory reference phases and application-oriented dynamic behaviour.

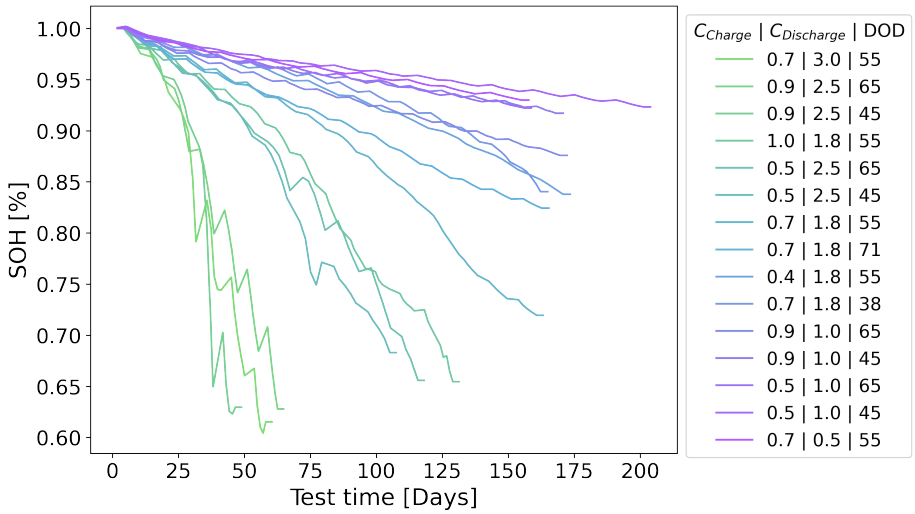


Figure 3.4: SOH progression across the 15 LFP cells over the full ageing campaign. Each trajectory is constructed from discrete capacity check-ups. Local increases between successive check-ups represent variations in measured available capacity and do not imply a reversal of the long-term ageing process.

The local upward movements in Figure 3.4 require a distinction between physical degradation and measured available capacity. The plotted points originate from repeated capacity check-ups and are not model predictions. A later check-up can yield a temporarily higher capacity because relaxation and reversible recovery, conditioning before the test, small temperature differences, current-integration uncertainty, and voltage cut-off detection all affect the measured result.[17] The available data do not isolate one of these effects as the unique cause of every increase. The correct interpretation is therefore a downward long-term ageing trend with locally non-monotonic measurement anchors. Linear interpolation carries these anchor variations into the displayed SOH trajectories.

3.1.3 Reference labels and dataset-side processing

The LFP dataset contains measurements from which the labels for Chapters 5 and 6 are constructed. For a sample interval Δt_k, the signed transferred charge is

$$Q_k = \frac{I_k \Delta t_k}{3600}, \tag{3.2}$$

and the measured capacity of a dedicated capacity-test interval $\mathcal{T}_{\mathrm{cap}}$ is

$$C_k = \sum_{j \in \mathcal{T}_{\mathrm{cap}}} |Q_j| \,. \tag{3.3}$$

The embedded-AI study uses the conventional nominal-capacity definition

$$\mathrm{SOH}_k = \frac{C_k}{C_{\mathrm{nom}}}, \tag{3.4}$$

where C_{nom} is the nominal cell capacity. The robustness benchmark instead normalizes the capacity to the first measured reference capacity of its selected trajectory,

$$\mathrm{SOH}_{\mathrm{ref},k} = \frac{C_k}{C_{\mathrm{ref},0}}, \tag{3.5}$$

which sets the beginning of that trajectory to one and isolates relative capacity loss during the benchmark. The two definitions express the same capacity-based concept but use different denominators for their respective evaluation objectives.[4, 18]

The SOC label is derived from Coulomb counting with drift compensation,

$$\mathrm{SOC}(t) = \mathrm{SOC}(t_0) - \frac{1}{C_{\mathrm{app}}} \int_{t_0}^{t} I(\tau)\, \mathrm{d}\tau, \tag{3.6}$$

where C_{app} is the capacity reference used by the respective processing path. The upper reference is reset to SOC = 1 only after the voltage has remained near the upper limit for 300 s. This sustained condition captures a constant-voltage full-charge phase and prevents a short voltage excursion from being interpreted as a full cell. For the robustness trajectory, the equivalent discrete reference is

$$\mathrm{SOC}_{\mathrm{ref}} = 1 + \frac{Q_c}{C_{\mathrm{ref},0}}, \tag{3.7}$$

with $Q_c = 0$ at the sustained upper-voltage anchor and $Q_c = -C_{\mathrm{ref},0}$ at the lower-voltage anchor.

These quantities are reference labels rather than online estimator inputs. Their construction uses complete capacity-test intervals and confirmed full-charge phases that are unavailable during ordinary deployment. The feature pipelines in Chapters 5 and 6 use only causal measurement-derived information and are described in the respective contribution chapters.

3.2 Embedded Targets and Execution Chain

The embedded work relies on two complementary targets: the final Custom BMS Platform hardware and an STM32H755-based evaluation-board setup for rapid firmware iteration and controlled implementation tests. The integrated BMS hardware is used whenever sensing, balancing, protection, communication, telemetry, and on-board state estimation must be validated as one coupled system rather than as isolated software modules, whereas the evaluation-board setup is used whenever estimator code, toolchain details, memory demand, and runtime behaviour have to be checked quickly before transfer to the full Custom BMS Platform.

In both cases, the deployment path follows the same practical sequence: state-estimation methods are first prototyped in Python or MATLAB, then rewritten as plain C modules, compiled into the STM32 firmware, and executed locally on the target hardware instead of off-board. During validation, the firmware records voltage, current, temperature, and relevant intermediate estimator variables. After each run, these traces are aligned with higher-fidelity host-side SOC and SOH reference labels. The workflow does not use a hardware-in-the-loop setup, which keeps the evaluation focused on firmware execution, measured board behaviour, and comparison against host-side reference traces. This split between evaluation-board work and integrated BMS operation is important for the present research because it separates pure implementation questions from questions of sensing, protection, and full system integration.

3.3 Custom BMS Platform

The custom laboratory hardware developed in this work is referred to simply as the Custom BMS Platform. It is a physically realised 4s laboratory BMS designed to monitor and protect the cells, execute on-board SoC and SoH estimation, perform balancing, and provide the technical basis for controlled experiments with embedded battery intelligence. A central reason for the in-house design is adaptability: in contrast to a commercial off-the-shelf BMS, the Custom BMS Platform gives direct access to the sensing chain, firmware structure, interfaces, and estimator integration, so that state-estimation methods can be exchanged, modified, and benchmarked repeatedly on the same hardware basis. The board should therefore be understood less as a finished product and more as a configurable research platform that was built deliberately to support rapid iteration, customisation, and

comparative testing.

3.3.1 System Objective and Laboratory Scope

The present implementation is a compact and reproducible 4s laboratory BMS for battery ageing studies and embedded state-estimation experiments at TU Berlin EET. Because only a small number of cells can be connected, the platform is intended for laboratory work, including ageing tests, cyclic and calendar experiments, pack and general hardware tests, and representative charging and discharging profiles, rather than for unrestricted field deployment in large battery racks or real applications. Exact field loads are not reproduced one-to-one in the laboratory. Instead, representative operating profiles are approximated within a controlled setup that allows calibration, diagnostics, repeated validation, and structured dataset generation.

This laboratory orientation is deliberate because it enables benchmark-style comparison of different state-estimation approaches under the same measurement chain, including classical model-based observers and modern data-driven or neural methods executed directly on the STM32 controller. The platform concept is explicitly geared toward experimentation: one hardware base should support multiple estimator variants without rebuilding the full BMS around each method. The validation campaigns include duty cycles derived from decentralized stationary-storage use cases with limited maintenance access, and these tests were used to record voltage drift, current ripple, temperature gradients, and embedded estimator outputs under realistic operating conditions.

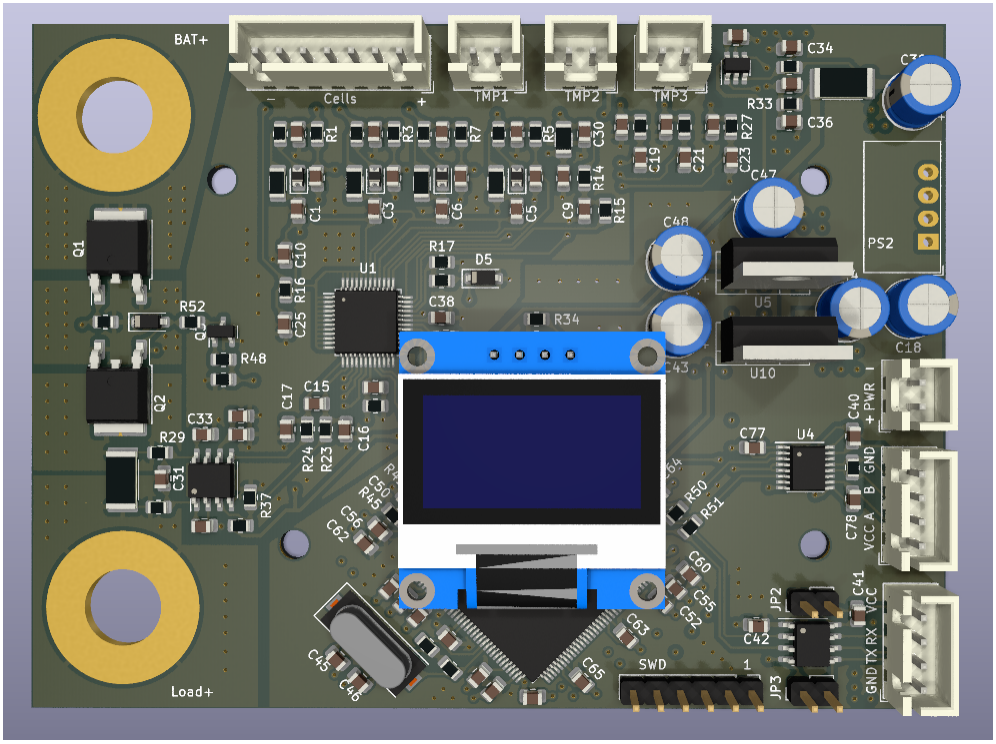


Figure 3.5: Board-level view of the Custom BMS Platform with integrated sensing, controller, communication, and auxiliary interfaces.

3.3.2 Hardware Architecture

The Custom BMS Platform can be understood as three tightly coupled hardware domains: the TLE9012DQU-based sensing and balancing stage, the current-measurement and protection interface, and the controller/communication island around the dual-core STM32H755. The board layout keeps the high-current zone, shunt, and relay path physically separated from the low-voltage controller and communication domain. This separation matters not only for safety, but also for electromagnetic compatibility and stable measurement quality.

The Infineon TLE9012DQU forms the measurement backbone of the pack: in the laboratory build it supervises four series-connected cells with buffered RC filtering and provides cell-voltage and temperature acquisition together with passive balancing capability. The active front-end includes balancing circuitry, while unused monitor inputs are terminated according to the 12-to-4 cell adaptation of the reference design so that measurement integrity is preserved on the active channels. Temperature instrumentation is extended beyond the standard TLE inputs by an NTC harness that can connect additional probes at cell terminals, busbars,

or ambient reference points, which provides a finer spatial thermal picture during ageing and state-estimation tests. The controller domain uses an STM32H755ZITx dual-core microcontroller. This design choice is central because it creates a clean architectural separation between conventional BMS functionality and the estimator layer investigated in this work.

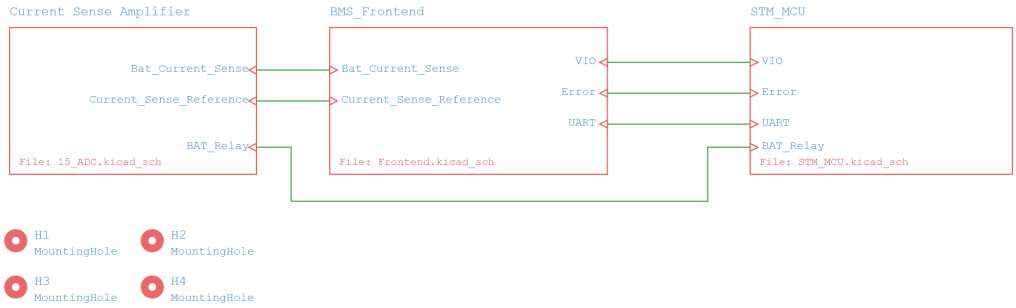


Figure 3.6: Reduced system-level block view of the Custom BMS Platform showing the interaction between current sensing, BMS front-end, and microcontroller domain.

3.3.3 Firmware and Execution Concept

The firmware concept is intentionally centred on the dual-core STM32 architecture. The Cortex-M4 core is used for conventional BMS functionality such as measurement handling, supervision, and general system management, whereas the Cortex-M7 core is reserved for the computationally heavier state-estimation routines. This split is important for the research because it makes the estimator layer exchangeable: different SoC and SoH methods can be integrated, tested, and replaced on the M7 side without redesigning the complete BMS firmware structure. The platform was built exactly for this purpose, namely that one stable hardware and BMS basis should support repeated experimentation with different embedded-AI methods, different estimator classes, and different implementation variants. From the perspective of this work, the central contribution of the firmware architecture is therefore not the low-level implementation detail itself, but the fact that it creates a reusable and customisable execution environment for embedded battery state estimation.

3.3.4 Experimental Relevance for Embedded AI

The Custom BMS Platform is not only a monitoring board, but an experimental platform for comparing estimator classes under realistic embedded conditions. Its main value for this research lies in controllability and openness: the same measurement hardware can be reused while state-estimation methods are changed, refined, compressed, or replaced. This makes it possible to compare classical model-based observers and data-driven or neural approaches on one common embedded basis instead of distributing them across unrelated hardware environments. The Custom BMS Platform therefore closes the gap between algorithm development and embedded validation, and it was chosen precisely because it can be modified and extended in ways that would be difficult with a purchased closed BMS. At the same time, the 4s implementation remains deliberately compact: it serves as a reproducible laboratory baseline from which later transfer to larger stacks or a 16s/48 V variant can be reasoned without changing the overall control and estimator architecture.

Chapter 4

Neural Network Architecture for Ageing Estimation in Stationary Storage Systems

This chapter develops the ageing-estimation study published by Rzepka et al. into the broader research argument.[1] The general differences between model-based and data-driven estimation, the role of Kalman filtering, and the foundations of MLP and recurrent architectures are discussed in Chapter 2. The present chapter concentrates on the study-specific question of whether physically meaningful history can make a comparatively simple feedforward model effective for stationary-storage SOH estimation.

4.1 Study Objective and Data Basis

SOH estimation is important for stationary storage because storage units must remain available over long maintenance intervals and under changing renewable-power and load conditions.[84, 85] Classical capacity tests provide reliable diagnostic references, but they interrupt normal operation and are not continuously available. The estimator must therefore learn to infer degradation from operational measurements.

The central assumption is that voltage, temperature, and current-throughput history can provide this information without requiring a highly elaborate model. A compact MLP is used deliberately so that the effect of feature representation can be separated from the benefit of recurrent architecture. The NMC cells, ageing scenarios, periodic capacity tests, resampling, irregular records, and SOH-label construction are documented in Section 3.1, Table 3.2, and Figure 3.1. This chapter begins with the feature engineering applied to those measurements and labels.

4.2 Preprocessing Method and Feature Engineering

4.2.1 Cumulative Current

The preprocessing pipeline is built around the argument that instantaneous current alone is not sufficient as an explanatory variable for ageing estimation. Cumulative current is introduced as the central preprocessing feature because it captures the two quantities that are most relevant at this stage: the time dependence of the measurements and the cumulative throughput stress experienced by the cell. In its basic form, integrating the absolute current over time encodes not only the present loading level but also the prior electrochemical exposure and, therefore, a compact proxy for the cyclic age that is not visible from an instantaneous sample alone,

$$Q(t) = I_{\text{cumulative}}(t) = \int_0^{\Delta t} |I(t)|\, dt. \tag{4.1}$$

The formulation is also physically motivated by the interaction between current and voltage over a sequence: their joint evolution carries ageing-relevant information that is closer to an impedance-like gradient than a single current reading. The preprocessing is therefore extended by distinguishing charging and discharging contributions, because both directions can induce different voltage gradients and different effective ageing signatures. The corresponding error reduction can be observed empirically, but because these effects are strongly collinear, the improvement cannot be attributed cleanly to one isolated mechanism alone. In this formulation, cumulative current becomes the feature that allows a relatively simple feedforward network to still access ageing-relevant temporal context without explicitly using a recurrent structure. Voltage and temperature are retained as further direct inputs, so the final feature set combines operating-state information with a compact representation of the prior load exposure.

4.2.2 Estimation of the State of Health

The SOH definition, the construction of SOH labels from periodic capacity tests, and the resulting progression across all 14 NMC cells are described in Section 3.1 (Equation 3.1, Figure 3.1). Temperature has a pronounced impact on degradation, which motivates the correlation analysis below as a more structured view of the relation between operating parameters and SOH.

4.2.3 Correlation Analysis

To better understand which measured quantities should be retained for learning, the SOH trajectories are supplemented with the correlation analysis summarized in Figure 4.1. This matrix provides a more structured view of the relation between operating parameters and SOH because the influence of several operating variables and their mutual interactions is not sufficiently clear from the SOH trajectories alone. The resulting heatmap indicates that temperature and cumulative current, both in positive and negative direction, show the clearest direct relation to SOH. In contrast, the time derivative of the voltage, although itself strongly temperature-dependent, does not show a comparably direct influence in the linear correlation view.

The correlation analysis can only reveal linear dependencies and therefore serves only as a starting point for feature selection rather than as a substitute for the later model-based evaluation. Non-linear effects may still be relevant and can subsequently be captured by the neural network. Even with this limitation, the analysis provides a concrete justification for retaining temperature and cumulative-current descriptors as core inputs for the final model.

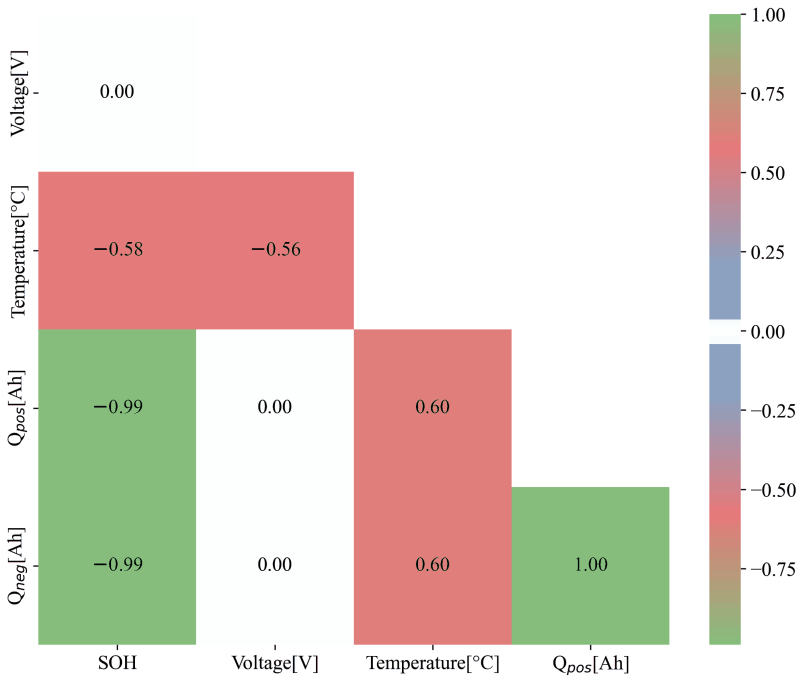


Figure 4.1: Correlation heatmap illustrating the linear relation between SOH and candidate input variables, including temperature and cumulative current Q in positive and negative direction. Correlation coefficients near $+1$ or -1 indicate strong linear dependence, whereas values near 0 indicate little or no direct linear relation.

4.2.4 Formation of the Lag Sequence

Each learning sample is constructed as a lag sequence of voltage, temperature, and cumulative current, with the SOH at the current point in time used as the corresponding label, as illustrated in Figure 4.2. This transforms the continuous measurement stream into structured sliding-window samples that can be processed by a feedforward network without passing the complete life history of an individual cell directly into the model. The inclusion of past input values is intended to improve prediction quality, while the sliding window defines the temporal duration of the usable history. The initial parameterisation uses a time resolution of 30 min and 12 lag steps, which corresponds to a history length of 6 h. This starting choice depends on the expected usage pattern, the charging and discharging durations, and the presence of short high-current phases in the data.

The time resolution has to match the actual charging and discharging behaviour. If the spacing is too coarse, relevant dynamics disappear. If it is too fine, the model sees an unnecessarily long and noisy representation that can make training less

effective. If short pulses fall within one interval, they are represented only through the averaged feature values of that interval. This design can also be linked to the sampling theorem: the spacing between data points has to remain compatible with the temporal structure of the charging and discharging cycles so that the network can still recognise the relevant periodic behaviour.[86] Because the raw measurements are resampled before sequence generation, the preprocessing step also acts like a low-pass filter and strongly attenuates high-frequency noise components, so noise is not treated as a dominant issue for the present estimation setup. A further benefit of the lag-sequence approach is that the training samples are no longer tied to one specific cell trajectory. Instead of memorising individual cells, the model is trained on many local windows drawn from the full dataset.

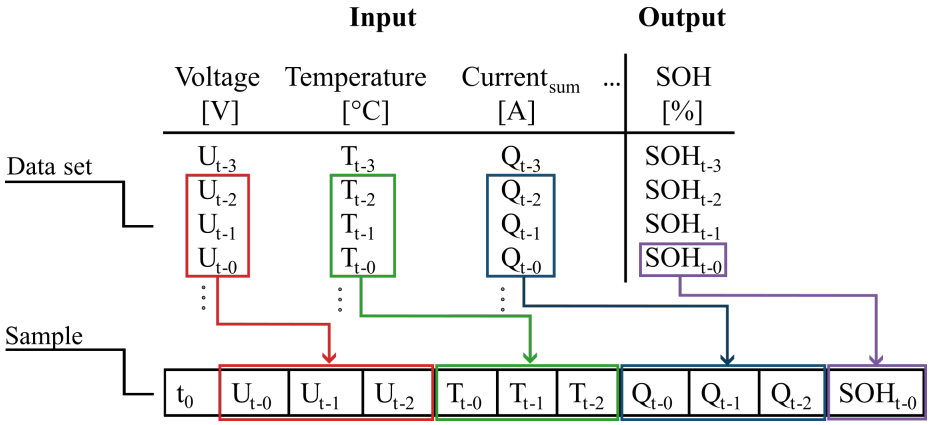


Figure 4.2: Illustration of lag-sequence construction from raw cell measurements into supervised MLP samples. Voltage, temperature, and cumulative current form the structured input history, while the SOH at the current time step serves as the corresponding label.

4.2.5 Training, Validation, and Test Splits

The available data are separated into training, validation, and test subsets, as summarised in Figure 4.3. The training split is used to adapt the network weights, while 20% of the available training sequences are reserved for validation. The validation split is used during training to monitor overfitting and generalisation behaviour and to support hyperparameter decisions without exposing the model to the final test cells. At the same time, several complete cells are withheld from the training process for the final performance assessment. In total, four out of fourteen cells are retained for this independent accuracy evaluation. This choice is important because the method is intended to generalise to previously unseen cells and not only

to previously unseen windows extracted from known cells. The explicit separation of training, validation, and test data therefore makes the resulting performance assessment more robust and more informative.

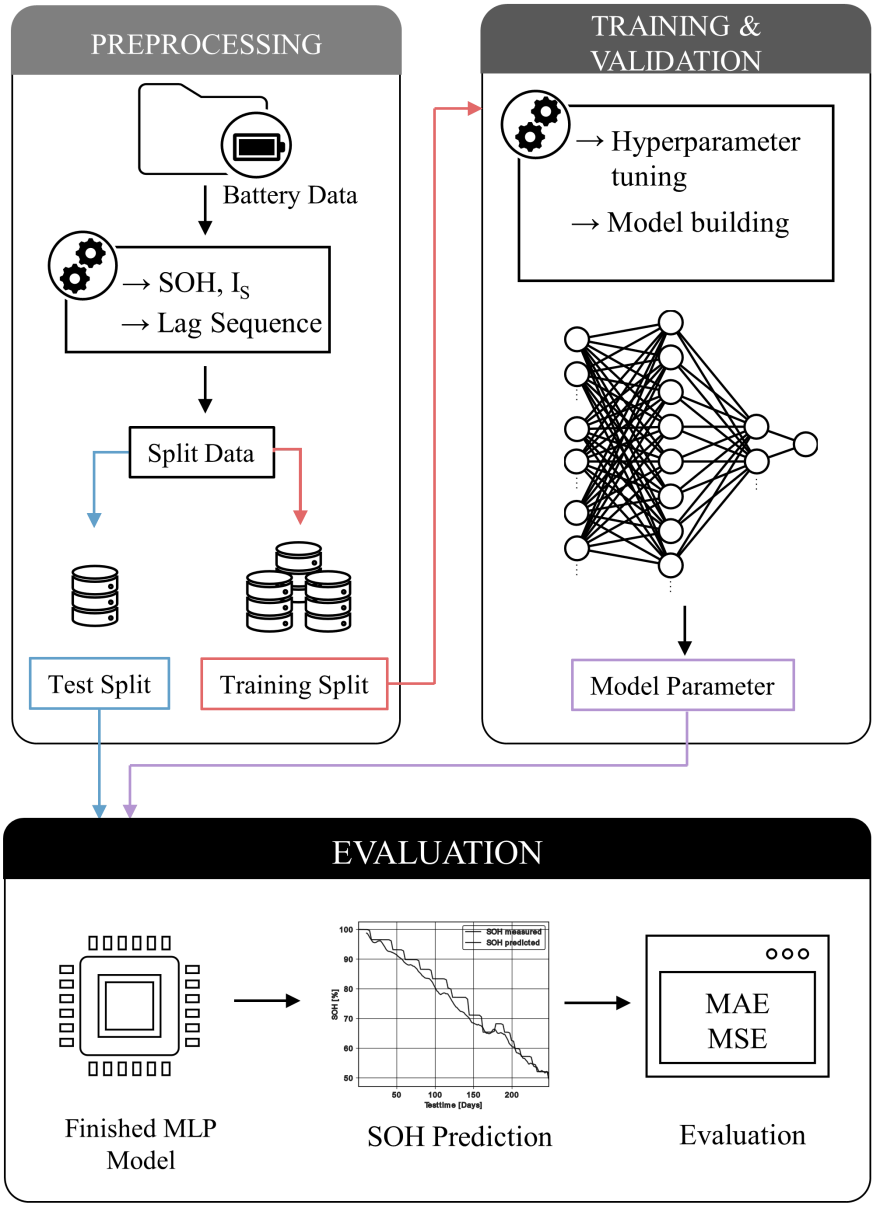


Figure 4.3: Workflow of the proposed MLP-based SOH estimation approach, from raw-cell preprocessing and data splitting through training and hyperparameter tuning to prediction and final evaluation.

4.2.6 Data Scaling

All input features are scaled to the interval between 0 and 1 before training.[87] This step is necessary because the numerical ranges of the features differ strongly: voltage remains in a narrow interval between 2,75 V and 4,2 V, whereas cumulative current can rise to several thousand ampere-hours across the full ageing experiment. Uniform scaling improves numerical stability, supports convergence during optimisation, and avoids a situation in which large-magnitude inputs dominate the network weights while smaller but still relevant signals are effectively ignored.[87] The same scaling convention must be applied consistently to the training, validation, and test data so that the learned mapping is evaluated under comparable feature ranges and the resulting performance metrics remain interpretable across all splits.

4.3 Multilayer Perceptron Neural Network Model Structure

The selected model type is a multilayer perceptron with fully connected layers. A convolutional interpretation is not pursued because the data do not exhibit a spatial image-like structure. Recurrent architectures such as LSTMs are also not used at this stage, because the temporal dependence is already encoded through the lag-sequence construction and because the recurrent alternatives are more resource-intensive and more difficult to tune in the chosen setting.[88] This architecture choice is also connected to the later hyperparameter-search setup: the chapter keeps the estimator architecture feedforward and shifts the main optimization effort to lag-window design, network depth, node count, and training parameters, which are then searched efficiently with Hyperband.[89]

The general feedforward principle is introduced in Section 2.4.2 and illustrated in Figure 2.2. The study-specific model receives the flattened voltage, temperature, and cumulative-current lag sequence and produces one linear SOH estimate. The initial search configuration is intentionally small and uses one hidden layer with eight nodes. Network depth and width are then selected through the hyperparameter search. In the tuned configuration, summarised in Table 4.1, the model grows into a deep dense architecture with nine hidden/output stages and a final linear regression output. The resulting structure remains conceptually feedforward, while

its concrete layer widths are adapted to the ageing-estimation task.

Table 4.1: Final tuned configuration of the SOH-estimation MLP after hyperparameter optimisation, listing the number of nodes and the activation function used in each layer of the network.

Layer	Nodes	Activation Function
0	128	ReLU
1	256	ReLU
2	256	ReLU
3	256	ReLU
4	128	ReLU
5	512	ReLU
6	128	ReLU
7	256	ReLU
8	256	ReLU
9	1	Linear

4.4 Evaluation Metrics and Hyperparameter Tuning of the Neural Network

This section defines the error metrics used for training, validation, and final testing before summarising the hyperparameter tuning of the MLP. During training and validation, model quality is monitored primarily with the mean squared error,

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \tag{4.2}$$

where n is the number of evaluated samples, y_i is the measured SOH value of sample i, and $\hat{y}_i$ is the corresponding prediction. After training, the independent test cells are evaluated with both MSE and mean absolute error,

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|. \tag{4.3}$$

MAE is particularly useful for interpretation because it remains in the same unit as the SOH target. The stored SOH and prediction series use the normalized 0 to 1 representation, but the original evaluation scripts multiply both series by 100 before calculating the reported MAE and MSE values. The ageing-estimation results in this chapter are therefore reported primarily on the 0 to 100% SOH scale, with MAE in percentage points and MSE in squared percentage points. Where comparison with later chapters benefits from the work-wide 0 to 1 convention, the corresponding normalized values are stated explicitly.

The hyperparameter search covers the number of lag steps, the temporal resolution of the lag sequence, network depth, number of nodes, batch size, kernel regularisation, optimizer type, and activation function. Within this set, the number of lag steps and the temporal resolution are the most method-specific parameters, because they define how much history is provided to the model and at which temporal granularity it is represented. The number of nodes controls the representational capacity of the network, whereas the network depth influences how strongly the model can build more abstract internal feature combinations across layers. Both parameters therefore affect the balance between flexibility, generalisation, and model complexity. Hyperband is used as the main search strategy because it distributes training resources adaptively, evaluates many candidate settings quickly, and allocates more epochs only to the most promising configurations.[89]

The study reports that ADAM outperforms RMSProp, Adadelta, and Adagrad for this task, that larger batch sizes reduce overfitting tendencies, and that explicit kernel regularisation does not lead to a meaningful improvement. Batch size also affects the training trade-off directly: larger batches are computationally efficient and converge reliably, whereas smaller batches can improve generalisation at the cost of longer or less stable optimisation. The optimizer analysis is complemented by an activation-function comparison, in which ReLU yields the lowest loss in the tested setup while sigmoid, tanh, and softmax perform worse. The models are trained on approximately 15,000 sequences with varying batch-size and epoch combinations, and larger batches converge to a comparable minimum while reducing overfitting tendencies.

4.5 Results and Discussion

This section evaluates how the two most influential sequence-design parameters, namely the number of lag steps and the temporal resolution, affect the final SOH-estimation performance. The result presentation and interpretation are kept together because the numerical errors depend directly on the chosen history length, the temporal aggregation, and the ageing trajectory of the evaluated cell. For the detailed analysis, Figure 4.4 compares three exemplary cells aged under different conditions. Four time resolutions are considered, namely 1 min, 10 min, 30 min, and 60 min, while the number of lag steps is varied between 8, 12, 16, and 20. All MAE values in this section are shown in percentage points of SOH to remain directly interpretable on the original 0 to 100% SOH scale.

Across the exemplary cells, the most favourable behaviour appears at intermediate temporal aggregation rather than at the shortest or longest tested sampling intervals. This matches the characteristic charge and discharge durations of the investigated scenarios, which typically fall in the range of approximately 20 min to 1 h. If the time resolution is too large, relevant behaviour is averaged out and important information is lost. If it is too small, the training process becomes less effective, the effective histories become unnecessarily long, and the resulting representation does not translate automatically into better prediction quality. The MAE matrix shows the smallest errors for 16 lag steps at a time resolution of 10 min, which corresponds to a history length of 160 min. Extending the sequence length beyond 20 steps is reported as disadvantageous because the resulting history becomes excessively long and does not improve prediction quality further.

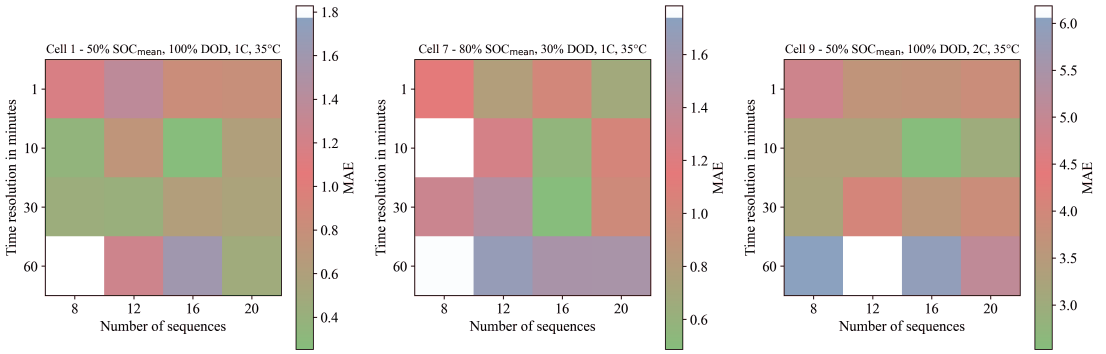


Figure 4.4: Matrix of MAE values shown in percentage points of SOH on the 0 to 100% SOH scale. The corresponding normalized MAE on a 0 to 1 SOH scale is obtained by scaling the displayed values accordingly. The colour scale indicates the magnitude of the prediction error and highlights that the most favourable operating region is found at intermediate temporal aggregation rather than at the shortest or longest tested interval.

Figure 4.5 compares the predicted and measured SOH trajectories. Cells 7 and 1 represent typical ageing behaviour and are reproduced well, whereas a cell with a pronounced U-shaped SOH progression leads to a substantially larger prediction error. Cells 7 and 1 are therefore used as representative examples of typical ageing behaviour, whereas cell 9 serves as a deliberately selected atypical case. Figure 4.6 complements this trajectory-based view with a direct comparison between predicted and measured SOH values. For the favourable individual cells, the reported best errors are MSE values of $0.410\,\%^2$ and $0.118\,\%^2$ and MAE values of $0.487\,\%$ and $0.253\,\%$. These correspond to normalized MAE values of 0.00487 and 0.00253 on a 0 to 1 SOH scale. Across the evaluated cases, the average MAE is $1.10\,\%$, corresponding to a normalized MAE of 0.0110, and the average MSE is $2.95\,\%^2$, corresponding to a normalized MSE of $2.95 \cdot 10^{-4}$. The prediction plots additionally show that a post-rolling filter is applied to reduce visual noise and to make the comparison between measured and estimated SOH easier to interpret.

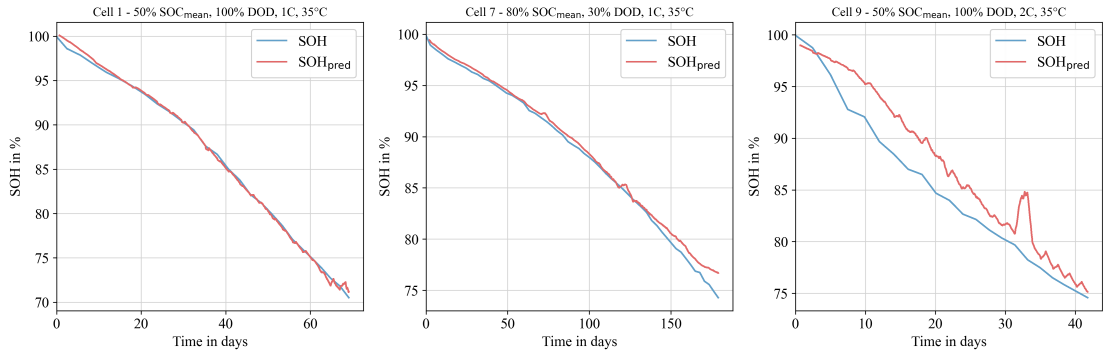


Figure 4.5: Comparison of predicted and measured SOH trajectories over time for three representative test cells. Cells 7 and 1 show the typical ageing progression most frequently observed in the dataset and are estimated accurately by the network, whereas Cell 9 was intentionally chosen as an atypical example with a pronounced U-shaped SOH curve. A post-rolling filter is applied to reduce visual noise and to make the agreement and deviation between measured and predicted trajectories easier to interpret.

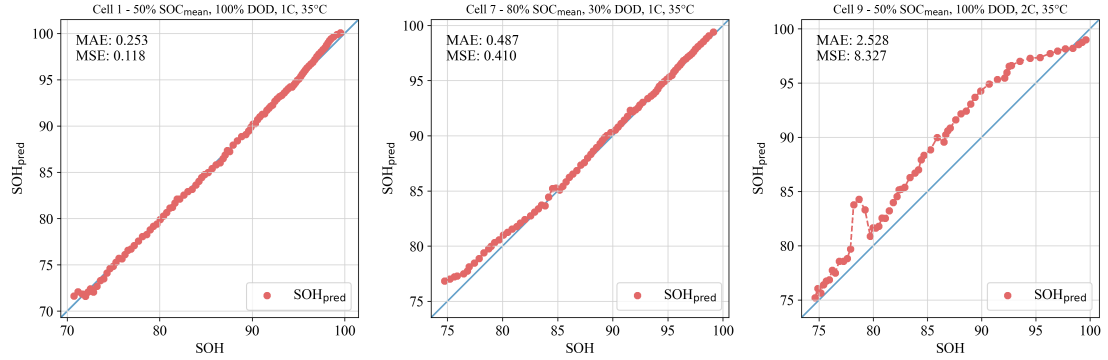


Figure 4.6: Scatter comparison between predicted and measured SOH values for the three representative test cells. Cells 7 and 1 exhibit only small dispersion and low absolute error, indicating accurate prediction, whereas Cell 9 shows substantially larger deviation in both MSE and MAE because of its atypical ageing behaviour. Error values are shown on the 0 to 100% SOH scale, with MAE in percentage points and MSE in squared percentage points.

The effect of removing the problematic cells 3 and 4 from the training basis is also examined. In that variant, the aggregate error improves only marginally to an MAE of 1.07 % (normalized MAE 0.0107) and an MSE of 2.52 %^{2} (normalized MSE $2.52 \cdot 10^{-4}$). At the same time, the behaviour on the atypical cell 9 becomes more

unstable, which indicates that cleaner data alone do not replace broader data diversity. The result section therefore already makes a broader methodological point: the exact optimum for sequence length and time resolution depends strongly on the available dataset, and direct comparison with other literature remains difficult if the underlying cell data differ substantially.

4.6 Conclusion

The ageing-estimation study shows that a meaningful SOH estimate can be obtained with a comparatively simple feedforward model when the preprocessing and feature engineering are aligned with the underlying ageing problem. The central result is not only the achieved average MAE of 1.10 % and average MSE of 2.95 %^{2}, but the way in which this accuracy is reached: cumulative-current features and lag-sequence construction supply temporal and throughput-related context to an otherwise memoryless MLP. The chapter therefore demonstrates that input representation can be as important as model class, especially when the target variable evolves slowly and is only observed directly during periodic capacity tests.

The results also define the practical boundary of the approach. The best lag-sequence setting in the investigated dataset uses 16 lag steps with a time resolution of 10 min, corresponding to a history length of 160 min, while the generally useful settings are found at intermediate temporal resolutions rather than at the shortest or longest tested intervals. This optimum is not universal. It depends on the charge and discharge durations, the operating profiles, and the available ageing trajectories. The atypical behaviour of cell 9 and the limited benefit of removing cells 3 and 4 show that neural SOH estimation is strongly constrained by data coverage and label quality. Cleaner data can reduce some error, but it cannot replace a broad dataset that contains the relevant ageing modes.

Within the present work, this chapter should therefore not be read as a claim that feedforward MLPs are universally preferable to recurrent or hybrid models. Its role is to establish a first data-driven baseline in which ageing-relevant temporal information is represented explicitly while the estimator architecture itself remains deliberately simple. Because the method relies on directly measured quantities such as voltage, temperature, and current-derived throughput features, the concept is not tied to one narrow cell technology in principle, but transfer to other chemistries or applications would require sufficiently rich training data and renewed validation

of the lag-sequence settings.

This baseline role is important for the broader research argument. Once a workable representation for ageing information has been identified, the next question is no longer only how well the estimator performs under nominal conditions, but how different estimator classes behave under deployment-relevant disturbances such as bias, noise, missing samples, and initialization errors. This motivates the shift toward the robustness benchmark in the following chapter. Building on this robustness perspective, the later chapters extend the analysis to recurrent LSTM and MLP models for SOC and SOH estimation, where the aim is no longer only to compare estimator concepts at a general level, but to examine whether stateful recurrent models offer advantages when temporal information has to be processed causally over long streaming sequences and ultimately transferred to embedded microcontroller execution.

Chapter 5

Robustness Benchmarking of Embedded Battery State Estimation

This chapter presents the robustness study submitted as a manuscript to the *Journal of Energy Storage*.[2] Its estimator classes build on the direct-measurement, model-based, hybrid, and neural foundations introduced in Chapter 2 and summarized in Figure 2.1.

5.1 Motivation and Study Objective

Reliable SOC and SOH estimation shapes safety margins, usable energy, operating limits, charge control, and degradation-aware management in mobile and stationary battery systems.[15, 26, 90] Clean-condition accuracy is not sufficient for this purpose because real BMS operation is exposed to sensor bias, additive noise, missing samples, irregular timing, restart-related initialization mismatch, and transient signal spikes.[7, 8, 9]

The literature provides extensive results for individual estimator families, but shared cross-family disturbance protocols remain less common. A direct integrator, a voltage-corrected observer, and a recurrent neural estimator can obtain similar average errors for different reasons and can therefore fail differently when the same input is disturbed. This chapter addresses that comparison gap by treating estimator quality as three connected properties: nominal baseline accuracy, convergence after state inconsistency, and robustness under disturbed measurements or measurement continuity faults.

The central objective is a measurement-only robustness benchmark in which estimator performance is isolated from broader system-level effects such as hardware-

resource optimization or implementation-effort comparison. The benchmark is deliberately restricted to disturbances applied only to measurable inputs and their timing, including quantization effects, bias, additive noise, missing samples, irregular sampling, burst dropout, voltage spikes, and initialization mismatch. This design keeps the scenarios physically interpretable and allows the resulting failure modes to be compared under reproducible conditions.

Four estimator classes are compared. DM denotes the direct measurement model, while HDM denotes its SOH-supported hybrid variant. HECM is the hybrid equivalent circuit model with voltage feedback. DD is the data-driven recurrent model. The SOH information is kept consistent across the SOH-aware variants. This isolates the effect of the SOC correction mechanism and supports decision-oriented model selection rather than a claim that one estimator family is universally superior.

5.2 Methodology

5.2.1 Estimator classes under test

Figure 5.1 summarizes the benchmark chain from measured signals to online feature reconstruction, disturbance injection, estimator execution, and global or local performance evaluation. The comparison covers four estimator classes with deliberately different correction mechanisms: DM is the fixed-capacity Coulomb-counting reference, HDM adds capacity adaptation through the shared online SOH estimate, HECM adds voltage-residual feedback through a SOH-aware two-RC observer,[25, 26] and DD represents the compact recurrent neural estimator with stateful GRU-based SOC prediction.[91, 90]

HDM, HECM, and DD all use the same LSTM-based online SOH estimator. The benchmark therefore compares how the SOC estimators use a common health context instead of comparing different SOH backbones. The causal online quantities required by several estimators, especially cumulative charge Q_c, equivalent full cycles EFC, derivatives, and hourly SOH aggregates, are reconstructed from the measured stream during benchmark execution. This prevents precomputed dataset-side features from entering only one estimator class.

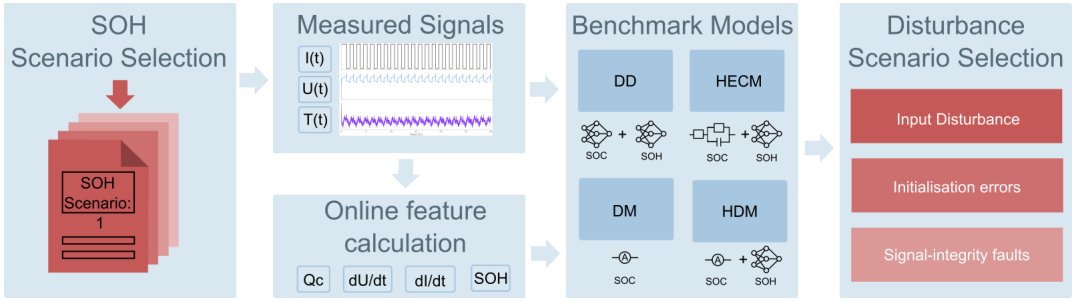


Figure 5.1: Methodology overview of the robustness benchmark. Measured current, voltage, temperature, and time signals are first converted into causal online auxiliary quantities such as cumulative charge Q_c, derivative channels, equivalent full cycles EFC, and shared SOH context. The resulting online stream is then processed by the four estimator classes under matched execution conditions, exposed to measurement-only disturbance scenarios, and finally evaluated through global and local performance metrics that separate baseline accuracy, convergence behavior, and disturbed-operation robustness.

5.2.2 SOC estimation methods

Direct-measurement model (DM)

The overview in Figure 5.1 is implemented through four estimator variants that differ in their SOC correction mechanism, but share the same causal measurement stream. The direct measurement branch first constructs the cumulative charge state

$$Q_c(k) = Q_c(k-1) + \frac{I_k \Delta t_k}{3600\,\mathrm{s\,h^{-1}}}, \tag{5.1}$$

where I_k is the measured current and Δt_k is the elapsed time between two samples. The division by $3600\,\mathrm{s\,h^{-1}}$ converts the current-time product into ampere-hours. In the DM estimator this charge state is mapped to SOC with a fixed nominal capacity,

$$\widehat{\mathrm{SOC}}_{\mathrm{DM}}(k) = \mathrm{clip}\left(1 + \frac{Q_c(k)}{C_{\mathrm{nom}}}, 0, 1\right). \tag{5.2}$$

Hybrid direct-measurement model (HDM)

The same integration core is also used in the HDM estimator. The difference is that the denominator is updated through the shared online SOH estimate,

$$C_{\mathrm{eff}}(k) = C_{\mathrm{nom}} \cdot \widehat{\mathrm{SOH}}(k), \tag{5.3}$$

which gives the hybrid direct-measurement SOC estimate

$$\widehat{\mathrm{SOC}}_{\mathrm{HDM}}(k) = \mathrm{clip}\left(1 + \frac{Q_c(k)}{C_{\mathrm{eff}}(k)}, 0, 1\right). \tag{5.4}$$

DM and HDM therefore differ only in the assumed usable capacity. Both variants keep the same constant-voltage full-charge reset logic: if the upper-voltage condition is sustained for the configured duration, the internal charge state is reset to $Q_c = 0$ and the SOC estimate is anchored at full charge.

Hybrid equivalent-circuit model (HECM)

The HECM extends the current-driven charge balance with voltage-residual correction. It is implemented as a two-RC observer with state vector

$$\mathbf{x}_k = \begin{bmatrix} \widehat{\mathrm{SOC}}_k & U_{1,k} & U_{2,k} \end{bmatrix}^\top, \tag{5.5}$$

where $U_{1,k}$ and $U_{2,k}$ denote the polarization voltages of the two RC branches. The prediction step uses the previous current sample and propagates the state according to

$$\mathbf{x}_k^- = \mathbf{A}_k \mathbf{x}_{k-1} + \mathbf{b}_k I_{k-1}, \tag{5.6}$$

with

$$\mathbf{A}_k = \mathrm{diag}\left(1, e^{-\Delta t_k/\tau_1}, e^{-\Delta t_k/\tau_2}\right), \qquad \mathbf{b}_k = \begin{bmatrix} \eta \Delta t_k / C_b \\ R_1\left(1 - e^{-\Delta t_k/\tau_1}\right) \\ R_2\left(1 - e^{-\Delta t_k/\tau_2}\right) \end{bmatrix}. \tag{5.7}$$

Here $C_b = C_0 \widehat{\mathrm{SOH}}$ is the SOH-scaled capacity used by the observer. The voltage prediction used for the correction step is

$$\hat{U}_k = \mathrm{OCV}\left(\widehat{\mathrm{SOC}}_k, \widehat{\mathrm{SOH}}_k, I_k\right) + U_{1,k} + U_{2,k} + R_i I_k. \tag{5.8}$$

The Kalman correction then uses the residual between this predicted terminal voltage and the measured voltage to update the internal state. The parameters R_i, R_1, R_2, τ_1, τ_2, OCV, and dOCV/dSOC are not treated as constants. Instead, they are interpolated from pre-identified SOC/SOH lookup surfaces with separate charge and discharge parameterizations. This is the main structural reason why HECM can re-anchor the integrated SOC estimate through voltage information while still using the same online SOH context as HDM and DD.

Data-driven SOC model (DD)

The DD estimator uses a stateful GRU instead of an explicit observer equation. At each online time step it receives the feature vector

$$\mathbf{z}_k = \begin{bmatrix} U_k & I_k & T_k & \widehat{\mathrm{SOH}}_k & Q_c(k) & \frac{\mathrm{d}U}{\mathrm{d}t}(k) & \frac{\mathrm{d}I}{\mathrm{d}t}(k) & \Delta t_k \end{bmatrix}^\top . \tag{5.9}$$

Figure 5.2(a) shows the block structure of this model. In its floating-point base configuration, a single GRU layer with hidden size 96 processes the eight-channel feature vector. An MLP head with one hidden layer of size 96, ReLU activation, and a sigmoid output then maps the recurrent state to a SOC estimate constrained to $[0, 1]$. The benchmark itself uses the deployment-prepared variant of this model, in which structured pruning reduces the hidden size from 96 to 67 and the weights are stored as INT8 values (Section 5.2.4). During training, the recurrent core is exposed to causal sequence segments of

$$L_{\mathrm{SOC}} = 2024 \tag{5.10}$$

consecutive time steps. During benchmark execution the same model is run statefully and causally, so the hidden state is carried forward sample by sample and the MLP head produces a continuous SOC trajectory without access to future samples.

The derivative channels in the DD input are rebuilt online from the disturbed measurement stream,

$$\frac{\mathrm{d}I}{\mathrm{d}t}(k) = \frac{I_k - I_{k-1}}{\Delta t_k}, \qquad \frac{\mathrm{d}U}{\mathrm{d}t}(k) = \frac{U_k - U_{k-1}}{\Delta t_k}. \tag{5.11}$$

Together with the explicit Δt_k channel, these quantities allow the neural model to distinguish local voltage or current dynamics from timing changes under irregular sampling. The inclusion of $Q_c(k)$ also ensures that the DD model receives the same charge-throughput information that drives DM and HDM.[91, 90]

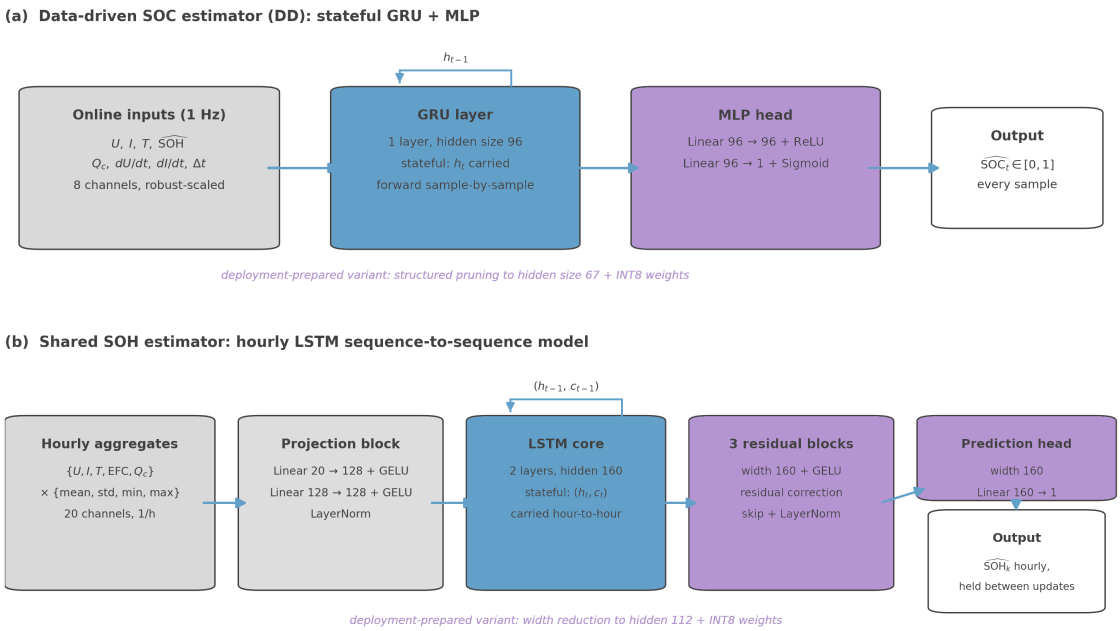


Figure 5.2: Block-level architectures of the data-driven estimator components used in the benchmark. (a) The DD SOC estimator processes the eight online input channels with a single stateful GRU layer of hidden size 96, followed by an MLP head with one ReLU-activated hidden layer of size 96 and a sigmoid output. (b) The shared SOH estimator forms a 20-dimensional hourly feature vector by computing four statistics for each of the five causal base signals $\{U, I, T, \mathrm{EFC}, Q_c\}$, projects this vector through a two-layer feature-projection block of width 128, processes the projected sequence with a two-layer stateful LSTM of hidden size 160, and maps the recurrent representation through three residual MLP blocks and a prediction head of width 160. The deployment-prepared variants used in the benchmark are described in Section 5.2.4.

5.2.3 SOH estimation method

HDM, HECM, and DD all use the same LSTM-based SOH estimator. Its input is reconstructed causally from the five online base signals $\{U, I, T, \mathrm{EFC}, Q_c\}$. For every hourly update, the mean, standard deviation, minimum, and maximum are computed over the samples contained in the respective hour. The resulting SOH input is therefore not a set of 20 independently measured channels, but a 20-dimensional feature vector obtained from five base signals and four aggregation statistics per signal.[92, 93, 94, 1]

Figure 5.2(b) summarizes the architecture. The first stage is a learned feature-

projection block with two linear layers of width 128, layer normalization, GELU activation, and dropout. This block transforms the heterogeneous hourly statistics into a task-specific representation before recurrent processing. The second stage is a two-layer unidirectional LSTM with hidden size 160. It is operated strictly causally: hidden and cell states are forwarded from one hourly aggregate to the next, and no future measurements are used. The third stage consists of three residual MLP blocks with width 160. In compact form, each residual block refines the recurrent representation as

$$\mathbf{y} = \mathrm{LayerNorm}(\mathbf{x} + \mathrm{Dropout}(W_2\,\mathrm{GELU}(W_1\mathbf{x}))), \tag{5.12}$$

so that the block learns a correction to the LSTM representation instead of replacing it completely. A final prediction head of width 160 maps the refined representation to one scalar SOH estimate for the current hourly step.

During benchmark execution the first SOH output is anchored with the configured initial SOH value, and later estimates are held piecewise constant between hourly updates. This common health context is then consumed differently by the SOC estimators: HDM uses it through C_{eff}, HECM uses it in the observer capacity and lookup scheduling, and DD receives it as one channel of its recurrent SOC input. The deployment-prepared SOH-LSTM variant used for the footprint-constrained benchmark is described in Section 5.2.4.

5.2.4 Embedded-oriented model preparation

The benchmark focuses on estimator behavior under disturbed measurements, but the compared variants are still constrained by an embedded reference budget. All four estimator classes are selected and prepared so that they remain deployable on a common STM32H755ZI-class target with a conservative 1 MB flash and 1 MB RAM reference limit. This prevents an unfair comparison between compact analytical estimators and neural models that would only work as large offline networks.

DM requires no compression because it consists only of the scalar charge-state update and the associated reset logic. HECM also does not require neural-network compression, but its flash demand is dominated by the two SOH×SOC lookup grids for charge and discharge parameterization of the ECM observer. The neural components used by HDM and DD are deployment-prepared before the robustness benchmark. For the SOC-GRU, structured width reduction and short finetuning

reduce the hidden size from 96 to 67, followed by INT8 weight quantization. For the SOH-LSTM, the deployment-prepared variant uses hidden size 112 after width reduction, finetuning, and INT8 quantization. The resulting reusable component footprints are listed in Table 5.1.

The table is a component-level estimate rather than a list of the four complete estimator variants. The complete variants are obtained by combining the SOC core used by each estimator with the shared SOH-LSTM where applicable: DM consists only of the Coulomb-counting core, HDM combines the Coulomb-counting core with the SOH-LSTM, HECM combines the 2RC observer with the SOH-LSTM, and DD combines the SOC-GRU with the SOH-LSTM. This gives lower-bound persistent footprints of 4.2 KiB flash and 16 B RAM for DM, 412.5 KiB and 1808 B for HDM, 850.8 KiB and 1844 B for HECM, and 436.2 KiB and 2060 B for DD. These RAM values count only architecture-intrinsic persistent numerical states, not temporary buffers, stack usage, framework overhead, or complete runtime memory. They therefore provide a strict lower bound, but they show that all four deployment-prepared benchmark variants stay below the 1 MB flash and RAM reference limits.

Table 5.1: Estimated persistent flash and RAM footprints of the reusable estimator components used in the deployment-prepared benchmark variants. For the recurrent neural components, flash estimates refer to the structured-pruned, finetuned, and INT8-quantized versions. RAM* counts only architecture-intrinsic persistent numerical states. Temporary buffers, framework overhead, stack usage, and system-level runtime memory are excluded.

Component	Flash [KiB]	RAM* [B]	Basis
DM core (Coulomb counting)	4.2	16	Four persistent scalar states in the online update logic
SOC-GRU core	27.9	268	One recurrent hidden state with size 67 stored as float32
SOH-LSTM core	408.3	1792	Two-layer LSTM with hidden and cell states of size 112 stored as float32
HECM core (2RC observer)	442.5	52	Flash dominated by two SOH×SOC parameter tables for charge and discharge parameterization. RAM: observer state $x \in \mathbb{R}^3$, covariance $P \in \mathbb{R}^{3\times3}$, previous current

5.2.5 Performance benchmark design

The benchmark perturbs only signals that a BMS controller would actually receive from its sensors: current, voltage, temperature, sample availability, and sampling timing. Model-internal quantities, such as LSTM hidden states or the model weights themselves, are never modified directly. This restriction keeps all disturbance scenarios physically interpretable: every failure mode tested corresponds to something that can go wrong at the sensor, wiring, or communication layer of a real embedded system, rather than to artificial tampering with the estimator internals.[9]

Figure 5.3 separates the resulting cases into input disturbances, initialization mismatch, and measurement-continuity faults, meaning disturbances of sample availability, sample timing, or frozen data updates. The numerical levels in Table 5.2 combine three roles: realistic acquisition limits, adverse but plausible measurement disturbances, and deliberate stress cases for recovery analysis.[19, 21, 23, 22,

95, 96] During every run, all four estimators receive the identical disturbed sensor time series, namely the same modified sequence of voltage, current, temperature, and timing samples, so that any difference in output is attributable to estimator structure rather than to differences in the input. Each estimator then reconstructs its auxiliary quantities (cumulative charge Q_c, equivalent full cycles EFC, current derivative, hourly SOH aggregates) causally and online from this disturbed time series, not from precomputed clean dataset columns. A sensor bias therefore propagates into the derived features exactly as it would in a real BMS, which keeps the comparison physically consistent across all estimator classes.

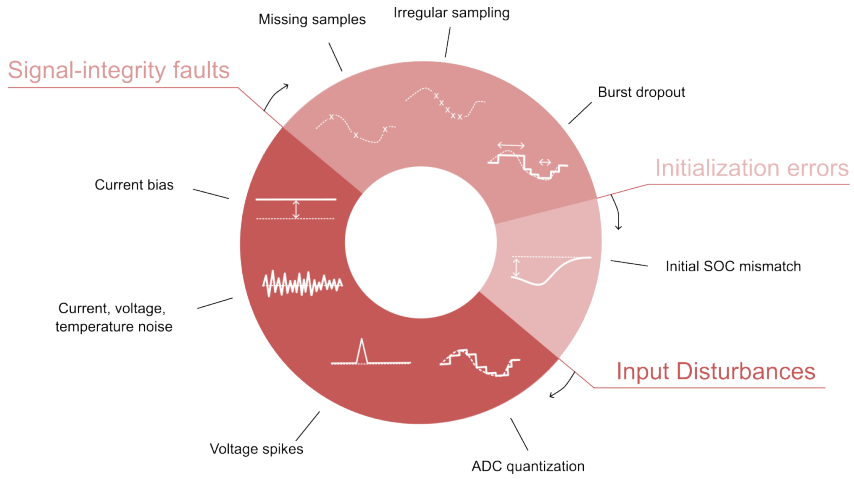


Figure 5.3: Disturbance taxonomy used in the robustness benchmark. Input disturbances, initialization mismatch, and measurement-continuity faults are separated so that drift, loss of correction, delayed recovery, and transient instability can be interpreted as different estimator-side effects.

Table 5.2: Scenario protocol used in the robustness benchmark. The entries define the fixed disturbance settings and the practical motivation behind each measurement or continuity/timing case.

Scenario	Benchmark setup	Practical rationale
ADC quantization	$\Delta I = 0{,}01\,\mathrm{A}$, $\Delta U = 0{,}005\,\mathrm{V}$, $\Delta T = 0{,}5\,^{\circ}\mathrm{C}$	Deployment-level discretization, sensor scaling, and rounding effects in embedded acquisition chains.[19, 21, 20]

Continued on the next page

Table 5.2 continued from the previous page

Scenario	Benchmark setup	Practical rationale
Current bias	Constant +0,05 A in the compact matrix, with an additional sweep of 0.5%, 1.5%, and 3.0% of $\|I\|_{\max}$	Sensor offset, gain error, miscalibration, and long-term drift stress on integration-driven estimators.[21, 23, 22]
Current noise	Gaussian noise with $\sigma_I = 0{,}10\,\mathrm{A}$ in the compact matrix, with a local-detail sweep up to $\sigma_I = 0{,}20\,\mathrm{A}$	Adverse current-measurement noise and noise-driven disturbance propagation.[7, 8]
Voltage noise	Additive Gaussian noise with $\sigma_U = 0{,}01\,\mathrm{V}$	Wiring noise, front-end noise, insufficient filtering, and EMI-sensitive correction channels.[7, 19, 95]
Temperature noise	Additive Gaussian noise with $\sigma_T = 1{,}0\,^{\circ}\mathrm{C}$	Sensor tolerance, conversion noise, thermal gradients, and sensor-to-cell mismatch.[20, 40]
Initial SOC mismatch	Additive initial mismatch of -0.10 SOC, corresponding to a 10-percentage-point offset. In DD, this is realized through perturbed online Q_c	Restart conditions, missing recent history, or weak initial anchoring under practical deployment.[7, 25]
Missing samples	Every 50th sample frozen on the nominal 1 s grid	Packet loss, blocked updates, logger dropouts, or periodic frozen-data events.[8, 96]
Irregular sampling	Uniform timing jitter of $\pm 0{,}1\,\mathrm{s}$ around the nominal 1 s grid	Scheduling jitter, bus latency, and asynchronous acquisition timing.[9, 8, 96]
Burst dropout	One central frozen gap of 3600 s	Communication interruption, frozen tasks, or severe delayed-information recovery stress.[9, 8]
Voltage spikes	One single-sample spike of $\pm 0{,}20\,\mathrm{V}$ every 1000 samples (1000 s)	Transient voltage-sample corruption caused by EMI or switching disturbances.[95]

The compact scenario matrix gives one common protocol for the main result figures. Current bias and current noise are additionally examined through local sweeps, while ADC quantization remains included as a deployment-level sensing case even though it is not expected to dominate the ranking. In this form, the benchmark is a controlled estimator comparison under matched disturbances, not a complete validation of a production BMS.

5.2.6 Metrics and evaluation criteria

The benchmark evaluates three performance dimensions: nominal accuracy, convergence behavior after state inconsistency, and robustness under both longer-lasting and short-term measurement disturbances. These three dimensions are not independent post-hoc labels, but the aggregation logic used later in the result interpretation and in the decision-oriented synthesis. Nominal accuracy is read from the clean baseline run, robustness is read from the additional error caused by each disturbed scenario relative to that estimator's own baseline, and convergence is read from the time needed to return to a baseline-related error band after an imposed inconsistency or interrupted signal stream.

For each estimator m and disturbed scenario s, the main global robustness penalty is therefore defined as

$$\Delta\mathrm{MAE}_{m,s} = \mathrm{MAE}_{m,s} - \mathrm{MAE}_{m,\mathrm{base}}. \tag{5.13}$$

Positive values indicate that the disturbance degrades the estimator relative to its own clean-operation reference. Values close to zero indicate that the disturbance does not materially change the full-trajectory average error. Global disturbed-operation performance is quantified through MAE, RMSE, bias, maximum error, and the 95th percentile absolute error. MAE and RMSE describe the average error level, bias describes systematic over- or underestimation, the maximum error reports the single largest deviation, and P95 is the absolute-error value that 95 % of all evaluated samples do not exceed. P95 therefore describes the larger-error range of the trajectory without being determined by the single worst sample.

Local disturbance performance is additionally analysed through scenario-specific quantities such as jump counts, output variance, absolute-error variance, excess rolling MAE, recovery time, and aligned post-disturbance error, because several scenarios do not primarily manifest as a collapse of full-run MAE but as short-

term output fluctuations, delayed drift, or slow re-synchronization. For scenarios in which the affected samples or time interval are explicitly known, the analysis separates the trajectory into pre-disturbance, disturbance, and post-disturbance phases. This makes it possible to quantify error growth during the disturbance, recovery time after the event, and residual error after the input stream has returned to normal. In the current benchmark phase, ADC quantization, current bias, voltage noise, temperature noise, missing samples, and irregular sampling are interpreted mainly through global metrics, whereas burst dropout, voltage spikes, current-noise detail, and initialization mismatch additionally require local evidence because their most relevant effects are episodic, recovery-related, or strongly localized in time.

Recovery after initialization errors is interpreted through two baseline-relative operating bands rather than through one universal fixed threshold. This keeps the recovery criterion tied to the normal clean-condition performance level of each estimator and avoids judging models with very different baseline accuracy against the same absolute tolerance. The strict band is defined as

$$B_{\text{strict}} = \max(\text{P95}_{\text{base}},\, 1.5 \cdot \text{MAE}_{\text{base}})\,. \tag{5.14}$$

In the strict criterion, P95_{base} ensures that the recovery band includes the larger errors that already occur during clean baseline operation, while $1.5 \cdot \text{MAE}_{\text{base}}$ provides a minimum margin around the average baseline error. Taking the maximum of both terms means that recovery is judged against the more tolerant of these two baseline-derived limits, so recovery under this band means re-entry close to the normal baseline regime without imposing unrealistically narrow limits. The fair band is

$$B_{\text{fair}} = \max(2 \cdot \text{P95}_{\text{base}},\, 3 \cdot \text{MAE}_{\text{base}},\, 0.02)\,, \tag{5.15}$$

and acts as a more deployment-tolerant criterion for estimators that re-enter a practically acceptable operating range without immediately returning to their strict near-baseline regime. The lower bound of 0.02 avoids unrealistically tight fair-band thresholds for very accurate baseline models. Both recovery bands require sustained re-entry over a fixed horizon so that the reported recovery times reflect stable resynchronization rather than an accidental short crossing of the threshold.

Reporting both bands avoids making the recovery conclusion depend on one manually chosen limit. If an estimator returns only to the fair band, it has reached a practically acceptable range but not its near-baseline regime. If it also returns to the

strict band, the recovery is stronger. This multi-scale metric design therefore evaluates performance as a structured combination of nominal accuracy, convergence behavior, global degradation, local disturbance response, and long-term drift.

Table 5.3: Connection between raw benchmark metrics, result figures, and the three performance dimensions used in the robustness interpretation. The table makes explicit how the later statements on accuracy, robustness, and recovery/convergence are derived from the same metric set rather than from separate evaluation criteria.

Dimension	Primary metric evidence	Role in the result interpretation	Typical result sections
Nominal accuracy	Baseline MAE, RMSE, P95 absolute error, maximum error, and bias from the undisturbed run	Quantifies clean-sensing estimator quality before any robustness penalty is applied. This is why an estimator can lead in accuracy even if it is not the most robust under disturbances.	Baseline performance and decision-oriented accuracy score
Disturbed-scenario robustness	$\Delta\mathrm{MAE}_{m,s}$ relative to the estimator's own baseline, complemented by disturbed-window MAE, P95 error, maximum error, bias, jump counts, and local excess rolling MAE where needed	Quantifies how strongly each disturbance changes the estimator's behavior. This supports scenario-specific robustness claims instead of ranking models only by absolute disturbed MAE.	Current bias, noise sensitivity, continuity/timing faults, voltage spikes, cross-scenario comparison

Continued on the next page

Table 5.3 continued from the previous page

Dimension	Primary metric evidence	Role in the result interpretation	Typical result sections
Convergence and recovery	First sustained re-entry into the strict or fair baseline-related error band after initialization mismatch or signal interruption. Non-recovery inside the inspected window is treated as weakest evidence	Quantifies whether an estimator can return to a useful operating regime after state inconsistency. This explains why recovery behavior is evaluated separately from both clean accuracy and global disturbed-operation MAE.	Initialization mismatch, burst-dropout recovery, decision-oriented recovery score

For the decision-oriented synthesis in Section 5.5.3, the same logic is condensed into relative lower-is-better scores across the four estimator classes. The accuracy score is based on the clean baseline errors, the robustness score is based on ΔMAE across the disturbed scenarios, and the recovery score is based on recovery time after initialization mismatch. This aggregation is used only as a compact decision aid. The individual scenario figures and local recovery analyses remain the primary evidence for the detailed conclusions.

5.3 Feature Reconstruction and Experimental Protocol

The benchmark uses one representative trajectory from the LFP DoE Cycle Ageing Dataset described in Section 3.1.[82] It extends from early-life operation into low-SOH operation, including sections below the common 80 % end-of-life reference level. All estimator classes receive this same trajectory and are evaluated against the fixed SOC and SOH labels defined in Equations 3.5 and 3.7.

5.3.1 Causal online feature reconstruction

The benchmark reconstructs every online feature from the same current, voltage, temperature, and timing stream received by the respective estimator. These features include cumulative charge Q_c, equivalent full cycles EFC, the derivative channels dU/dt and dI/dt, and hourly SOH-context aggregates. The channels $\{U, I, T, \mathrm{EFC}, Q_c\}$ are summarized over each hourly interval by their mean, standard deviation, minimum, and maximum.

The EFC feature is reconstructed from absolute charge throughput,

$$Q_{\mathrm{thr}}(k) = \sum_{j=1}^{k} \frac{|I_j|\Delta t_j}{3600}, \qquad \mathrm{EFC}(k) = \frac{Q_{\mathrm{thr}}(k)}{C_{\mathrm{nom}}}. \tag{5.16}$$

Here C_{nom} is the nominal cell capacity used in the benchmark feature pipeline. Unlike the SOC and SOH labels, EFC is an online feature and is therefore recomputed from the current and timing stream of each benchmark run.

This distinction is central to the disturbance design. The reference labels remain unchanged for scoring, whereas Q_c, EFC, dU/dt, dI/dt, and the hourly aggregates are rebuilt from the clean or disturbed inputs. A current bias therefore changes not only the measured current but also every causal feature that depends on current. This prevents undisturbed dataset-side variables from leaking into a disturbed benchmark run.

5.3.2 Test trajectory and benchmark scenarios

All estimator classes are run on the same benchmark trajectory and the same scenario protocol from Table 5.2. This ensures that differences in the results are attributable to estimator structure and disturbance propagation rather than to different data splits or label definitions. The reported scenarios cover the nominal baseline, current bias, additive current noise, voltage noise, temperature noise, initial SOC mismatch, missing samples, irregular sampling, burst dropout, voltage spikes, and ADC quantization. Together they probe long-term integration drift, voltage-correction sensitivity, disturbance propagation through derived features, frozen updates, timing uncertainty, and local post-event recovery. Input disturbances are applied directly to measured channels, initialization errors are injected only where structurally meaningful, and continuity/timing cases manipulate sample availability or timing while preserving online execution logic.

5.3.3 Implementation details

All compared estimators are executed under online conditions. Disturbed inputs are processed sample by sample, and the required auxiliary quantities are rebuilt directly from the disturbed signals rather than copied from precomputed dataset columns. This applies in particular to Q_c, EFC, dU/dt, dI/dt, and the hourly SOH aggregates, so that no hidden dataset-side variables leak into the benchmark runs.

For the recurrent neural estimators, the benchmark uses the intended stateful embedded execution mode. The SOH-LSTM carries its hidden and cell states from one hourly update to the next, while the SOC-GRU preserves its recurrent state along the sample-wise measurement stream. The neural variants used in the benchmark correspond to the compact pruned-and-quantized architectures introduced above. In freeze and dropout scenarios, disturbance propagation is handled consistently by freezing or distorting the derived online features together with the corresponding measured inputs, which is necessary for a physically meaningful comparison between model classes.

The direct-measurement estimators include the same sustained-CV full-charge reset criterion that is also used to define the full-charge reference point in preprocessing, whereas the data-driven initial-SOC test is implemented through an initial offset of the online Q_c feature because the neural estimator does not expose an explicit SOC state variable. The simulation environment therefore keeps the base nominal capacity used for online feature reconstruction fixed across all estimator classes, while all models receive the same disturbed input streams and are evaluated against the same reference labels and global and local metrics. This is one of the main reasons why the benchmark remains close to later embedded deployment: the estimators are not evaluated as offline regressors, but as causal state estimators under matched measurement-side constraints.

5.4 Results

The following evaluation uses the clean benchmark trajectory as the reference point and then examines how each estimator changes when the same online signal stream is affected by bias, noise, missing data, timing faults, spikes, ADC quantization, or initialization mismatch. The results are therefore organized not only by global error level, but also by local recovery and transient behaviour, because deployment quality depends on both the magnitude of the error and the way in which it develops

over time. Numerical SOC errors are kept on the normalized 0 to 1 scale used in the benchmark tables and figures, while percentage-point equivalents are given in the prose where they support interpretation.

5.4.1 Baseline performance

Under clean measurements, the hybrid direct-measurement model is the nominal-accuracy leader on the selected ageing benchmark trajectory. It reaches MAE = 0.0024 (0,24 %), followed by HECM with 0.0090 (0,90 %), DD with 0.0100 (1,00 %), and DM with 0.0311 (3,11 %). This baseline ranking is the reference point for the later ΔMAE comparisons and recovery bands, but it is not yet a deployment recommendation by itself. The complete baseline metric set, including RMSE, P95 error, maximum error, and signed bias, is listed in Appendix Table A.1. Figure 5.4 visualizes the same clean-condition comparison.

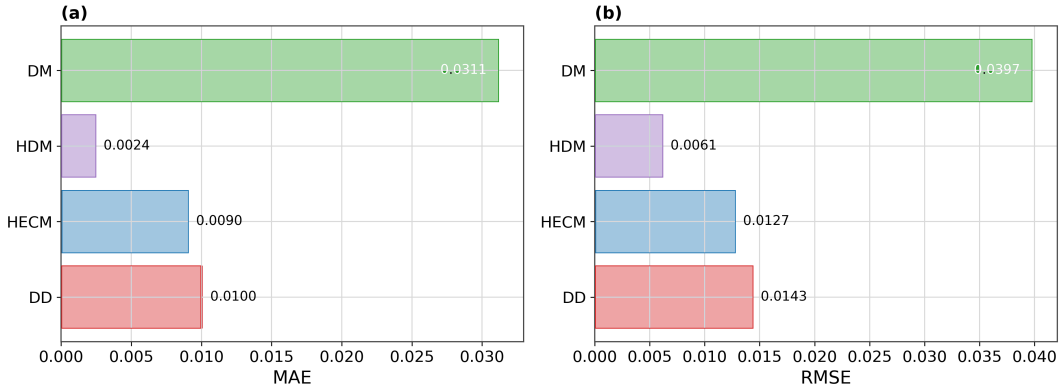


Figure 5.4: Baseline performance on the selected ageing benchmark trajectory. The summary compares the nominal MAE, RMSE, 95th-percentile error, and signed bias of the four estimator classes and therefore provides the clean-condition reference point for all later disturbed-scenario comparisons. The hybrid direct-measurement model provides the lowest baseline error, whereas the direct-measurement model shows the largest nominal deviation. Error values shown in the figure are SOC fractions on a 0 to 1 scale. Multiplication by 100 gives percentage points.

5.4.2 Current-bias sensitivity

Persistent current bias is the most severe integration stress in the benchmark. The dedicated sweep applies offsets of 0.5%, 1.5%, and 3.0% of the maximum absolute

current of the benchmark trajectory, so the percentages describe the bias magnitude relative to $|I|_{\max}$ rather than an error percentage of SOC. Across this sweep, HECM remains the most robust estimator, DD is intermediate, and DM and HDM degrade strongly because the biased current accumulates directly through their integration path. For the strongly biased case reported in the compact scenario matrix, HECM reaches MAE = 0.0932, whereas DD, HDM, and DM increase to 0.2859, 0.3068, and 0.3143, respectively. The full MAE, RMSE, and P95 values are collected in Appendix Table A.2. Figure 5.5 shows the corresponding local bias example, sweep trend, and trajectory divergence.

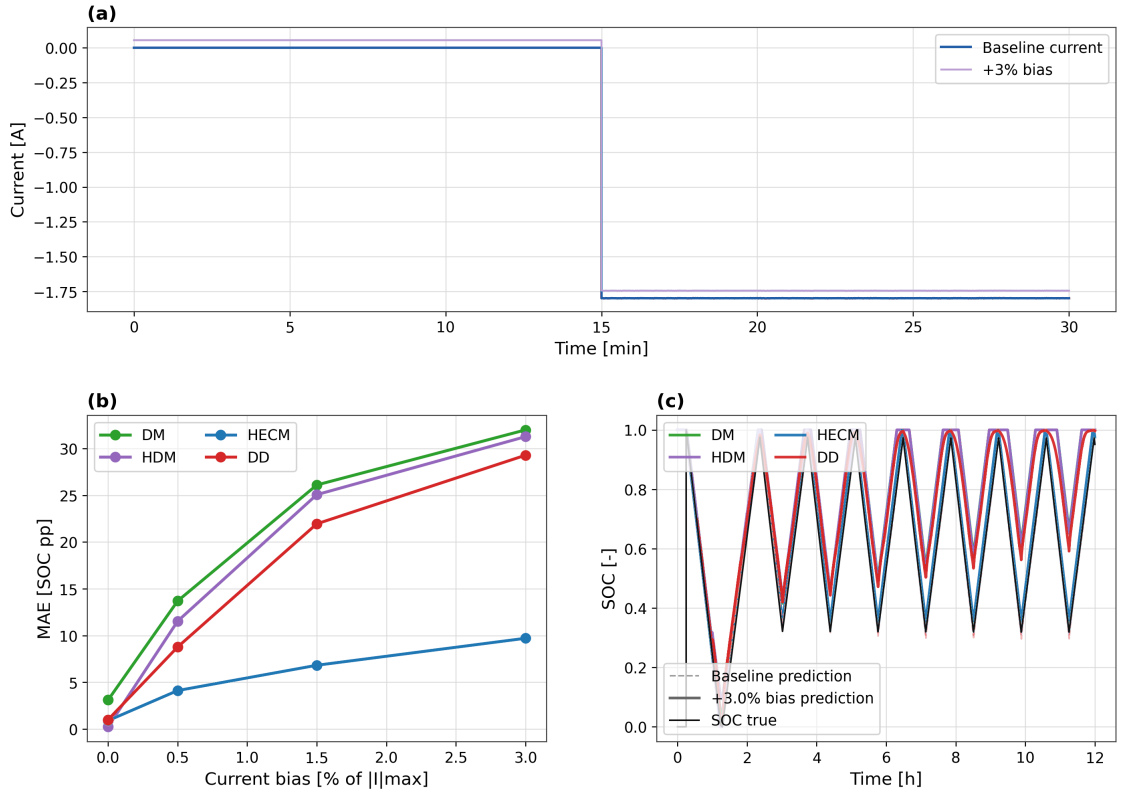


Figure 5.5: Sensitivity to current-measurement bias. The figure combines a representative local example of the applied bias, the global degradation of SOC accuracy across the matched bias sweep, and the resulting SOC divergence over time for a strongly biased case. Together, these views show why persistent current bias clearly separates the integration-based estimators from the voltage-corrected observer variant.

5.4.3 Noise sensitivity

The noise block is interpreted separately from the current-bias scenario because the injected noise terms are zero-mean disturbances rather than persistent offsets. The relevant question is therefore not only whether SOC drifts over the full trajectory, but also how a corrupted measurement channel propagates into observer correction, neural input features, and local output variability. For current noise, this propagation occurs through the measured current itself and through current-derived quantities such as Q_c, $\mathrm{d}I/\mathrm{d}t$, EFC, and the hourly SOH aggregation. In the compact benchmark matrix, the high-current-noise case uses $\sigma_I = 0{,}10\,\mathrm{A}$. The additional sweep in Figure 5.6 extends beyond this value to make local output-variability trends visible. Under the high-current-noise case, HECM shows the largest global penalty with $\Delta\mathrm{MAE} = 0.0075$, whereas HDM, DD, and DM change by 0.0019, 0.0009, and -0.0001, respectively. This does not mean that current noise is irrelevant for the other estimators. Rather, it shows that zero-mean noise mainly appears through local variability and disturbance propagation instead of through the same long-term integration drift caused by persistent bias.

Voltage noise with $\sigma_U = 0{,}01\,\mathrm{V}$ mainly tests the voltage-correction path in HECM and the voltage-dependent input path in the learned estimators. In this scenario HECM remains essentially unchanged at MAE = 0.0090, while DM, HDM, and DD increase to 0.0524, 0.0751, and 0.0698, respectively. Temperature noise with $\sigma_T = 1{,}0\,^{\circ}\mathrm{C}$ is weaker in the global metrics for DM and HECM, but it still affects the SOH-aware paths because temperature enters the learned health context: HDM increases to MAE = 0.0067 and DD to 0.0181. The numerical global and local noise metrics are given in Appendix Tables A.2 and A.3.

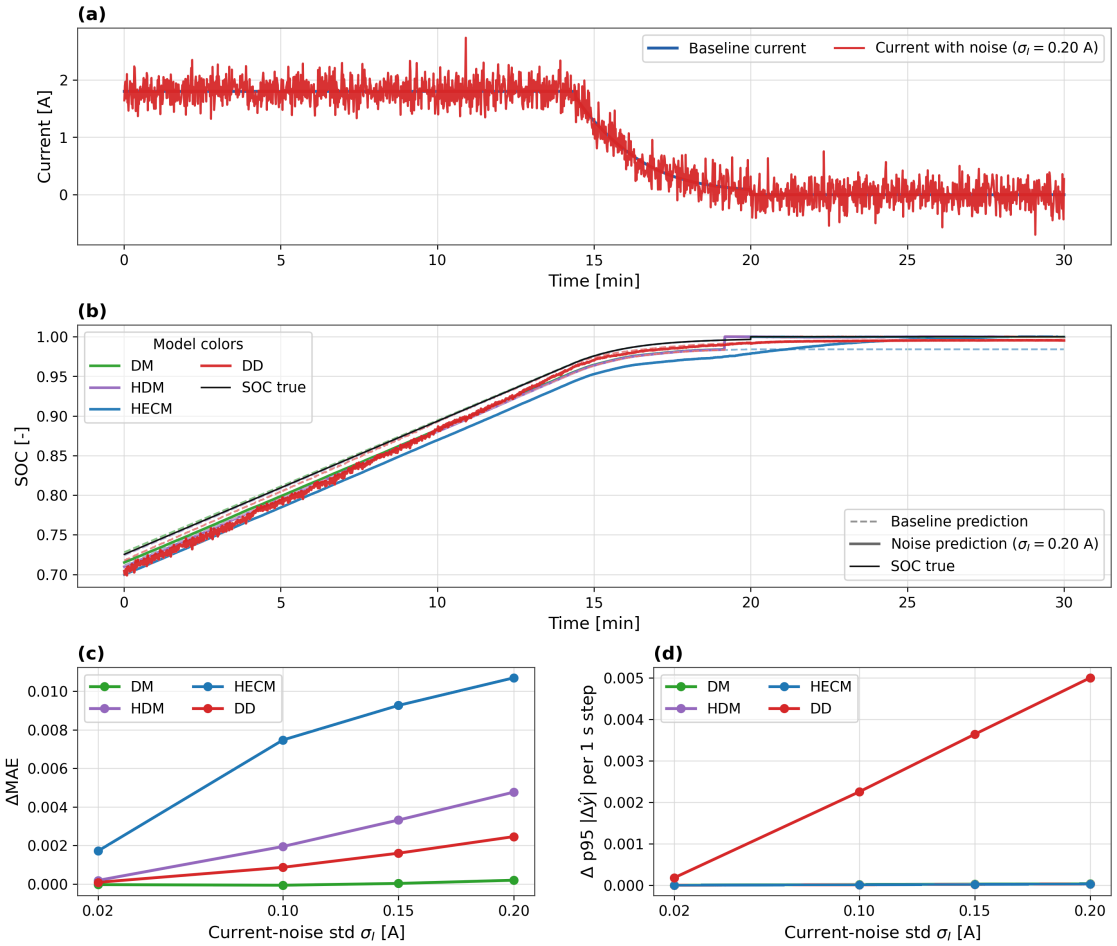


Figure 5.6: Noise sensitivity under additive current noise. The figure combines a representative disturbed window, a matched SOC comparison in the same interval, the global MAE penalty in the compact benchmark matrix, and the local output-variability trend across an extended current-noise sweep. This makes visible that zero-mean current noise is mainly a local disturbance-propagation test rather than the same type of long-term drift test as persistent current bias.

5.4.4 Initialization error and recovery

Initialization mismatch is a recovery problem rather than a simple full-run MAE problem, because the relevant question is whether the estimator can re-synchronize after starting from a wrong SOC value. The benchmark therefore uses a reset-free evaluation window and injects an initial mismatch of -0.10 on the normalized SOC scale, meaning that the online estimate starts 10 percentage points too low. This disturbance represents a deliberately visible restart inconsistency, for example after

missing recent history or weak anchoring of the initial SOC. For DD, the mismatch is implemented as an offset of the online cumulative-charge feature instead of an artificial reset of an inaccessible latent state, which keeps the test aligned with deployment-time observables.

Recovery is evaluated with the strict and fair baseline-related bands defined in Section 5.2.6. These thresholds are not times and not fixed SOC setpoints. They are admissible absolute SOC errors on the normalized 0 to 1 scale. They differ between estimator classes because they are computed from each estimator's own clean-condition MAE and P95 error. The strict band marks return to the near-baseline regime, while the fair band is a more tolerant operating range. In the inspected window, DD returns to the strict band after about 1.05 h and to the fair band after about 1.00 h. DM returns after about 1.17 h and 0.25 h, respectively. HECM returns after about 2.37 h and 1.16 h. HDM does not re-enter either band. The exact threshold values and recovery times are listed in Appendix Table A.3. Figure 5.7 shows the corresponding trajectory-level recovery behavior.

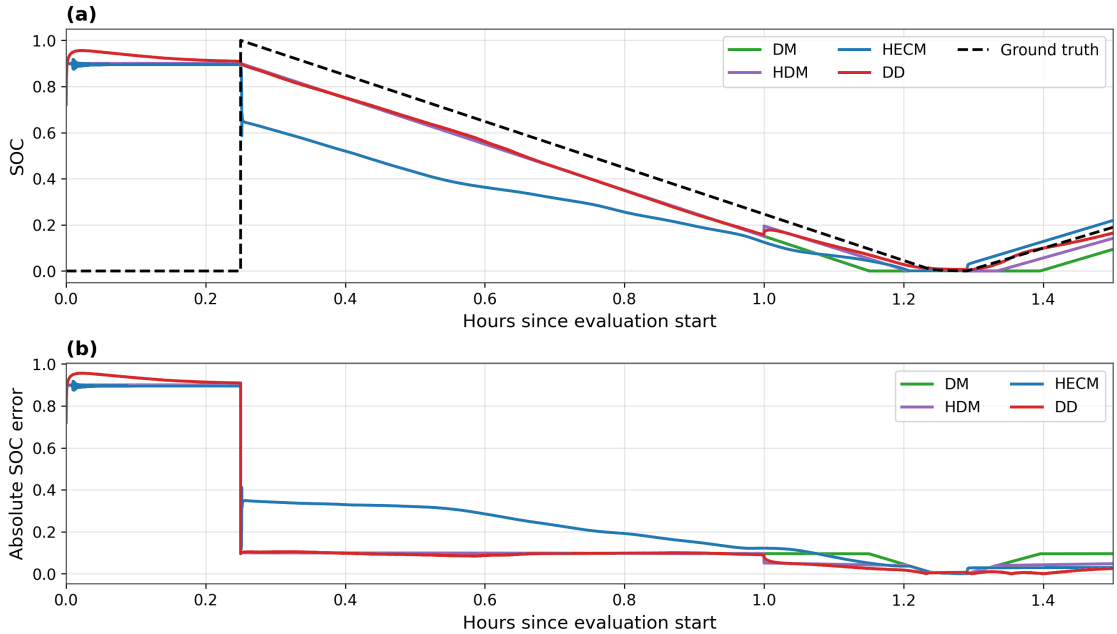


Figure 5.7: Recovery after initialization mismatch in a reset-free evaluation window. The figure shows the disturbed SOC trajectories after the imposed initial-state error together with the corresponding absolute SOC error over time, so that re-synchronization to the baseline-relative recovery bands becomes directly visible. This local view is required because full-run averages alone do not capture restart behavior adequately.

5.4.5 Missing samples, irregular sampling, and burst dropout

This block covers disturbances that affect the continuity and timing of the measurement time series rather than the amplitude of one sensor channel alone. In this context, signal integrity means that the sampled input stream remains complete, time-consistent, and updated. It does not refer only to electrical signal quality. The three cases therefore belong together: individual samples can be frozen, the sampling interval can jitter, or the input stream can be held constant over a longer dropout window.

Missing samples are the strongest global case in this block when the effect is measured by ΔMAE relative to each estimator's own baseline. Under the every-50th-sample freeze, DD changes least with $\Delta\text{MAE} = 0.0016$, whereas DM, HDM, and HECM show larger increases of 0.0076, 0.0079, and 0.0029, respectively. Irregular sampling with ±0,1 s timing jitter remains nearly neutral in the present implementation, with ΔMAE values close to zero for all estimator classes. Burst dropout is different: the ΔMAE values remain moderate, but one central 3600 s frozen-data gap causes long post-gap re-synchronization times for all estimators. The recovery times are 27.35 h for DM, 27.41 h for HDM, 28.56 h for HECM, and 27.44 h for DD. Figures 5.8, 5.9, and 5.10 therefore combine the global ΔMAE summary with transition and recovery views. The underlying metrics are listed in Appendix Tables A.2 and A.3, and the scenario-to-evidence mapping is summarized in Appendix Table A.4.

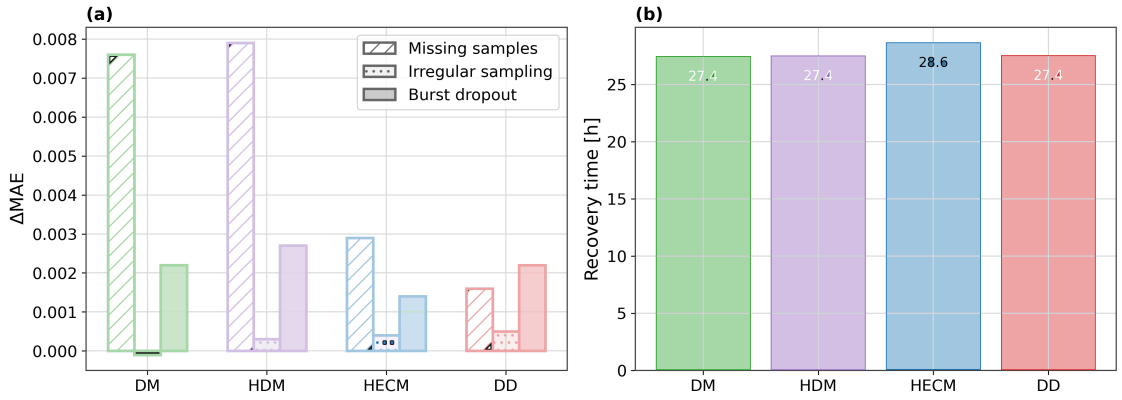


Figure 5.8: Continuity and timing disturbance summary. The figure compares the global ΔMAE penalties for missing samples, irregular sampling, and burst dropout, and it complements these full-run penalties with the burst-dropout recovery times. This compact summary shows why missing samples, timing irregularity, and longer frozen-data gaps cannot be judged from one metric alone.

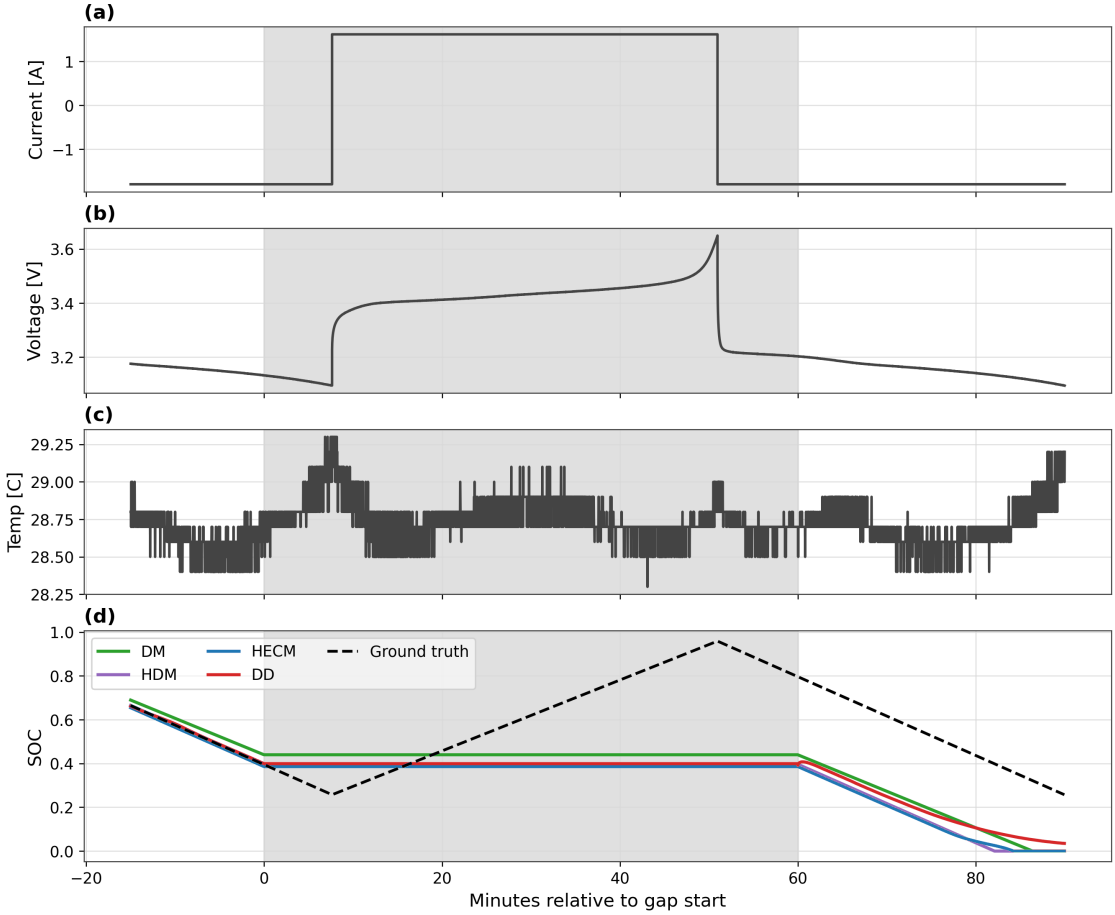


Figure 5.9: Local transition into the burst-dropout interval. The shaded region marks the frozen-data window and makes visible how the estimator outputs behave while the input stream is held constant. This view isolates the immediate local response that is hidden by full-run aggregate metrics.

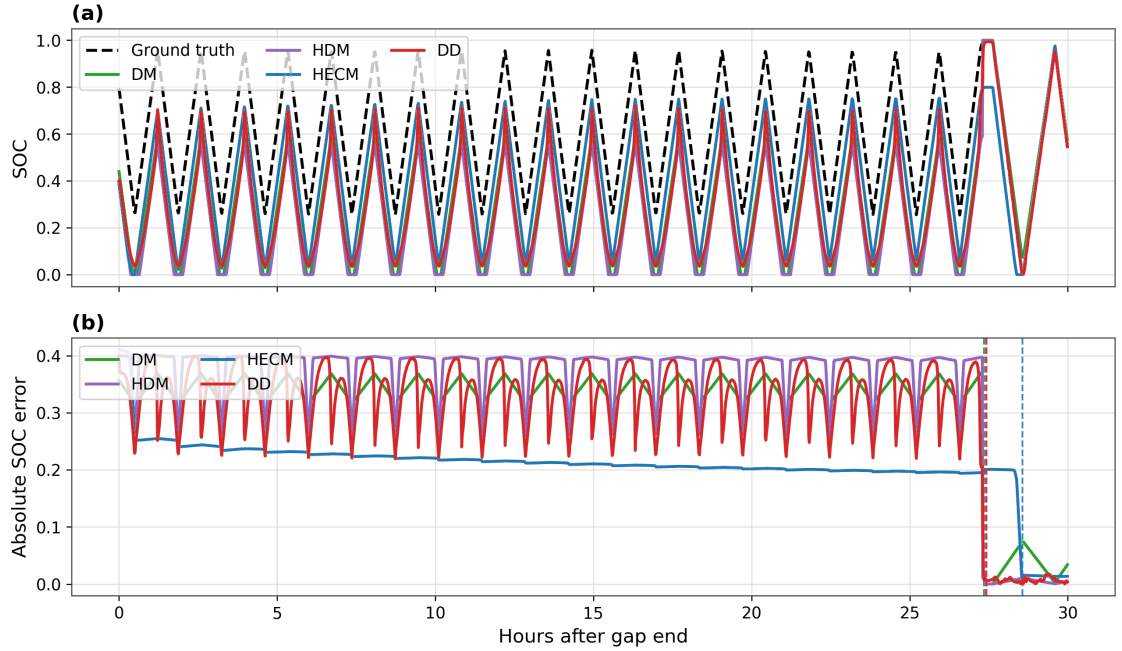


Figure 5.10: Post-gap recovery after burst dropout. The figure shows the disturbed SOC trajectories after the frozen interval together with the corresponding absolute SOC error, so that the delayed return toward the baseline bands becomes directly visible. It therefore complements the moderate full-run ΔMAE values with the practically more revealing local recovery behavior.

5.4.6 Spike sensitivity

Single voltage spikes lead to strongly different local reactions even when the full-run MAE remains nearly unchanged for several classes. The spike protocol injects one ±0,20 V single-sample spike every 1000 samples, corresponding to every 1000 s on the nominal 1 s time base. HECM is almost unaffected under this protocol, whereas DD shows the clearest spike sensitivity in both the representative local window and the aligned post-spike excess-error response. In the global metrics, the DD MAE increases from 0.0100 to 0.0144, while the changes for DM, HDM, and HECM remain smaller or close to their baseline level. The post-spike excess error is evaluated event-wise rather than as a global full-trajectory percentile: for each of the first 40 valid spike events, the SOC-error trajectory is aligned in a window from −60 s to 180 s around the injected spike, the mean absolute error over the final 10 s before the spike defines the local pre-spike baseline, and the largest positive increase above this baseline after the spike is stored as the event severity.

The compact severity value in Figure 5.11 is the 95th percentile of these event-wise peak increases. The figure therefore emphasizes local spike behavior instead of relying only on signed full-run averages. The corresponding global and local metrics are listed in Appendix Tables A.2 and A.3.

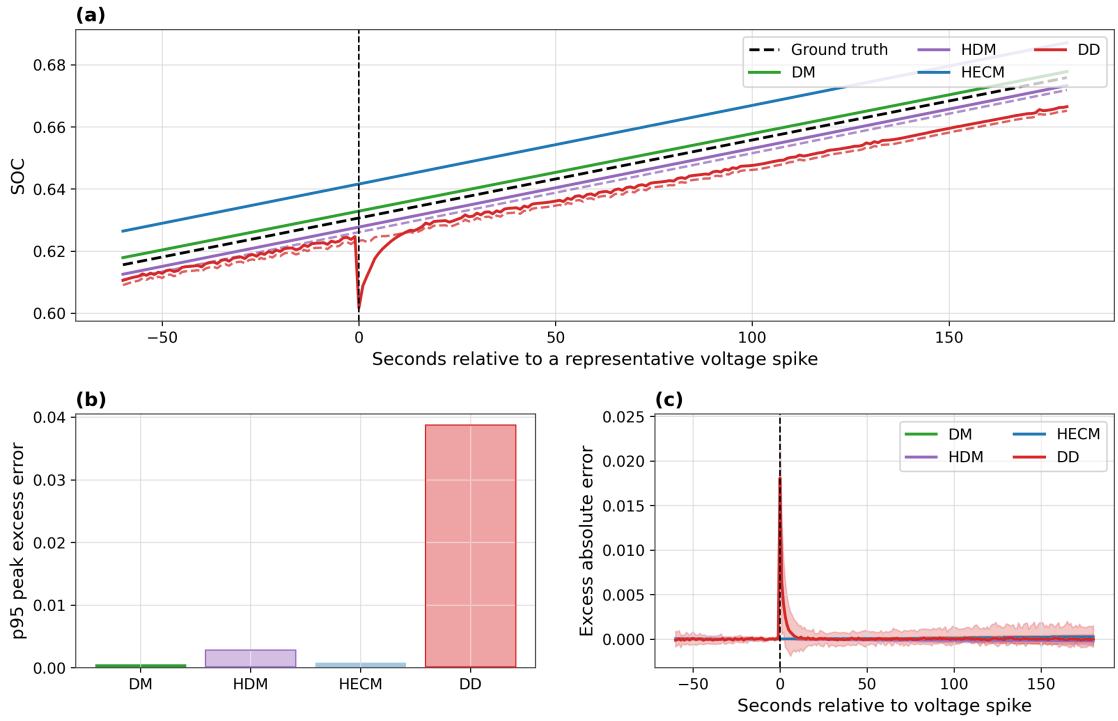


Figure 5.11: Voltage-spike sensitivity. The figure combines a representative local SOC response around an injected spike, a compact spike-severity summary based on the 95th percentile of event-wise post-spike peak excess errors, and the aligned post-spike excess-error behavior across repeated spike events. This local perspective is necessary because isolated spike effects can remain understated in signed full-run averages even when the transient response differs strongly across estimators.

5.4.7 Cross-scenario comparison

The cross-scenario comparison confirms that no single estimator dominates all criteria. HDM is the clean-condition leader, HECM is strongest in the scenarios that most clearly change the estimator ranking, especially persistent current bias and voltage noise, and DD is most attractive when missing-sample stability or recovery after initialization mismatch is prioritized. ADC quantization remains comparatively weak in this benchmark and is kept in the scenario space for complete-

ness rather than because it changes the main ranking. Figure 5.12 condenses the disturbed-scenario ΔMAE values into one heatmap, while Appendix Tables A.2, A.4, and A.5 provide the numerical details. Appendix Figure A.1 adds the dedicated ADC-quantization illustration from the benchmark protocol.

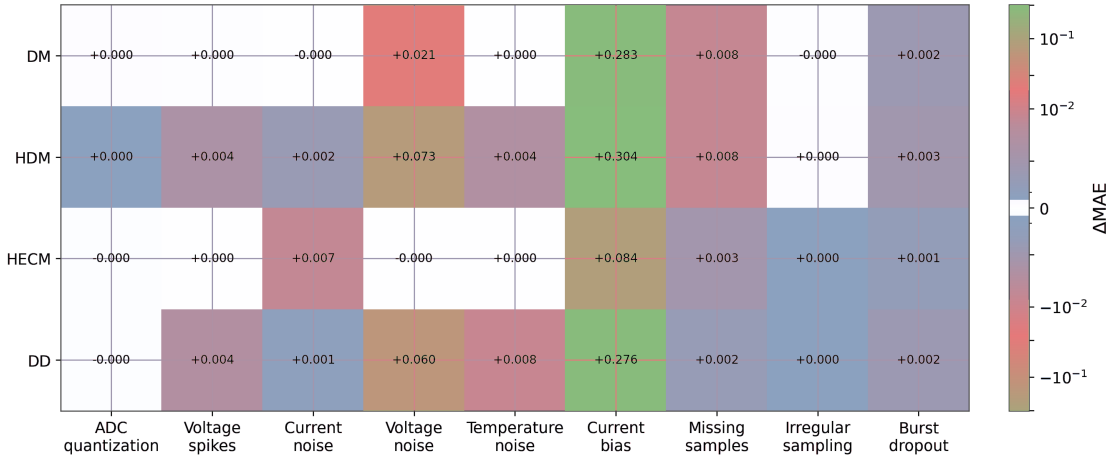


Figure 5.12: Cross-scenario error increase relative to the baseline case. The heatmap condenses the disturbed-scenario penalties, including the ADC-quantization case, and makes estimator-specific ranking shifts visible across disturbance classes. It therefore serves as the compact scenario-overview view for identifying where robustness advantages are stable and where they are disturbance-specific. Error values in the figure are SOC fractions on a 0 to 1 scale. Multiplication by 100 gives percentage points.

5.5 Discussion

5.5.1 Model-specific performance behavior

Because the shared LSTM-based SOH support has already been fixed in the methodology, the following comparison focuses on the SOC-estimator families under a common learned health-context signal rather than comparing independently designed SOH models.

The direct-measurement model behaves as the expected drift-prone reference. Persistent current disturbances accumulate directly into the SOC estimate, and missing information cannot be corrected through an additional feedback path. HDM improves nominal calibration by adapting the effective capacity through the shared LSTM-based SOH context, but it remains structurally exposed to the

same current-integration failure mode and additionally inherits the dynamics of this learned health signal.

HECM benefits from interpretable parameterization, explicit voltage-residual correction, and causal observer structure. Combined with the same shared online SOH support used by the other SOH-aware variants, this explains why it is the overall robustness leader under persistent current bias and voltage noise. The same result does not mean that all noise channels are irrelevant for HECM: current noise can still affect the observer locally through current-dependent parameter scheduling and update terms. DD behaves as a compact context-rich estimator with channel-dependent strengths and weaknesses. It is especially strong under missing samples and initialization mismatch, but more exposed to voltage spikes and to propagated noise in derived inputs such as $\mathrm{d}I/\mathrm{d}t$, Q_c, EFC, and the SOH context. In the present benchmark, this should be read as a design lesson of the compact deployed instance rather than as a hard class limit, because the data-driven family still retains a larger tuning space through architecture, retraining, feature selection, and state-handling choices than the more fixed-form direct or ECM-based alternatives.

5.5.2 Accuracy and Performance Trade-Offs

A central insight of the chapter is therefore that low clean-condition MAE does not imply strong disturbed-operation performance. The trade-off is not only between accuracy and computational complexity, but also between calibration strength, correction capability, recovery speed, disturbance propagation, and the degree to which a model relies on one dominant trigger variable. The benchmark also shows that estimator comparison should not collapse everything into one leaderboard: a deployment-oriented comparison must separate nominal accuracy, disturbance robustness, recovery after inconsistency, and implementation feasibility.

5.5.3 Implications for BMS deployment

If lowest nominal SOC error under clean sensing is the main design priority, HDM is the recommended class because it achieves the lowest baseline MAE of 0.0024 (0,24 %). The corresponding trade-off is strong sensitivity to current bias and the lack of recovery to the baseline bands in the inspected initialization-mismatch window. If the primary priority is best overall robustness under uncertain measurement quality, HECM is the strongest choice because it is most resilient under

current bias, remains nearly unchanged under voltage noise, and still preserves competitive baseline accuracy. The trade-off is the larger flash footprint and slower strict-band recovery than DD. If fastest resynchronization after restart or wrong initial state dominates the design, DD is the strongest option because it shows the fastest strict-band recovery in the local initialization analysis, with the trade-off of greater sensitivity to voltage spikes and weaker performance than HECM under current bias.

If resilience to missing samples and signal freezing dominates the design, DD again becomes attractive because it shows the smallest MAE increase under missing samples and strong post-disturbance stability, while clean-condition accuracy remains below HDM and voltage-driven disturbances require a careful feature-pipeline design. If iterative development headroom and future model refinement dominate the design strategy, DD is also attractive because architecture, features, and training can be updated directly while keeping the deployment target fixed. The present compact instance is still weaker than HECM in the strongest bias- and voltage-driven stress cases, but it retains the clearest path for systematic model iteration. If the main objective is maximum implementation simplicity and the largest resource margin, DM remains the simplest class by far, but it also shows the weakest robustness under biased current measurements and missing-sample stress.

The practical message of the chapter is therefore not that one estimator wins universally. Estimator selection must instead be tied to the disturbance profile, recovery requirements, implementation constraints, and expected model-maintenance strategy of the target BMS. The footprint analysis in Section 5.2.4 further narrows this decision space: all four deployment-prepared variants remain within the common embedded budget, persistent state memory is negligible for all classes, HECM is the tightest flash case, and DM keeps the largest implementation margin.

Table 5.4: Decision-oriented recommendation map derived from the robustness benchmark. The table condenses the scenario-wise findings into dominant deployment priorities, the estimator class most strongly supported by the present evidence, and the corresponding trade-off that must still be considered in design decisions.

Primary deployment priority	Recommended class	Main supporting benchmark evidence	Main trade-off to consider
Lowest nominal SOC error under clean sensing	HDM	Lowest baseline MAE on the benchmark trajectory (0.0024, 0,24 %)	Strong sensitivity to current bias and no return to the baseline bands in the inspected initialization-mismatch window
Best overall robustness under uncertain measurement quality	HECM	Strongest current-bias robustness, nearly unchanged under voltage noise, and competitive baseline accuracy	Larger flash footprint and slower strict-band recovery after initialization mismatch than DD
Fastest re-synchronization after restart or wrong initial state	DD	Fastest strict-band recovery in the local initialization analysis	More sensitive than HECM to voltage spikes and less robust than HECM under current bias
Highest resilience to missing samples and signal freezing	DD	Smallest MAE increase under missing samples and strong post-disturbance stability	Clean-condition accuracy remains below HDM, and voltage-driven disturbances require careful feature-pipeline design

Continued on the next page

Table 5.4 continued from the previous page

Primary deployment priority	Recommended class	Main supporting benchmark evidence	Main trade-off to consider
Largest development headroom for iterative model improvement	DD	Architecture, features, and training can be updated directly while retaining embedded-feasible deployment and strong performance in recovery and missing-sample scenarios	Present compact instance is still weaker than HECM in the strongest bias- and voltage-driven stress cases
Simplest implementation and largest resource margin	DM	Minimal architecture and smallest flash footprint by a wide margin	Weakest robustness under biased current measurements and missing-sample stress

Table 5.4 condenses the benchmark into a deployment-oriented recommendation map so that the reader is not left with a large set of disconnected scenario results. Figure 5.13 complements this table with a compact visual synthesis of the same evidence. The plotted values are relative scores rather than additional physical measurements. For each lower-is-better metric, the four estimator classes are normalized by their minimum and maximum as $(x_{\max} - x)/(x_{\max} - x_{\min})$, so that the best class within the present comparison receives 1 and the weakest class receives 0. The accuracy dimension averages the normalized baseline MAE, RMSE, and P95 error, the robustness dimension averages the normalized ΔMAE values across the disturbed scenarios, and the recovery dimension uses the strict recovery time after the initial-SOC perturbation, with non-recovery in the inspected window treated as the weakest score.

The three profiles in Figure 5.13 use illustrative priority weights for accuracy, robustness, and recovery. The respective weight triples are

$$(0.60, 0.20, 0.20), \qquad (0.20, 0.60, 0.20), \qquad (0.20, 0.20, 0.60).$$

The profiles emphasize accuracy, robustness, and recovery, respectively. They illus-

trate different deployment compromises and do not replace the raw scenario results or the recommendation map in Table 5.4.

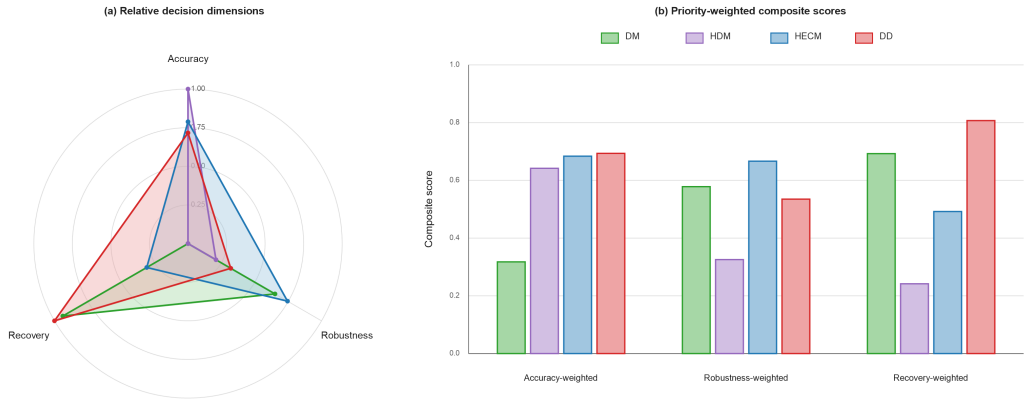


Figure 5.13: Decision-oriented synthesis of the robustness benchmark. Panel (a) collapses the benchmark into three relative decision dimensions, namely nominal accuracy, disturbed-scenario robustness, and recovery behavior, using lower-is-better metrics normalized across the four estimator classes. Panel (b) applies three illustrative deployment-priority profiles and is intended as a visual complement to Table 5.4, not as a replacement for the raw scenario-wise evidence.

5.5.4 Limitations and future extensions

The current benchmark phase is based on one representative dataset trajectory and should later be extended to additional cells, broader ageing states, and cross-cell validation before stronger generalization claims are made. The spike study currently uses isolated spike events. Clustered disturbances, longer spike sequences, and hardware-level disturbance injection may reveal additional model differences. The initialization test for DD remains an online-state proxy based on Q_c perturbation rather than an explicit latent-state reset. This is methodologically justified, but it should still be interpreted as a class-specific approximation. The next benchmark stages should add the remaining extended scenarios, more cells, pack-level experiments, and later hardware-in-the-loop or real embedded runtime measurements while keeping the same performance-oriented evaluation logic.

5.6 Conclusion

Chapter 5 shows that SOC-estimator quality for battery management cannot be reduced to clean-condition accuracy alone. It compares four representative families under the same disturbed measurement protocol. These are direct measurement, SOH-supported direct measurement, hybrid equivalent circuit estimation, and data-driven neural estimation. The purpose is not to declare one model universally best. Instead, the analysis identifies the strengths and weaknesses that become visible when different modelling paradigms are exposed to bias, noise, missing information, timing disturbance, spikes, and initialization mismatch.

The clean-condition ranking alone would point to HDM, because it achieves the lowest baseline error with MAE = 0.0024 (0,24 %). The disturbed scenarios change that interpretation. Under the emphasized disturbance profile, HECM is the strongest overall robustness compromise because its voltage-feedback structure limits several measurement-driven error mechanisms. This becomes clearest under persistent current bias and voltage noise, where HECM remains substantially more stable than the integration-dominated variants. DD does not dominate the full benchmark, but it is the most attractive class when recovery after initialization mismatch or resilience to missing samples is prioritized. DM provides the simplest reference and the largest resource margin, but it exposes the expected vulnerability to drift and missing information.

If one broadly useful estimator class had to be preferred for the particular disturbance profile examined here, the evidence supports the hybrid equivalent-circuit approach as the strongest compromise. Its combination of interpretable physical parameterization, voltage-based correction, and shared data-driven SOH support gives ECM-based estimation a clear role in embedded BMS design. At the same time, the data-driven class remains important because it has the largest development headroom: architecture, features, training data, and state handling can be adapted directly while preserving the same benchmark framework. The present compact DD instance should therefore not be read as the final limit of neural SOC estimation, but as evidence that neural estimators must be trained and feature-engineered to use current, voltage, temperature, timing, and health context in a balanced way instead of relying too strongly on one dominant trigger variable.

The broader implication is that practical BMS evaluation needs more than MAE or another average accuracy score. Average error remains necessary, but it must be complemented by local disturbance response, recovery behavior, tail errors, signal-

continuity sensitivity, and the way disturbed measurements propagate through derived online features. The benchmark is deliberately extensible: the next stages should add more cells, broader ageing states, clustered or longer disturbance events, and eventually hardware-in-the-loop or real embedded runtime validation. Chapter 6 builds on this robustness perspective and turns to recurrent neural SOC and SOH estimation on microcontrollers, where causal state handling, compression, quantization, and runtime feasibility become part of the estimator design itself.

Chapter 6

Embedded Artificial Intelligence for SOC and SOH Estimation on Microcontrollers

This chapter develops the embedded-AI study available as an SSRN preprint and integrates its deployment results with the wider research framework.[3]

6.1 Embedded Deployment Objective

Online battery management requires SOC and SOH estimates that remain accurate while meeting strict memory and timing budgets. This combination is not guaranteed by a low offline test error alone. The estimator must preserve its recurrent state over long streams, use the same feature scaling as during model development, fit into flash and SRAM, and complete every update before the next sample arrives.[97, 98, 18, 15, 26]

The general estimation architectures and their trade-offs are introduced in Chapter 2. Well-calibrated model-based observers and recent sequence models can both provide high accuracy under their respective evaluation conditions.[38, 25, 99, 100] Hybrid studies further combine parameter identification, learned correction, filtering, and recurrent prediction.[101, 102] The remaining question addressed here is not which estimator family obtains the lowest reported error across unrelated datasets, but how two transparent compression routes affect the same stateful SOC and SOH models from workstation replay to microcontroller execution.

Compact battery estimators have also been demonstrated on microcontrollers.[73, 72, 74, 92, 11] Their evidence commonly focuses on one state variable, model family, compression method, or target platform. The remaining gap is a common and inspectable route that applies both structured pruning and weight quantization to

stateful SOC and SOH networks, replays them under the same temporal conditions, and connects their numerical behaviour to linked-firmware memory and measured kernel time. The contribution of this chapter is this comparative deployment benchmark, not a new recurrent cell or a new compression algorithm.

The application setting is modular stationary storage in remote microgrids within the MG-FARM programme.[76, 77, 103, 78] Each storage module must operate autonomously on an STM32H753ZI-class controller. The chapter therefore quantifies how structured hidden-unit pruning and post-training recurrent-weight quantization alter the accuracy, flash footprint, RAM demand, and inference cost of stateful LSTM + MLP estimators under identical data and deployment conditions. Three elements form the core contribution: a traceable PyTorch-to-C path with explicit state and scaling handling, a matched Base–Pruned–Quantized comparison for SOC and SOH, and an evidence-bounded analysis that links architecture-level operation counts to continuous replay and hardware measurements.

6.2 Feature Engineering and Online Processing

All experiments use the LFP DoE Cycle Ageing Dataset and the reference labels described in Section 3.1.[82, 104] The present section defines only the model-specific feature vectors, scaling, state handling, and causal SOH post-processing.

The SOC estimator consumes the six-dimensional feature vector

$$\mathbf{x}_t^{\mathrm{SOC}} = \left[U(t),\, I(t),\, T(t),\, Q_c(t),\, \tfrac{\mathrm{d}U}{\mathrm{d}t}(t),\, \tfrac{\mathrm{d}I}{\mathrm{d}t}(t)\right], \tag{6.1}$$

The derivative channels are causal timestamp-aware backward differences. For sample $k > 0$,

$$\Delta t_k = \max(t_k - t_{k-1}, 10^{-6}), \tag{6.2}$$

$$\left.\frac{\mathrm{d}U}{\mathrm{d}t}\right|_k = \frac{U_k - U_{k-1}}{\Delta t_k}, \qquad \left.\frac{\mathrm{d}I}{\mathrm{d}t}\right|_k = \frac{I_k - I_{k-1}}{\Delta t_k}. \tag{6.3}$$

Only the preceding sample and timestamp are required, so the online calculation has constant cost. For the reported STM32 benchmark, however, the derivative and cumulative channels were prepared on the host and transmitted as part of the complete feature vector. The measured neural-kernel time consequently excludes sensor acquisition and feature construction. This boundary isolates the effect of

model compression from that of the feature front end.

The SOH estimator uses a slower-varying feature stack,

$$\mathbf{x}_t^{\mathrm{SOH}} = \Big[t,\, U(t),\, I(t),\, T(t),\, \mathrm{EFC}(t),\, Q_c(t)\Big], \tag{6.4}$$

with cumulative timestamp t, equivalent full cycles $\mathrm{EFC}(t)$, and cumulative charge state $Q_c(t)$. The EFC definition based on absolute charge throughput is given in Equation 5.16. All channels are robustly normalised with the training-set median m_k and interquartile range r_k,

$$\tilde{x}_{t,k} = \frac{x_{t,k} - m_k}{r_k + \varepsilon}, \qquad \varepsilon = 10^{-6}, \tag{6.5}$$

which reduces sensitivity to outliers and keeps host-side replay and MCU execution numerically aligned.[105, 106]

The SOC and SOH labels follow the Coulomb-counting and capacity-check-up procedures in Section 3.1. Their definitions are given in Equations 3.6 and 3.4. Sustained constant-voltage full-charge phases and periodic complete capacity tests provide information that is unavailable during ordinary online operation. These quantities are therefore targets for training and scoring, not estimator inputs. During deployment, only the measurement-derived feature stream and the internal hidden and cell states are available. Both states are propagated sample by sample without sliding windows or resets at artificial segment boundaries.

The displayed SOH trajectory passes through two causal post-processing stages. Each raw prediction $\hat{y}_t$ is first aligned to the initial label by $\hat{y}_t^{(0)} = (y_0/\hat{y}_0)\hat{y}_t$. Stage 1 restricts positive and negative proposal changes to

$$\ell_t = \min\Big(10^{-4}|y_{t-1}^{(1)}|, 10^{-5}\Big) \tag{6.6}$$

and then applies an exponential moving average with $\alpha_1 = 0.02$. Stage 2 processes this complete intermediate trajectory with $\alpha_2 = 10^{-6}$ and limits only downward changes,

$$e_t^{(2)} = (1-\alpha_2)e_{t-1}^{(2)} + \alpha_2 y_t^{(1)}, \qquad y_t^{(2)} = \max\Big(e_t^{(2)}, y_{t-1}^{(2)} - 2\times 10^{-8}\Big). \tag{6.7}$$

For a linear exponential moving average sampled every Δt, the equivalent time

constant and cutoff frequency are

$$\tau = -\frac{\Delta t}{\ln(1-\alpha)}, \qquad f_c = \frac{1}{2\pi\tau}. \tag{6.8}$$

At 1 Hz, stage 1 has $\tau = 49{,}5\,\mathrm{s}$, $f_c = 3{,}22\,\mathrm{mHz}$, and an EMA-only 90 % response time of about 114 s. Stage 2 corresponds to $\tau = 11.57$ days, $f_c = 1.59\times10^{-7}$ Hz, and a 90 % response time of 26.65 days. These frequency values apply only to the linear averaging elements. The rate limiters are nonlinear and cannot be represented by one cutoff frequency. Figure A.4 later separates the influence of compression from the accuracy and delay introduced by these stages.

Figure 6.1 summarizes the stateful LSTM + MLP structure used for both tasks.

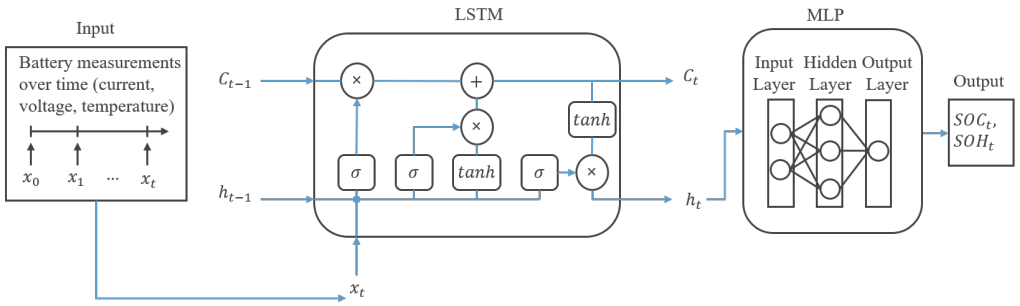


Figure 6.1: Stateful LSTM + MLP estimator structure used for both SOC and SOH estimation. Preprocessed feature streams are processed step by step by a single-layer LSTM whose hidden and cell states are propagated across time. A lightweight MLP head then maps the hidden representation to either SOC or SOH. The SOC instance uses hidden size 64 with an MLP width of 64 and a sigmoid output, the SOH instance hidden size 128 with an MLP width of 128 and a linear output. This structure is the basis for the later pruning, quantization, and MCU export steps.

6.3 LSTM Architecture and Training Setup

Both SOC and SOH estimators employ the LSTM update defined in Equation 2.14, but with task-specific feature sets and sizes, as summarised in Figure 6.1. The SOC model uses a single-layer LSTM with hidden size $H = 64$ that consumes a six-dimensional feature vector $\{U, I, T, Q_c, \mathrm{d}U/\mathrm{d}t, \mathrm{d}I/\mathrm{d}t\}$. Its MLP head contains one ReLU-activated hidden layer of width 64 and a sigmoid output neuron that constrains the estimate to $[0, 1]$. The SOH model uses the same overall structure

with hidden size $H = 128$ and an MLP hidden width of 128. It consumes the feature vector $\{t, U, I, T, \mathrm{EFC}, Q_c\}$ and ends in a linear output neuron because the slowly varying SOH target does not benefit from a saturating output activation.

In contrast to the hourly aggregated SOH estimator of Chapter 5, both models operate directly on the sample-wise 1 Hz stream. The hidden and cell states are propagated continuously between samples. The implementation follows the standard PyTorch gate order and reproduces the same matrix operations, state update, and activation functions in C.[32, 59] The application-specific architecture in Figure 6.1 therefore extends the generic cell in Figure 2.3 by adding the six-channel feature input and the task-specific MLP output heads.

For input dimension $D = 6$, hidden size H, and MLP width M, the weight-related multiply–accumulate count of one streaming update is

$$N_{\mathrm{MAC}}(H) = 4H(D + H) + HM + M. \tag{6.9}$$

This count excludes bias additions, activation functions, indexing, and memory traffic. If a fraction p of the recurrent channels is removed, the retained dimension is $H_p = (1 - p)H$. The corresponding analytical reduction follows as

$$\begin{aligned} R_{\mathrm{MAC}}(H, p) &= 1 - \frac{N_{\mathrm{MAC}}((1-p)H)}{N_{\mathrm{MAC}}(H)} \\ &= \frac{4(2p - p^2)H^2 + p(4D + M)H}{4H^2 + (4D + M)H + M}, \\ \lim_{H\to\infty} R_{\mathrm{MAC}}(H, p) &= 2p - p^2. \end{aligned} \tag{6.10}$$

The quadratic recurrent contribution is reduced more strongly than the linear input and MLP terms. At the evaluated $p = 0.30$, the large-model limit is 51 %. The actual integer dimensions yield a reduction from 22,080 to 12,124 MACs for SOC and from 85,120 to 46,208 MACs for SOH, corresponding to 45.1 % and 45.7 %. Figure A.7 visualizes the finite-size effect and the general dependence on pruning fraction.

This restriction to a single-layer LSTM + MLP architecture is deliberate. The focus is not an extensive architecture search over GRUs, Transformers, or deeper recurrent stacks, but a transparent and reproducible embedded tool flow for stateful estimators whose hidden-state handling can later be re-implemented faithfully in C. A single-layer LSTM reduces to one compact gate computation per step and keeps pruning, quantization, and firmware export directly traceable. This differs from the

shared SOH context model used in Chapter 5, which is an hourly sequence model with feature projection, a two-layer LSTM, residual refinement, and a deployment-prepared hidden size of 112 for the robustness benchmark. Chapter 5 uses that model to provide a common health context for comparing SOC-estimator families. The present chapter instead asks how stand-alone recurrent SOC and SOH estimators can be compressed, exported, and measured on the microcontroller. The two architectures therefore serve different experimental purposes, and their results should not be interpreted as a direct architecture ranking.

Both SOC and SOH models are trained with mean-squared-error loss against the corresponding reference label at each training sample, while validation additionally tracks MAE, RMSE, and R^2. Model parameters, configurations, and optimiser states are retained so that the subsequent compression and export stages can be reproduced deterministically. Training is carried out on NVIDIA A100-class GPU hardware in mixed precision (FP16), and both tasks use the AdamW optimiser, cosine-annealing warm restarts, gradient clipping, and mini-batch training.[107, 108, 109, 33] For SOC, the learning rate is 1.5×10^{-4}, the batch size is 512, and the sequence chunk length is 2024. For SOH, the learning rate is 2.0×10^{-4}, the batch size is 128, and the chunk length is 2048. Although deployment is fully stateful and step-wise, training uses truncated backpropagation through time over these chunks: within each chunk, the hidden and cell states are propagated exactly as in deployment, so the stateful update mechanism is already learned during optimisation while the chunking still enables randomized sampling, faster convergence, and better generalization across the diverse ageing trajectories.

The base models are first evaluated in streaming mode on the full ageing trajectory of a representative LFP cell and serve as the numerical reference for all subsequent experiments. The same trained parameter sets are then pruned, quantized, and exported to STM32 firmware. Identical feature streams ensure that accuracy, flash, RAM, latency, and the timing-derived energy proxy refer to the same stateful sequence. Figure 6.2 summarizes this end-to-end path from training to embedded benchmarking.

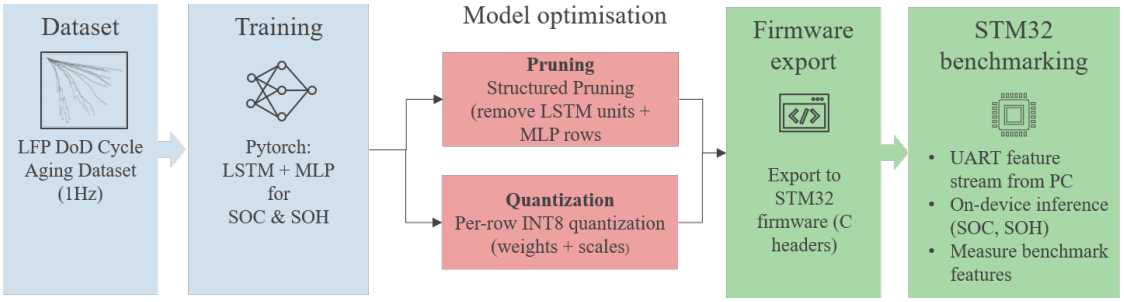


Figure 6.2: End-to-end deployment pipeline from the LFP ageing dataset and PyTorch training to pruning, quantization, C-header export, and final STM32 benchmarking. The same trained SOC and SOH parameter sets form the basis of the floating-point, pruned, and quantized deployments, which keeps the later comparisons numerically consistent.

6.4 Structured Pruning Configuration

The grouped magnitude criterion in Equation 2.23 and the local ranking principle in Equation 2.26 are instantiated here as one-shot structured post-training pruning. The implementation removes complete LSTM channels so that the exported tensors remain dense and the recurrent state becomes smaller. This structure directly matches the manually implemented C kernel and avoids an irregular element-wise sparsity mask. For hidden unit h, the implemented score combines the L2 norms of its input and recurrent rows across all four gates,

$$s_h = \sum_{g\in\{i,f,o,g\}} \left(\|W_{\text{ih}}^{(g)}[h,:]\|_2 + \|W_{\text{hh}}^{(g)}[h,:]\|_2 \right), \tag{6.11}$$

which follows the gate-grouping principle used in structured recurrent-network pruning.[47, 110, 111]

After sorting the scores, the retained index set $\mathcal{R}$ contains the H' highest-ranked units. The score itself uses only the input and recurrent gate rows. Once a channel is removed, the same index must also be removed from both LSTM bias vectors, the recurrent input dimension, the hidden and cell states, and the input dimension of the MLP head. The dense reduced tensors are formed by

$$\begin{aligned} \widetilde{W}_{\text{ih}}^{(g)} &= W_{\text{ih}}^{(g)}[\mathcal{R},:], \\ \widetilde{W}_{\text{hh}}^{(g)} &= W_{\text{hh}}^{(g)}[\mathcal{R},\mathcal{R}], \\ \widetilde{W}_{\text{MLP},1} &= W_{\text{MLP},1}[:,\mathcal{R}]. \end{aligned} \tag{6.12}$$

The same indices therefore remove gate rows, recurrent columns, and the associated MLP inputs. Figure 6.3 makes the scoring and slicing process numerically traceable through a reduced example with $H = 3$ and $D = 2$. The displayed coefficients are illustrative rather than weights taken from the trained estimators, while the arrangement of the gate blocks and all removal operations follow the implementation. For each candidate channel, the L2 saliency score is the sum of its four input-weight row norms and four recurrent-weight row norms. A small score therefore indicates a channel with low aggregate weight magnitude. In the example, channel h_2 has the smallest score of 0.15, compared with 4.46 for h_1 and 5.23 for h_3. Selecting h_2 removes the highlighted row from every gate block and the corresponding recurrent and MLP input columns. The retained set $\mathcal{R} = \{h_1, h_3\}$ then forms new dense tensors with $H' = 2$.

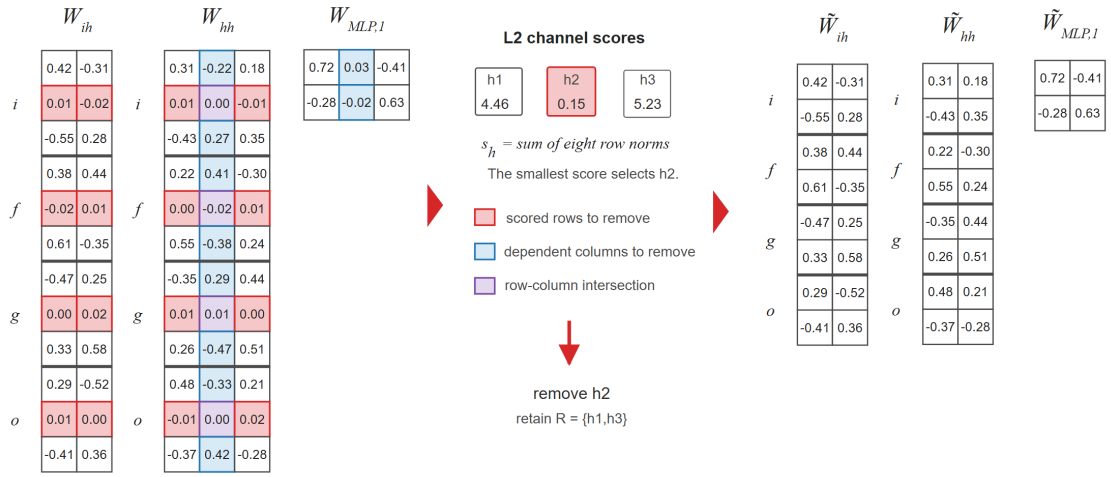


Figure 6.3: Illustrative numerical example of gate-consistent LSTM pruning. Red cells identify the rows included in the L2 saliency score, blue cells identify dependent columns that must be removed with the selected channel, and purple cells belong to both groups. Removing the lowest-scoring channel h_2 and retaining $\mathcal{R} = \{h_1, h_3\}$ produces the smaller dense tensors shown on the right. The numerical values are provided to make the scoring principle transparent, while the evaluated SOC and SOH estimators apply the same operations to their full hidden dimensions.

SOC is reduced from 64 to 45 units and SOH from 128 to 90 units, in both cases approximately 30 %. This fraction is one moderate operating point rather than the result of an exhaustive rate search. It was chosen to provide a substantial dimensional reduction while retaining useful accuracy. Determining an optimum

would require separate pruning and fine-tuning runs for several candidate fractions.

The transformation is a one-shot structured removal followed by brief post-pruning fine-tuning. It is not an iterative pruning schedule and does not use pruning-aware training. The L2 score was selected because it is deterministic once the model is trained, groups the four gate rows consistently, and maps directly to smaller dense arrays. Gradient-based criteria would additionally require training data and a backward pass, while activation-based scores would depend on a calibration stream and a temporal aggregation rule. No experimental ranking of these alternatives was performed, so the L2 criterion is treated as a reproducible implementation choice rather than a universally optimal method. Figures A.2 and A.8 document the selected channels and the scope of this rationale.

6.5 INT8 Quantization and Manual STM32 Export

The general quantization design space is introduced in Section 2.5.2 and summarized in Figure 2.7. The implementation evaluated here retains the Base topology but changes the stored representation of the two recurrent matrices W_{ih} and W_{hh}.[13, 66, 112] Each row r of either FP32 matrix is mapped independently,

$$s_r = \max\left(\frac{\max_k |(W^{\mathrm{fp32}})_{r,k}|}{127}, \varepsilon_q\right), \tag{6.13}$$

$$(W^{\mathrm{int8}})_{r,k} = \mathrm{clip}\left(\mathrm{round}\left(\frac{(W^{\mathrm{fp32}})_{r,k}}{s_r}\right), -127, 127\right), \tag{6.14}$$

$$(\widehat{W})_{r,k} = s_r (W^{\mathrm{int8}})_{r,k}. \tag{6.15}$$

The lower bound ε_q protects an all-zero row. Although the signed INT8 data type spans -128 to 127, the symmetric quantizer uses only $\{-127, \ldots, 127\}$ and leaves -128 unused. This keeps zero exact and gives equal positive and negative ranges. With nearest-integer rounding and the row maximum as scale, finite values do not clip and satisfy

$$\left|(W^{\mathrm{fp32}})_{r,k} - (\widehat{W})_{r,k}\right| \leq \frac{s_r}{2}. \tag{6.16}$$

Figure 6.4 shows the row-wise transformation and the resulting export representation. Every matrix row receives an individual FP32 scale. The recurrent weights are stored as INT8 codes, while scales and biases are exported separately for the manually implemented kernel.

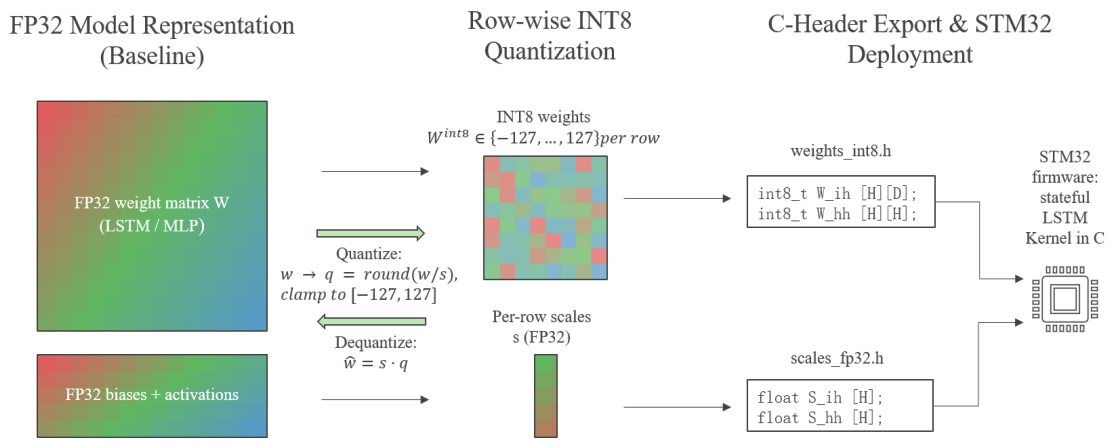


Figure 6.4: Row-wise weight-only INT8 quantization and embedded export used for the recurrent matrices. Floating-point matrix rows are represented by integer codes and individual floating-point scales. Biases and activations remain in floating point, while the stored recurrent weight matrices require fewer bytes.

The complete mixed-precision gate pre-activation can be written as

$$\mathbf{a}_t = \mathbf{s}_{\text{ih}} \odot \left(W_{\text{ih}}^{\text{int8}} \mathbf{x}_t\right) + \mathbf{s}_{\text{hh}} \odot \left(W_{\text{hh}}^{\text{int8}} \mathbf{h}_{t-1}\right) + \mathbf{b}_{\text{fp32}}. \tag{6.17}$$

In the inspected C kernels, each INT8 recurrent weight is converted to FP32 and multiplied by its row scale inside the accumulation. Only W_{ih} and W_{hh} use INT8 storage. The scales, biases, hidden and cell states, gate activations, MLP weights and arithmetic, and final output remain FP32. The recurrent matrices account for about 80 % of the raw Base constants, so this boundary targets the dominant persistent storage while avoiding an additional quantization source in the recurrent state. It is therefore a weight-only mixed-precision implementation, not a full-integer network, and it cannot deliver the RAM and arithmetic benefits of activation and state quantization. Figure A.9 makes this precision boundary explicit.

The exported parameters are written as C headers and integrated into manually implemented stateful LSTM + MLP firmware. This route exposes all scale operations and state updates and avoids ambiguity in generated support for continuous recurrent state.[113, 114, 115] Streaming checks against the source implementation typically produce MAE differences below 10^{-4}.

6.6 Benchmark Methodology and KPI Logic

The hardware benchmark uses an STM32 Nucleo-144 development board with an STM32H753ZI microcontroller, 2 MB flash, and 1 MB SRAM.[116, 11] For each task, the Base FP32 model, the structured-pruned FP32 model, and the INT8-weight mixed-precision model receive the same ordered feature stream. The recurrent states are not reset during the complete replay.

The host analysis produces trajectory errors, distributions, and local views. It also divides the naturally ordered output into ten equal-count segments without resetting the hidden or cell state. Target MAE and P95 are calculated in every segment, together with the mean absolute difference between each compressed output and Base. This separates a general increase in task difficulty later in the sequence from a compression-specific drift. The corresponding evidence is collected in Figure A.3.

6.6.1 Limited Software-Level Fault Sensitivity

Two bounded software experiments extend the nominal replay. First, randomly unavailable output reports at rates of 1, 5, and 10 % are replaced by the most recent valid estimate while the underlying inference continues. The same hold-last policy is tested for ten deterministic gaps of 60, 600, and 3600 samples. This experiment addresses report availability rather than a disturbance of the recurrent computation.

The second experiment inserts a one-sample fault before inference and follows its propagation through the recurrent state. Each exported C model processes a continuous 8000-sample segment from one representative evaluation trajectory at 1 Hz. After at least 2048 clean warm-up samples, 30 independent events are distributed over the stream. Ten affect voltage, ten current, and ten temperature. Bits are indexed from zero at the least significant position. Bit 22 is therefore the highest-order bit of the 23-bit FP32 fraction field. It is flipped for one input value and one sample only, leaving sign and exponent unchanged. Clean and disturbed branches start from the same recurrent state and then receive the same 60 unmodified inputs without resetting the LSTM.

With $k = 0$ denoting the affected sample and $k = 1, \ldots, 60$ the clean continuation, the window contains 61 values and spans a 60-s recovery horizon. The

disturbed-to-clean deviation is

$$d_{e,k} = 100 \left| \hat{y}_{e,k}^{\text{fault}} - \hat{y}_{e,k}^{\text{clean}} \right| \tag{6.18}$$

in percentage points. Peak deviation, the residual after 60 s, and the first sustained return below $\max(0.1 d_{e,\text{peak}}, 10^{-4}\,\text{pp})$ characterize propagation independently of the ordinary target error. The associated accuracy change is

$$\Delta\text{MAE}_{61,e} = \frac{100}{61} \sum_{k=0}^{60} \left(\left| \hat{y}_{e,k}^{\text{fault}} - y_{e,k} \right| - \left| \hat{y}_{e,k}^{\text{clean}} - y_{e,k} \right| \right). \tag{6.19}$$

A positive value means that the fault increases target MAE. A negative value can occur when an imperfect disturbed prediction happens to move closer to the label.

6.6.2 STM32 Measurement Boundary and Indicators

Feature vectors are sent by UART to dedicated SOC and SOH firmware. End-to-end latency covers transmission, scheduling, and return of the estimate. Neural-kernel time is measured around the LSTM and MLP with the data watchpoint and trace cycle counter and averaged over 10,000 streaming updates. This interval includes instructions and memory accesses executed by the kernels but excludes UART transfer, host processing, sensor acquisition, and host-side feature construction.

Six-feature scaling is inside the timed SOC call. For SOH, scaling occurs immediately before the measured interval in Base and Pruned, while the Quantized forward path includes it. This small affine cost was not separated and is absent from the static operation counts. The inspected projects use the hard-float interface and Cortex-M7 floating-point unit with compiler optimization disabled. The results therefore describe transparent reference kernels rather than a production-optimized integer library.

For normalized label y_n and estimate $\hat{y}_n$, the signed percentage-point error is $e_n = 100(\hat{y}_n - y_n)$. The reported metrics are

$$\text{MAE} = \frac{1}{N} \sum_{n=1}^{N} |e_n|, \qquad \text{RMSE} = \sqrt{\frac{1}{N} \sum_{n=1}^{N} e_n^2}, \tag{6.20}$$
$$\text{P95} = Q_{0.95}\left(\{|e_n|\}_{n=1}^{N} \right),$$

where P95 is the absolute-error threshold below which 95 % of samples lie. MAE and RMSE include every sample.

Flash is read from the linked firmware image. RAM combines initialized and zero-initialized static storage with the largest observed streaming stack use. Energy was not measured separately for each model. The reported quantity is the timing-derived proxy

$$E_{\text{est}} = P_{\text{assumed}} t_{\text{inf}}, \qquad P_{\text{assumed}} = 0{,}5\,\text{W}. \tag{6.21}$$

The power value is a representative assumption for the complete development board. Hence, relative changes in E_{est} are identical to changes in kernel time. The proxy cannot separate processor arithmetic, flash and SRAM access, UART activity, regulator losses, or other board consumers.

Table 6.1: STM32 engineering KPIs for the SOC deployments under full-stream replay. The table separates end-to-end latency from DWT-measured kernel time. Energy is the proxy from Equation 6.21, not an independent power measurement.

Model	Latency [ms]	Inference [ms]	Flash [KB]	RAM [KB]	E_{est} [mJ]
Base FP32	12.70	1.40	105.32	4.93	0.70
Pruned FP32	12.20	0.80	62.27	4.03	0.40
Quant INT8	18.48	6.99	52.48	3.96	3.49

Table 6.2: STM32 engineering KPIs for the SOH deployments under full-stream replay. Energy is estimated with the same constant-power timing proxy used for SOC.

Model	Latency [ms]	Inference [ms]	Flash [KB]	RAM [KB]	E_{est} [mJ]
Base FP32	34.88	22.73	335.00	8.69	11.37
Pruned FP32	24.69	12.72	182.41	6.96	6.36
Quant INT8	41.23	29.21	138.00	6.70	14.60

6.7 Streaming Prediction Accuracy

The evaluation starts with full-sequence streaming replay over the complete ageing trajectory. Figures 6.5 and 6.6 therefore provide the central long-horizon view of

how the dense, pruned, and quantized deployments behave under identical stateful replay conditions. The PC reference replay and the exported STM32 firmware process the same ordered feature stream with the same stateful update logic, so the reported values are full-stream averages over the same trajectory rather than short-window averages from selected excerpts. Differences between variants therefore reflect pruning, quantization, and numerical representation rather than a change of evaluation data.

For SOC, the tracking remains tight across the entire dynamic range. The residuals stay centred around zero, and the quantized model overlaps the full-precision baseline so closely that the two traces are often visually indistinguishable. The full-stream SOC MAE values are 2,68 % for Base, 2,34 % for Pruned, and 2,79 % for Quantized. These are percentage-point errors on the 0–100 % SOC scale. The corresponding RMSE values are 3,81 %, 3,13 %, and 3,88 %. Thus, recurrent-weight quantization retains the useful stateful behaviour of the original network over the evaluated sequence. Pruned produces the lowest aggregate error for this particular trained model and data split, although some dynamic sections show a larger deviation from Base.

For SOH, degradation must be inferred from a much slower signal. The capacity tests that define the labels are not presented to the estimator, whereas open-circuit-voltage sweeps, pulse tests, profile sections, and ordinary operating data remain in the causal input stream. The displayed trajectories pass through the complete post-processing chain defined around Equation 6.7, including initial alignment, the first EMA with its symmetric step limiter, and the second EMA with the downward-rate restriction. After this processing, the full-stream SOH MAE values are 0,85 % for Base, 1,46 % for Pruned, and 1,41 % for Quantized. The respective RMSE values are 1,15 %, 1,87 %, and 1,89 %. All three variants reproduce the long-term loss of capacity, but both compressed versions have a higher aggregate error than Base in this evaluation.

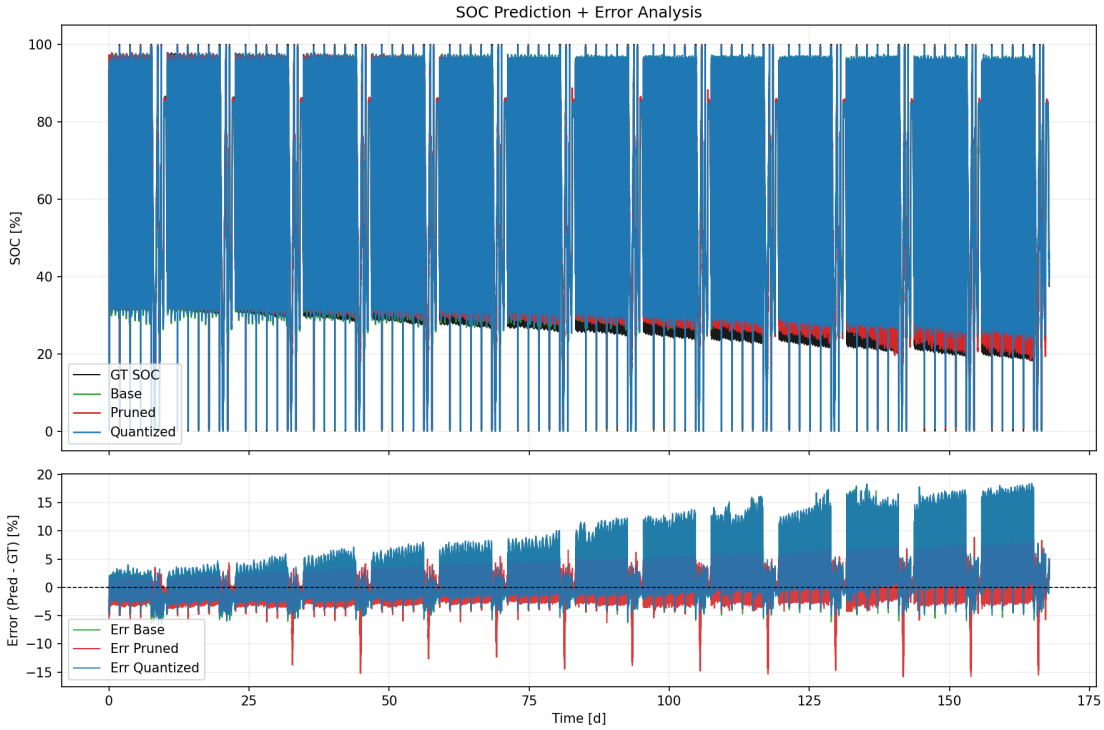


Figure 6.5: Streaming SOC dashboard for the base, pruned, and quantized SOC deployments over the full ageing trajectory. The upper panel compares the three stateful predictions with the SOC reference label, while the lower panel shows the instantaneous prediction error. The near-overlap of the dense and quantized traces is one of the central observations of the chapter.

6.8 Temporal Stability and SOH Filter Interaction

The complete output sequence is divided into ten consecutive parts of equal sample count to distinguish ageing-related changes in task difficulty from a steadily growing compression error. Recurrent states continue across every boundary. For SOC, the MAE of Base rises from 1.116 percentage points in the first part to 4.538 percentage points in the final part. Quantized changes from 1.234 to 4.600 percentage points over the same interval. Despite this common rise in target error, the mean absolute difference between Quantized and Base decreases from 0.332 to 0.271 percentage points. The corresponding Pruned-to-Base difference increases from 0.924 to 2.592 percentage points, even though Pruned has the smallest MAE over the complete trajectory. The SOH segment errors vary with the ageing phase, but neither compressed estimator develops a continuously increasing distance from

Base. Figure A.3 contains the complete segment-wise comparison.

The filter study evaluates every SOH model before post-processing, after stage 1, and after the final stage using an identical input order. Stage 1 yields MAE values of 1.677, 1.644, and 1.926 percentage points for Base, Pruned, and Quantized. The final values for this controlled sequence are 1.476, 1.695, and 1.028 percentage points. Accordingly, post-processing can alter both the numerical error and the ordering of the variants. Figure A.4 shows this interaction together with the response characteristics calculated in Equation 6.8. The final filter can reduce short-term variation substantially, but its EMA-only 90 % response time of 26.65 days at 1 Hz must be considered together with the reported accuracy.

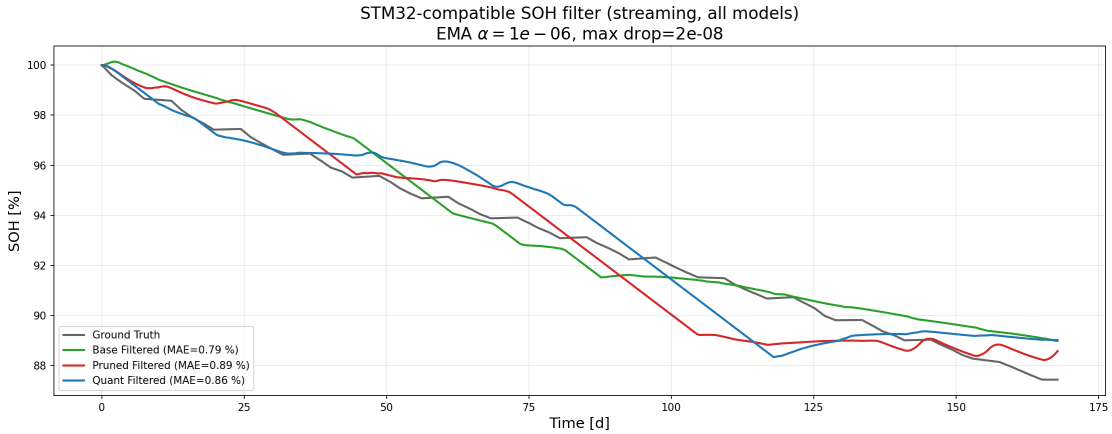


Figure 6.6: Streaming SOH dashboard with causal filtering for the three deployment variants. The upper panel shows the reference SOH together with the base, pruned, and quantized predictions after online smoothing, while the lower panel displays the corresponding prediction errors. The figure highlights the much slower degradation dynamics and the stronger role of causal post-processing in the SOH task.

6.9 Error Distribution and Local Transient Behaviour

Figures 6.7 and 6.8 analyse the statistical distribution of the prediction errors. For SOC, the distribution is narrow and approximately symmetric. The median absolute errors are 1,81 % for Pruned, 1,89 % for Base, and 2,03 % for Quantized. The pruning diagnostics in Figure A.2 verify that the 19 removed channels are the lowest-ranked channels under the defined L2 saliency criterion and compare the central recurrent-weight distributions before and after removal. These observations establish the implemented selection and the lower error of this particular Pruned

model, but they do not prove a regularisation mechanism or statistical superiority across training initialisations. For SOH, the distribution is broader, which is consistent with the greater difficulty of estimating capacity fade from operational data. Nevertheless, most errors remain within an approximately ±2% band around the reference trajectory.

Global metrics alone would still hide local failures. The chapter therefore also zooms into a high-dynamic pulse segment between 30k and 40k seconds (Figure 6.9) and into a representative check-up sequence between 657k and 897k seconds (Figure 6.10). In the pulse region, the quantized model follows the reference transients with high fidelity and captures voltage-recovery phases accurately, whereas the pruned model smooths some sharper local features. In the check-up phase, all variants remain stable over the extended charge and discharge interval. These local views complement the segment-wise temporal analysis rather than replacing it.

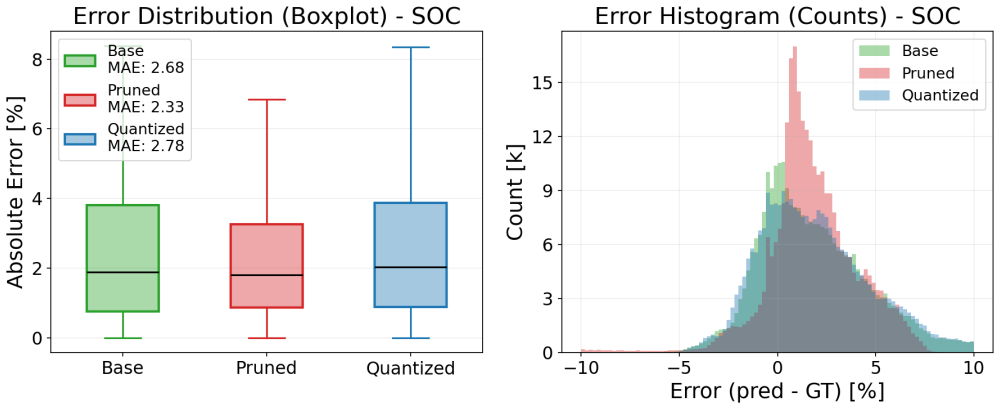


Figure 6.7: SOC error distribution for the embedded SOC deployments, including MAE boxplots and an error histogram. The narrow spread around zero confirms unbiased estimation, while the boxplots make visible that the pruned SOC model attains the smallest median error and interquartile range.

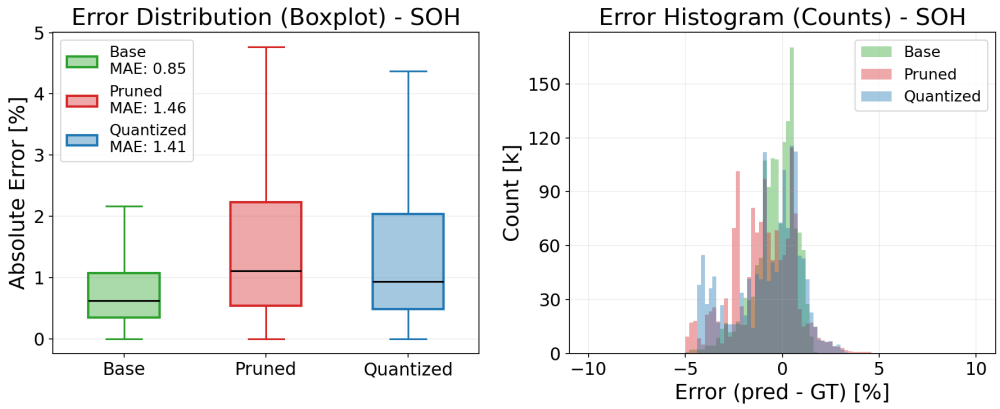


Figure 6.8: SOH error distribution for the embedded SOH deployments, including MAE boxplots and an error histogram. The spread is naturally broader than for SOC, but the majority of errors remains within a practically useful tolerance band around the reference trajectory.

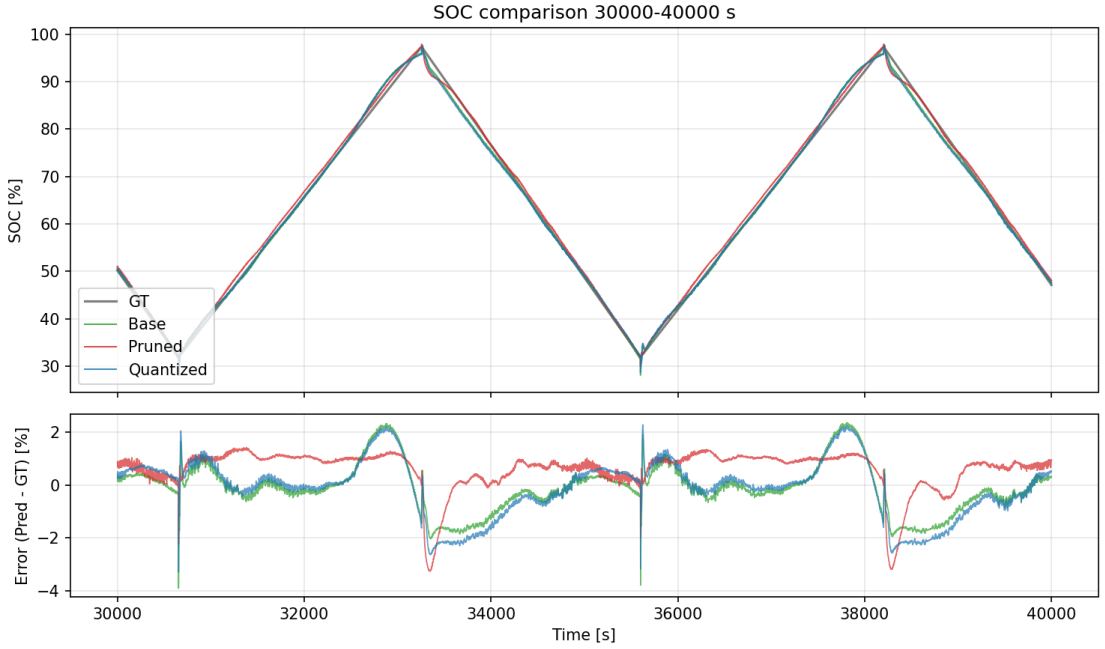


Figure 6.9: Detailed pulse-region analysis of embedded SOC streaming replay. The figure focuses on a highly dynamic load segment and shows how the three deployment variants capture fast transients and voltage-recovery behaviour under identical stateful inference conditions.

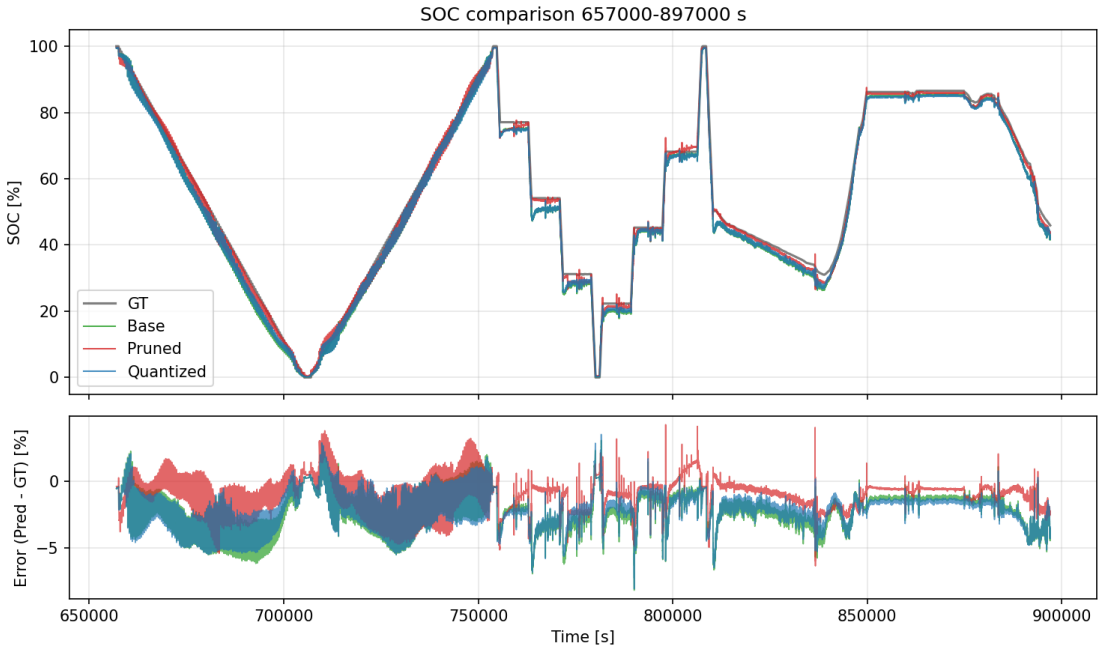


Figure 6.10: Detailed check-up-region analysis of embedded SOC streaming replay. The figure covers a representative diagnostic interval with long integration horizons and illustrates the stability of the stateful deployments during extended charge/discharge behaviour.

6.10 Results of the Limited Software Fault Tests

Holding the last valid estimate during randomly unavailable output reports has almost no effect on aggregate accuracy for report-loss rates up to 10 %. In every model, the resulting MAE change remains below 0.001 percentage points. Longer deterministic report gaps expose a clearer SOC effect. Ten gaps of 3600 samples each increase SOC MAE by approximately 0.05 percentage points, whereas the slowly changing SOH estimate remains almost unaffected. Since inference continues beneath the reporting layer, this part of the test characterises output availability and not corruption of the recurrent calculation.

The one-sample input-buffer experiment produces a distinct transient response. Across the SOC variants, median peak deviations between the faulted and clean branches lie between 3.29 and 3.82 percentage points. Their P95 values range from 16.18 to 23.81 percentage points. Every one of the 30 events per model satisfies the recovery criterion within 60 s, with median recovery times between 10.0 and 12.5 s. Quantized behaves similarly to Base, while Pruned combines the smallest median

peak with the largest upper-tail peak.

For SOH, median peak deviations range from 0.83 to 1.59 percentage points and P95 values from 3.37 to 5.85 percentage points. Within the specified 60-s horizon, 93.3 % of Base events, 90.0 % of Pruned events, and 86.7 % of Quantized events recover. The corresponding median times among recovered events are 9, 7, and 5 s. The target-related $\Delta\mathrm{MAE}_{61}$ values remain much smaller than the instantaneous peaks. For SOC, their medians range from 0.046 to 0.079 percentage points and their P95 values from 1.42 to 1.59 percentage points. For SOH, the median range is -0.0009 to 0.0012 percentage points and the P95 range is 0.053 to 0.122 percentage points. A negative event value only means that the altered prediction happened to approach an already imperfect target value. It is not an improvement in fault tolerance. Figure A.6 and Table A.6 provide the model-specific results.

The test shows that a single corrupted measurement can temporarily influence the recurrent state after the affected input has disappeared. No persistent SOC divergence or non-finite output occurs under the defined events. The analysis is deliberately bounded to input-buffer corruption and temporary report loss. It does not establish robustness against damaged weights, recurrent-state memory faults, communication failures, or physical sensor faults.

6.11 Resource Efficiency, Latency, and Trade-Off Summary

The KPI tables already show that pruning and quantization affect SOC and SOH differently, but Figures 6.11 and 6.12 condense the same trade-off visually across model size, memory, and timing. For each deployment, the model complexity can first be estimated from the parameter count under idealized storage assumptions, namely four bytes per FP32 weight and one byte per INT8 weight. The measured flash footprint is then taken from the STM32 linker output and is systematically larger because it also includes code, constant tables, and runtime support, a gap that is particularly visible for the larger SOH firmware. The RAM view in Figure 6.11 is based on the measured static data sections plus worst-case stack usage during streaming, and therefore approximates the true runtime requirement of the deployed kernels more realistically than a pure parameter-memory count.

For SOC, pruning reduces flash from 105,32 kB to 62,27 kB and kernel time from 1,40 ms to 0,80 ms. The associated E_{est} changes from 0,70 mJ to 0,40 mJ. Quantized

requires only 52,48 kB of flash, but its mixed-precision kernel increases inference time to 6,99 ms and therefore raises the proxy to 3,49 mJ. For SOH, pruning changes flash from 335,00 kB to 182,41 kB, kernel time from 22,73 ms to 12,72 ms, and E_{est} from 11,37 mJ to 6,36 mJ. Quantized reaches the smallest SOH flash footprint of 138,00 kB, while its 29,21 ms kernel time gives $E_{\text{est}} = 14{,}60$ mJ. These energy values contain no additional power information beyond the measured inference time.

The latency distribution in Figure 6.12 reveals the same hierarchy. On-device SOC inference remains in the sub-10 ms regime even for the quantized kernel. The SOH updates require 12,72 ms for Pruned, 22,73 ms for Base, and 29,21 ms for Quantized. Including UART overhead still leaves end-to-end latencies below roughly 20 ms for SOC and below roughly 50 ms for SOH, which is comfortably inside typical stationary-storage BMS timing limits.[4, 18]

The analytical operation counts clarify why storage reduction and runtime do not necessarily follow the same ratio. Base and Quantized have unchanged topologies and therefore both require 22,080 model MACs for SOC and 85,120 for SOH. In the implemented mixed-precision expressions, however, applying the FP32 row scales introduces 17,920 additional source-level multiplications for SOC and 68,608 for SOH. Counting one MAC and one extra multiplication as one operation gives the same static Quantized-to-Base ratio of approximately 1.81 for both tasks. The measured time ratios differ considerably at 4.99 for SOC and 1.29 for SOH. Figure A.10 compares these quantities directly. Integer-to-float conversion, indexing, loops, load and store activity, nonlinear functions, cache behaviour, and the absence of an optimised integer dot-product kernel are not represented by the simple count. Since no operation-level or memory-bandwidth profiling was performed, these implementation details explain plausible contributors but not a measured causal decomposition.

For a compact comparison across competing objectives, variant v is assigned the dimensionless score

$$U_v(\mathbf{w}) = w_A \frac{\text{MAE}_v}{\text{MAE}_{\text{Base}}} + w_F \frac{F_v}{F_{\text{Base}}} + w_R \frac{R_v}{R_{\text{Base}}} + w_E \frac{E_{\text{est},v}}{E_{\text{est,Base}}}, \qquad \sum_j w_j = 1, \quad (6.22)$$

where all weights are non-negative. Here, F and R denote measured flash and RAM, while E_{est} is the timing-derived proxy. The main summary uses $w_A = w_F = w_R = w_E = 0.25$. In this equal-priority case, U is the arithmetic mean of the four ratios, Base is exactly one, and a smaller value indicates a more favourable combined operating point. The value should not be read as a physical efficiency or

accuracy metric. It states how a model ranks for a chosen set of priorities.

To test whether that conclusion depends on equal weighting, the weights are varied over all 1771 non-negative combinations on a 0.05 grid whose sum is one. Pruned is the preferred SOC variant in 94.64 % of these combinations and Quantized in 5.36 %, while Base is never first. For SOH, Pruned ranks first in 53.36 %, Quantized in 17.79 %, and Base in 28.85 %. Thus, the SOC recommendation is stable across most priorities. The SOH choice depends more strongly on the application, with Base becoming attractive when accuracy dominates and Quantized when flash is assigned the largest importance. Figure A.5 shows both the priority sweeps and the complete ranking distribution.

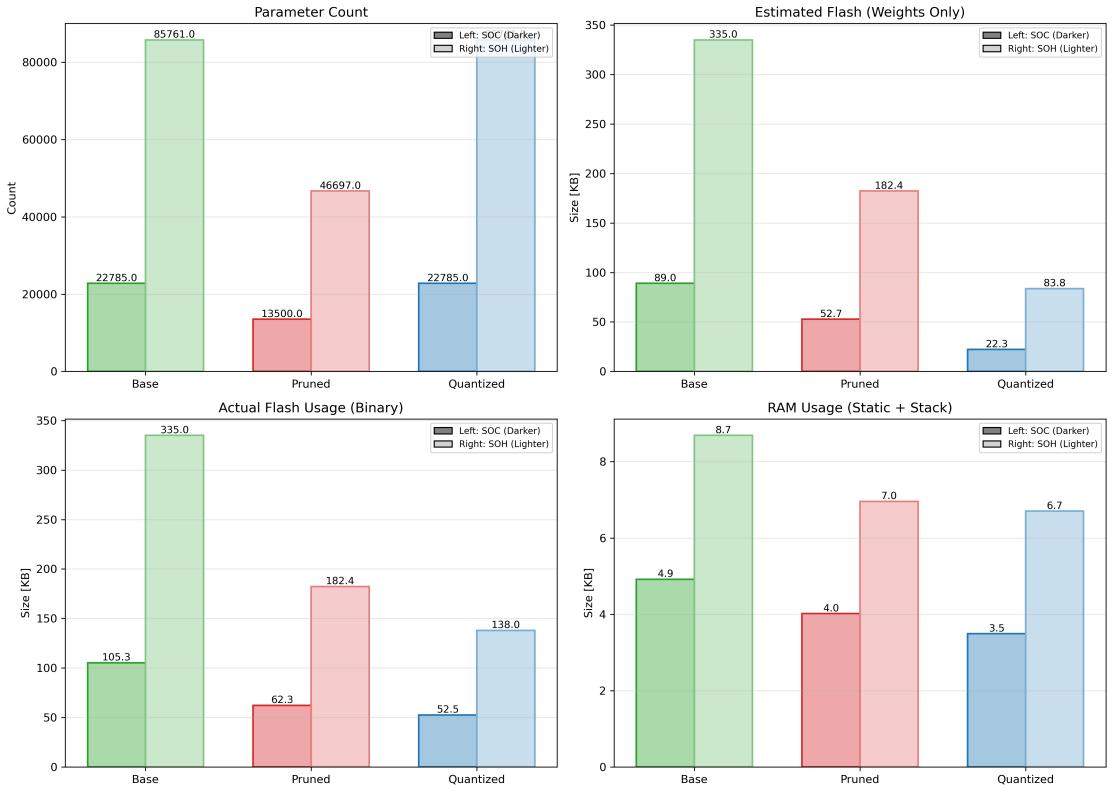


Figure 6.11: Resource landscape of the embedded SOC and SOH variants. The figure compares parameter count, idealized flash estimates from weight storage alone, measured flash on the STM32 firmware, and measured RAM usage. The separation between estimated and measured flash clarifies the additional code and runtime overhead of the embedded implementation.

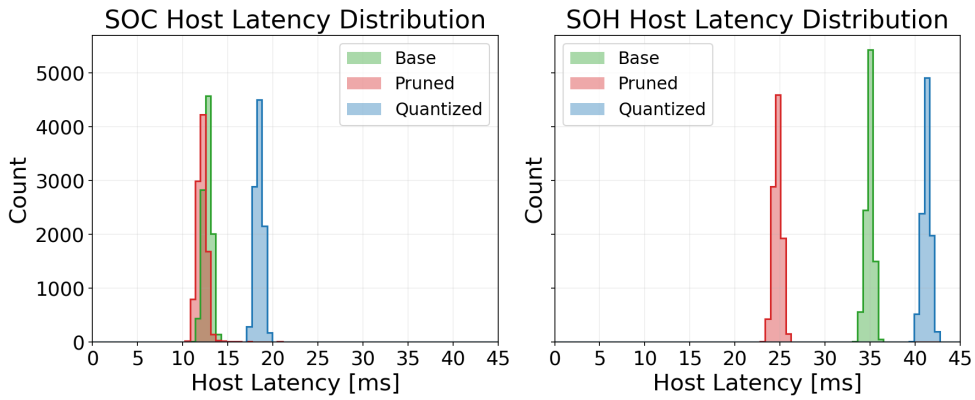


Figure 6.12: Latency distributions for SOC and SOH variants on the STM32H753ZI. The histograms summarize repeated on-device measurements and make visible the distinction between faster pruned kernels and slower quantized kernels with additional de-quantization overhead.

Table 6.3: Composite pruning and quantization summary. Values are reported as SOC/SOH. ΔMAE is given in percentage points of full scale. Flash, RAM, and E_{est} are relative changes with respect to the corresponding Base model. The utility score uses the equal weights in Equation 6.22, with lower values indicating the more favourable combined operating point.

Variant	ΔMAE [pp]	ΔFlash [%]	ΔRAM [%]	ΔE_{est} [%]	U
Pruned	−0.35 / +0.61	−41 / −46	−18 / −20	−43 / −44	0.71 / 0.91
Quantized	+0.10 / +0.56	−50 / −59	−20 / −23	+399 / +29	1.83 / 1.03

6.12 Discussion

6.12.1 Compression Behaviour of SOC and SOH Estimators

Pruning and quantization produce different outcomes for the two estimation tasks. In SOC estimation, Pruned achieves an MAE that is 0.35 percentage points below Base, while Quantized is 0.11 percentage points above it. The saliency and weight-distribution analysis confirms that the specified L2 procedure removed the intended low-ranked channels. It does not establish why the Pruned model has the lower test error. Only one trained model and one predefined split are compared for each task,

since the main objective is the embedded comparison of the compression routes. Repeated initialisations, cross-validation, and a matched fine-tuning control for the unpruned model would be required before interpreting the observed difference as a statistically stable training effect.

For SOH, both compressed variants show a bounded accuracy penalty. Interpretation requires particular care because the target changes slowly, the label is anchored by periodic capacity tests, and the reported online trajectory passes through two strong causal filter stages. The controlled comparison demonstrates that filtering can change both MAE and the order of Base, Pruned, and Quantized. Raw-network error, filtered error, and response delay therefore describe different properties and should be reported together when a post-processed SOH signal is used for BMS decisions.

6.12.2 Embedded Operating Points

The variants define distinct engineering operating points and do not follow a single accuracy order. Base offers the simplest numerical path but occupies the most flash. Structured pruning reduces the recurrent dimension and the input width of the MLP head. For SOC, this lowers flash from 105,32 kB to 62,27 kB and inference time from 1,40 ms to 0,80 ms. For SOH, the respective values change from 335,00 kB to 182,41 kB and from 22,73 ms to 12,72 ms. Within this common FP32 kernel structure, analytical and measured ratios agree closely. Pruned retains 54.9 % of the Base MACs and 57.2 % of its time for SOC, compared with 54.3 % and 56.0 % for SOH.

Quantized occupies only 52,48 kB for SOC and 138,00 kB for SOH, making it attractive when non-volatile memory is the binding constraint. Yet the retained FP32 activations, recurrent states, MLP, and accumulations prevent the model from becoming a fully integer implementation. Each recurrent INT8 value is converted and combined with its row scale inside the accumulation. The static counts alone predict the same 1.81 operation ratio for SOC and SOH, while the measured time ratios are 4.99 and 1.29. The difference cannot be attributed to one cause without cycle-level and memory-bandwidth profiling. It can include conversions, indexing, memory behaviour, nonlinear functions, and compiler-generated loop code. Scale hoisting, fused scale and bias handling, DSP-oriented dot products, or activation and state quantization are possible routes toward a more efficient kernel.

6.12.3 Latency, Utility Score, and BMS Deployment

The distinction between communication-dominated system latency and pure on-device inference time is important for interpreting the results. The UART-based measurements report end-to-end latencies of 12,20 ms, 12,70 ms, and 18,48 ms for the pruned, dense, and quantized SOC variants. The corresponding SOH latencies are 24,69 ms, 34,88 ms, and 41,23 ms. The actual neural computation is substantially shorter. Tables 6.1 and 6.2 show that dense and pruned SOC inference remains below 1,5 ms, and even the quantized SOC kernel stays below 10 ms. The SOH updates require more time because of the larger hidden state and post-processing, with 12,72 ms for pruning, 22,73 ms for the dense model, and 29,21 ms for quantization. These margins are compatible with the low-rate sampling and diagnostic time scales typical of stationary-storage BMS operation.[4, 18]

The utility score in Table 6.3 combines normalized accuracy, flash, RAM, and timing-proxy terms. It does not replace hard requirements, but it exposes whether the benefits of a compression variant outweigh its penalties under explicit priorities. With equal weights, SOC Pruned reaches $U = 0.71$, whereas SOC Quantized reaches $U = 1.83$ because its timing overhead outweighs its flash reduction. The SOH values of 0.91 for Pruned and 1.03 for Quantized describe a closer trade-off. The weight sensitivity adds the information that the SOC recommendation remains stable over most priority combinations, while the preferred SOH variant changes when accuracy or flash is emphasised. This distinction makes the score useful as a transparent decision aid rather than as a universal model ranking.

6.12.4 Limitations and Future Extensions

The quantitative evidence belongs to one LFP ageing campaign at 25 °C, the selected SOC and SOH architectures, and one STM32H753ZI implementation. Removing the same hidden channels or storing the same recurrent matrices as INT8 gives predictable changes in parameter storage for a fixed topology. Retaining estimation accuracy is more dependent on the cell chemistry, operating range, label construction, and training coverage. Results from other battery systems therefore support the general feasibility of compact estimators, but not the direct transfer of the MAE values or the preferred compression level.[72, 94, 1]

The MAC relation is independent of the microcontroller, whereas measured flash and timing depend on the clock, compiler settings, floating-point and signal-processing support, memory organisation, and cache. The energy numbers in this

chapter are obtained from the constant 0,5 W assumption in Equation 6.21. They are not component-level power measurements and cannot reveal the separate contributions of computation, flash, SRAM, UART, or regulators.

The two compression routes are post-training methods. Structured pruning is performed once and followed by short fine-tuning, while quantization starts from the original FP32 model. Quantization-aware training and iterative or pruning-aware training could adapt the parameters to their constrained representation.[66, 117] They require a separate training comparison and are outside the present benchmark.

Host-side derivative and cumulative-feature construction deliberately isolates neural inference in the timing comparison. The transient input-buffer analysis adds a bounded test of fault propagation through the recurrent state, but it is not a complete safety assessment. Further work should include variable sampling, causal derivative filtering, corrupted recurrent states and parameters, communication errors, and physical sensor or MCU faults. Moving the complete pipeline to the custom BMS platform would also add scheduler interaction, sensing-chain behaviour, and pack-level communication. On the model side, relevant extensions include additional battery systems and operating ranges, repeated training runs, combined pruning and quantization, and comparisons with compact GRUs, transformer-style models, and hybrid observers.[32, 118, 25, 40]

6.13 Conclusion and Outlook

This chapter evaluates two transparent compression paths for stateful LSTM + MLP SOC and SOH estimators under a common STM32 benchmark. Base, Pruned, and Quantized are compared using the same continuous feature stream, C implementation boundary, error definitions, linked-firmware memory, and cycle-counter timing. Energy per inference is represented only by the constant-power timing proxy E_{est}.

For SOC, Pruned reduces flash by 40.9 %, RAM by approximately 18 %, and inference time by 42.9 %, while its MAE is 0.35 percentage points below Base for the evaluated model and split. Quantized reduces flash by 50.2 % and RAM by approximately 20 %, but raises inference time by 399 % and MAE by 0.11 percentage points. For SOH, Pruned saves 45.5 % flash, approximately 20 % RAM, and 44.0 % inference time, with an MAE increase of 0.61 percentage points. Quantized saves 58.8 % flash and approximately 23 % RAM, while inference time rises by 28.5 % and MAE by 0.56 percentage points. Under the fixed-power assumption,

the percentages for E_{est} are identical to those for inference time.

The continuous ten-part analysis reveals no quantization-specific monotonic divergence from Base. Following one-sample input-buffer faults, all SOC events recover under the defined criterion within 60 s. For SOH, 6.7–13.3 % of events, depending on the model, do not meet that criterion inside the same horizon. The SOH comparison further demonstrates that compression and post-processing delay cannot be assessed independently because the two-stage filter changes both error magnitude and model ordering.

For the present firmware, structured pruning is the more balanced operating point, whereas recurrent-weight quantization is most attractive when flash dominates and its mixed-precision timing overhead is acceptable. This conclusion is bounded to the evaluated data, models, implementation, and build configuration. Further work should add externally measured board power, kernel profiling, wider battery and operating coverage, repeated training runs, direct comparisons with compression-aware training, and internal as well as hardware-facing fault classes. Joint pruning and quantization remains a promising route for combining a smaller recurrent state with compact weight storage.

Chapter 7

General Discussion, Conclusion, and Outlook

7.1 Integrated Synthesis of the Research Questions

The global result of this work can be read along the three research questions introduced in Chapter 1. The first question asks how comparatively simple neural models can be made time- and ageing-aware without immediately moving to large recurrent architectures. Chapter 4 answers this through representation rather than model complexity: cumulative-current information and lag-sequence history inject degradation-relevant temporal context into an otherwise memoryless MLP. The resulting average SOH error of about 1,10 % shows that the structure of the input can be as important as the structure of the network, especially for slowly evolving ageing targets.

The second question asks how estimator quality changes under realistic measurement and operating conditions. Chapter 5 shows that the answer cannot be obtained from a clean-condition MAE alone. Under nominal signals, the SOH-supported direct-measurement estimator reaches the lowest SOC MAE of about 0,24 %. Under a strong persistent current bias, however, it drifts to roughly 30 % error, while the voltage-corrected equivalent-circuit observer remains closer to 9 %. The data-driven recurrent estimator is not the universal winner either, but it is strongest under missing-sample stress and fastest in recovery after initialization mismatch. The main result is therefore not a fixed ranking of models, but the need to evaluate accuracy, disturbance sensitivity, and convergence behaviour as separate properties.

The third question asks what remains of neural estimation once embedded hardware is treated as part of the problem. Chapter 6 shows that stateful recurrent SOC and SOH estimators can be executed on the STM32 target with comfortable timing

margins, but that compression changes the evaluation. Structured pruning is the most balanced lever for the SOC estimator because it reduces flash, RAM, inference time, and the corresponding timing-derived energy indicator, with an equal-weight utility score of $U = 0.71$. INT8 weight quantization reduces non-volatile memory more strongly, but the current mixed-precision path reconstructs effective floating-point weights during inference and raises SOC kernel time, which contributes to $U = 1.83$. The embedded result is thus not merely that a neural estimator can be deployed, but that model architecture, compression method, firmware implementation, and measured hardware behaviour must be selected together.

Across these questions, the research moves from representation design to disturbed-operation validation and finally to measured embedded execution. The custom BMS platform and the shared measurement chain provide the system context that makes this path practically meaningful. Without this layer, the work would remain an offline comparison of algorithms. With it, estimator design can be connected to sensing, firmware, memory, timing, and energy constraints.

7.2 Findings Across BMS Requirements

Table 7.1: Synthesis of the contribution chapters with respect to the three requirement dimensions used throughout this work.

Research component	Performance	Hardware / embedded	Deployment logic
Neural ageing estimation	SOH accuracy through representation quality	indirect relevance	lag-sequence and feature design for compact models
Robustness benchmark	nominal accuracy, disturbance sensitivity, and recovery	embedded feasibility as boundary condition	matched multi-scenario evaluation across estimator classes
Embedded AI implementation	retained accuracy after compression	flash, RAM, and latency measured on MCU, with a timing-based energy proxy	pruning, quantization, export, and firmware validation
BMS platform and datasets	measurement basis for evaluation	sensing chain and controller architecture	reproducible integration and estimator exchange

Table 7.1 shows why the three requirement dimensions introduced in Figure 1.1 are not interchangeable. The performance dimension is not limited to mean error: it also includes robustness against disturbed measurements, recovery after state inconsistency, and the ability to retain useful behaviour when the input stream is incomplete or corrupted. This is why the robustness chapter is not just another accuracy experiment, but the point where nominal model ranking is deliberately challenged.

The hardware dimension changes the same comparison again. A model that is attractive in software must still fit the flash and RAM budget, meet the update-rate requirement, and avoid excessive energy demand. The embedded chapter therefore

evaluates retained accuracy after pruning or quantization rather than unconstrained floating-point accuracy alone. This is a different criterion from the one used during model training.

The deployment dimension links these two views. It includes the feature pipeline, recurrent-state handling, model export, firmware implementation, and the ability to exchange estimators on a reproducible measurement chain. This work therefore treats deployment as a design requirement in its own right, not as a final formatting step after a model has already been selected.

7.3 Transferable Design Lessons

Four design lessons can be transferred from the individual chapters to BMS estimator design more generally.

The first lesson is *feature representation before model complexity*. Chapter 4 shows that neural networks do not have to become large merely because the battery state is history-dependent. With cumulative-current features and lag-sequence inputs, a compact MLP can exploit temporal and throughput-related information even though the estimator itself has no recurrent state. This principle also applies to recurrent models: an LSTM or GRU benefits from meaningful causal input channels just as much as a feedforward model does. Chapter 6 adds the complementary result that more expressive time-dependent models can still be made embedded-feasible when their architecture is structured well enough for pruning and quantization. The design message is therefore not MLP versus LSTM. It is to design the representation first, choose only as much model complexity as the task requires, and then optimize the selected architecture for deployment.

The second lesson is that *single-metric accuracy is not enough for deployment decisions*. A clean-condition MAE is a necessary reference, but it does not reveal why an estimator fails. Chapter 5 shows that there is no universally best estimator class under all tested conditions. Current bias, additive noise, missing samples, burst dropout, irregular sampling, voltage spikes, ADC quantization, and initialization mismatch each probe a different weakness of the estimation chain. The practical value of the robustness benchmark is therefore not a fixed leaderboard. It is a way to identify the weak points of a chosen estimator so that they can be addressed through feature design, retraining, observer tuning, filtering, reset logic, or fault handling before deployment.

The third lesson is that *compression must be matched to the active embedded constraint*. Pruning and quantization are useful for different reasons. Structured pruning removes recurrent channels and downstream matrix rows, so it can reduce flash, RAM, inference time, and energy at the same time. Quantization reduces the weight storage more strongly, but the present STM32 implementation shows that it can increase inference time when INT8 weights have to be reconstructed through software-side scaling during every update. If flash memory is the limiting resource, quantization can be the decisive step. If latency or energy is limiting, pruning can be the more balanced option. The most promising direction is therefore not to treat pruning and quantization as alternatives, but to combine them: first reduce the state size and operation count, then quantize the remaining weights to reduce the persistent footprint.

The fourth lesson is that *deployment constraints have to shape model design from the beginning*. The estimator architecture, feature pipeline, firmware implementation, and target hardware are coupled. A very complex model may look attractive in an offline benchmark, but it is harder to export, optimize, validate, and maintain on a microcontroller than a compact or structurally regular model. Model design should therefore include the expected BMS target from the start: available sensor signals, sampling rate, memory budget, arithmetic support, update frequency, and firmware integration path. Validation should also not end with an offline test. The estimator should be replayed on the target controller or in a hardware-facing test stand where timing, memory, communication, and numerical behaviour can be measured together. The Custom BMS Platform is important for this reason: it provides the environment in which state-estimation software can be tested as deployable BMS functionality rather than only as a model file.

7.4 Position Within the State of Research and Practical Implications

Compared with much of the battery-estimation literature, the contribution of this research is not only a new model or a single accuracy result. Many existing studies focus on one dimension at a time: a neural architecture under curated signals, an observer under a specific fault assumption, or an embedded implementation after the model has already been chosen. The present work connects these layers. It links representation design, disturbance-based benchmarking, compression, MCU mea-

surement, and BMS hardware integration into one deployment-oriented evaluation chain.

This positioning matters because BMS hardware is becoming more capable, with stronger microcontrollers, embedded-AI support, and tighter interaction between sensing ICs and application processors. These developments make neural and hybrid estimators more realistic for BMS deployment, but they do not remove the need for rigorous validation. More capable hardware expands the design space, but it does not make a nominal MAE sufficient.

The practical implication is a clear design order. Estimator selection should begin with the target use case and the expected sensing conditions, not with an architecture preference. The feature representation should then be designed for the target state, the estimator should be tested under clean and disturbed scenarios, the compression strategy should be matched to the resource bottleneck, and the final model should be verified on the intended controller or an equivalent embedded path. The present research therefore contributes less a single winning estimator than a structured way to judge estimator classes under BMS-relevant conditions.

7.5 Limitations and Transferability

The results must be interpreted within the boundary conditions of the studies. The ageing-estimation chapter uses an NMC cyclic-ageing dataset, whereas the robustness and embedded chapters use an LFP design-of-experiments campaign. Their numerical results are therefore complementary rather than directly interchangeable. The robustness benchmark focuses on one representative ageing trajectory to isolate estimator-side behaviour, which improves interpretability but limits cross-cell statistics.

The embedded results are also platform-specific. They refer to STM32H7-class hardware, hand-written recurrent kernels, and the selected measurement setup for latency and energy. Other controllers, vendor toolchains, memory hierarchies, or fixed-point kernels could change the quantitative balance between pruning and quantization. Similarly, the custom BMS platform is a laboratory system rather than a field-deployed product, so its role is to demonstrate a reproducible validation chain rather than final operational qualification.

The transferable result is therefore the evaluation logic, not every numerical value. Estimator classes should be compared under matched nominal, disturbed,

recovery, and embedded conditions, and the final judgement should be based on how the method behaves across this complete chain.

7.6 Overall Conclusion

This research examined AI-based battery-state estimation from the perspective of a practical BMS rather than from the perspective of an isolated offline model. The work began with the observation that SOC and SOH estimators are only useful in operation if they satisfy more than one requirement at the same time. They must represent the relevant battery history, remain reliable under imperfect measurements, fit within embedded hardware limits, and be transferable into a reproducible firmware and validation chain. The central conclusion is therefore that deployment-oriented battery intelligence is not defined by the lowest clean-data error, but by the behaviour of the full estimator path from data representation to hardware execution.

The contribution chapters address this path step by step. The ageing-estimation study shows that simple models can become time-aware when the input representation contains physically meaningful history, as demonstrated by the MLP-based SOH estimator with cumulative-current and lag-sequence features. The robustness benchmark then shows why such accuracy claims have to be stress-tested under realistic measurement disturbances: bias, noise, missing samples, timing irregularities, voltage spikes, ADC quantization, and initialization mismatch can change the ranking of estimator classes and reveal weaknesses that clean-condition metrics hide. The embedded-AI study extends the argument to microcontroller execution and demonstrates that model quality after pruning, quantization, export, and on-device measurement is the relevant quantity for BMS deployment. The hardware chapter provides the system basis that makes this chain concrete by offering an adaptable Custom BMS Platform with accessible sensing, firmware partitioning, and estimator integration.

Taken together, the work contributes a requirement-based way of evaluating battery-state estimators. The purpose is not to declare one estimator family universally superior. Direct-measurement, model-based, hybrid, and data-driven approaches each have useful operating regions and failure modes. The contribution is instead to show how these methods can be compared in a BMS-relevant way: first by choosing a suitable representation, then by extending the evaluation beyond

clean-condition accuracy to disturbance and recovery behaviour, then by selecting pruning, quantization, or their combination according to the actual hardware bottleneck, and finally by validating the result on the controller. This evaluation chain is the main outcome of the research.

7.7 Remaining Gaps

The conclusions must be interpreted within the experimental scope of the work. The studies use two cell chemistries and two ageing datasets, namely an NMC cyclic-ageing campaign for the MLP-based SOH study and an LFP design-of-experiments campaign for the robustness and embedded-deployment studies. These datasets provide complementary evidence, but they do not form a single unified cross-chemistry benchmark. The robustness benchmark intentionally focuses on one representative ageing trajectory to isolate estimator-side behaviour, which limits the statistical breadth across cells. The embedded results are tied to STM32H7-class hardware, hand-written C kernels, and the specific measurement and replay workflow used in this work.

Several open points therefore remain. The robustness scenarios have so far been evaluated mainly as controlled replay experiments rather than as live disturbance-injection tests on the complete Custom BMS Platform. The embedded deployment results confirm memory, timing, energy, and numerical behaviour for the investigated models, but they do not yet cover a broader family of recurrent, feedforward, transformer-style, and hybrid neural-equivalent-circuit estimators. The hardware platform itself is a compact laboratory system and not yet a field-scale BMS for larger packs. These limitations do not weaken the methodological conclusion, but they define where further validation is required before the quantitative results can be generalized.

7.8 Outlook

Future work should first broaden the empirical basis. The same evaluation chain should be applied to additional cells, chemistries, ageing states, temperature ranges, and operating profiles so that the observed relationships between accuracy, robustness, recovery, and compressibility can be tested beyond the present trajectories. This is especially important for separating general estimator behaviour from

dataset-specific effects. A broader benchmark would also make it possible to quantify how much training coverage is required before data-driven estimators can be trusted outside the ageing and load conditions represented in the original data.

A second direction is to move the disturbance protocols closer to the embedded platform. Bias, noise, missing samples, timing irregularities, and spikes should be injected into the live firmware and measurement chain of the Custom BMS Platform rather than only into host-side replay data. This would close the loop between the robustness and embedded-deployment parts of the work and would reveal interactions that may only appear when disturbed signals, compressed models, firmware scheduling, communication overhead, and controller timing occur together.

A third direction is architecture and compression co-design. The present work shows that pruning and quantization are not neutral post-processing steps and that their benefit depends on the architecture and on the firmware kernel. Future work should therefore compare MLPs, GRUs, LSTMs, compact transformer-style models, and neural-augmented equivalent-circuit models under the same accuracy, robustness, memory, latency, and energy criteria. A particularly relevant next step is the combined use of pruning and quantization, because pruning can reduce state size and operation count while quantization can reduce the remaining persistent weight memory. The goal would be to turn the qualitative design lesson developed here into quantitative selection rules for embedded BMS estimator architectures.

Finally, the Custom BMS Platform should be extended from a compact laboratory system toward larger pack configurations and longer autonomous operation. This includes transfer to higher cell counts, pack-level thermal and electrical effects, longer cycling experiments, and eventually online or continual adaptation on the controller. Such adaptation would be the logical next step for long-lived decentralized and mobile systems: the estimator would not only be deployed once, but would remain maintainable as the battery ages and as the operating environment changes. In this sense, the long-term objective remains a BMS in which AI-based state estimation is not an offline add-on, but an integrated, testable, and resource-aware part of the battery system itself.

Bibliography

[1] Florian Rzepka, Philipp Hematty, Mano Schmitz, and Julia Kowal. "Neural Network Architecture for Determining the Aging of Stationary Storage Systems in Smart Grids". In: *Energies* 16.17 (Aug. 22, 2023), p. 6103. DOI: 10.3390/en16176103. URL: https://www.mdpi.com/1996-1073/16/17/6103 (visited on 12/04/2025) (cited on pp. 2, 5, 6, 43, 57, 76, 129).

[2] Florian Rzepka, Qinnan Chen, and Julia Kowal. "Performance Benchmarking of Embedded Battery State Estimation: A Measurement-Only Comparison of Direct-Measurement, Hybrid, and Data-Driven Estimators". Manuscript submitted to the Journal of Energy Storage. 2026 (cited on pp. 2, 5, 6, 71).

[3] Florian Rzepka and Julia Kowal. *Embedded Artificial Intelligence in Battery Management Systems: Pruning and Quantization for Efficient State-of-Charge and State-of-Health Estimation*. SSRN. Feb. 24, 2026. DOI: 10.2139/ssrn.6298526. URL: https://ssrn.com/abstract=6298526 (visited on 07/23/2026) (cited on pp. 2, 5, 6, 105).

[4] Gregory L. Plett. *Battery Management Systems: Volume 1: Battery Modeling*. Artech House Power Engineering and Power Electronics. Boston London: Artech House, 2015. 1 p. (cited on pp. 3, 7–10, 50, 125, 129).

[5] Long Zhou, Xin Lai, Bin Li, Yi Yao, Ming Yuan, Jiahui Weng, and Yuejiu Zheng. "State Estimation Models of Lithium-Ion Batteries for Battery Management System: Status, Challenges, and Future Trends". In: *Batteries* 9.2 (Feb. 13, 2023), p. 131. DOI: 10.3390/batteries9020131. URL: https://www.mdpi.com/2313-0105/9/2/131 (visited on 06/24/2026) (cited on pp. 3, 9).

[6] Ming Jiang, Dongjiang Li, Zonghua Li, Zhuo Chen, Qinshan Yan, Fu Lin, Cheng Yu, Bo Jiang, Xuezhe Wei, Wensheng Yan, and Yong Yang. "Advances in Battery State Estimation of Battery Management System in Elec-

tric Vehicles". In: *Journal of Power Sources* 612 (Aug. 2024), p. 234781. DOI: 10.1016/j.jpowsour.2024.234781. URL: https://linkinghub.elsevier.com/retrieve/pii/S037877532400733X (visited on 06/24/2026) (cited on pp. 3, 9).

[7] Xue Li, Jiuchun Jiang, Caiping Zhang, Le Yi Wang, and Linfeng Zheng. "Robustness of SOC Estimation Algorithms for EV Lithium-Ion Batteries against Modeling Errors and Measurement Noise". In: *Mathematical Problems in Engineering* 2015 (2015), pp. 1–14. DOI: 10.1155/2015/719490. URL: http://www.hindawi.com/journals/mpe/2015/719490/ (visited on 03/23/2026) (cited on pp. 4, 71, 81).

[8] F. Naseri, C. Barbu, A.C.S. Jensen, P. Setoodeh, J.A. Romero Baena, N. Hevele, and E. Schaltz. "Toward Reliable Wireless BMS: Robust Battery State Estimation Under Noise and Uncertainty". In: *IEEE Transactions on Vehicular Technology* (2025), pp. 1–12. DOI: 10.1109/TVT.2025.3638613. URL: https://ieeexplore.ieee.org/document/11271354/ (visited on 03/23/2026) (cited on pp. 4, 71, 81).

[9] Franziska Berger, Dominik Joest, Elias Barbers, Katharina Quade, Ziheng Wu, Dirk Uwe Sauer, and Philipp Dechent. "Benchmarking Battery Management System Algorithms - Requirements, Scenarios and Validation for Automotive Applications". In: *eTransportation* 22 (Dec. 2024), p. 100355. DOI: 10.1016/j.etran.2024.100355. URL: https://linkinghub.elsevier.com/retrieve/pii/S2590116824000456 (visited on 03/23/2026) (cited on pp. 4, 71, 79, 81).

[10] Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, and Song Han. "Tiny Machine Learning: Progress and Futures". In: *IEEE Circuits and Systems Magazine* 23.3 (Aut. 2023), pp. 8–34. DOI: 10.1109/MCAS.2023.3302182. arXiv: 2403.19076 [cs]. URL: http://arxiv.org/abs/2403.19076 (visited on 12/02/2025) (cited on pp. 4, 21).

[11] Mina Naguib, Phillip Kollmeyer, Carlos Vidal, Josimar Duque, Oliver Gross, and Ali Emadi. "Microprocessor Execution Time and Memory Use for Battery State of Charge Estimation Algorithms". In: WCX SAE World Congress Experience. Detroit & Online, Michigan, United States, Mar. 29, 2022, pp. 2022-01–0697. DOI: 10.4271/2022-01-0697. URL: https://saemobilus.sae.org/papers/microprocessor-execution-time-memory-use-battery-state-charge-estim

ation-algorithms-2022-01-0697 (visited on 12/03/2025) (cited on pp. 4, 21, 105, 115).

[12] Song Han, Jeff Pool, John Tran, and William J. Dally. *Learning Both Weights and Connections for Efficient Neural Networks*. Version 3. 2015. DOI: 10.48550/ARXIV.1506.026 URL: https://arxiv.org/abs/1506.02626 (visited on 12/02/2025). Pre-published (cited on pp. 4, 22, 26, 28).

[13] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference*. Version 1. 2017. DOI: 10.48550/ARXIV.1712.05877. URL: https://arxiv.org/abs/1712.05877 (visited on 12/02/2025). Pre-published (cited on pp. 4, 34, 37, 39, 113).

[14] Peter Kurzweil and Otto K. Dietlmeier. *Elektrochemische Speicher: Superkondensatoren, Batterien, Elektrolyse-Wasserstoff, Rechtliche Rahmenbedingungen*. Wiesbaden: Springer Fachmedien Wiesbaden, 2018. DOI: 10.1007/978-3-658-21829-4. URL: http://link.springer.com/10.1007/978-3-658-21829-4 (visited on 04/14/2026) (cited on pp. 7, 8, 45).

[15] Prashant Shrivastava, P. Amritansh Naidu, Sakshi Sharma, Bijaya Ketan Panigrahi, and Akhil Garg. "Review on Technological Advancement of Lithium-Ion Battery States Estimation Methods for Electric Vehicle Applications". In: *Journal of Energy Storage* 64 (Aug. 2023), p. 107159. DOI: 10.1016/j.est.2023.107159. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X2300556X (visited on 01/14/2025) (cited on pp. 8, 71, 105).

[16] M. Kassem, J. Bernard, R. Revel, S. Pélissier, F. Duclaud, and C. Delacourt. "Calendar Aging of a Graphite/LiFePO4 Cell". In: *Journal of Power Sources* 208 (June 2012), pp. 296–305. DOI: 10.1016/j.jpowsour.2012.02.068. URL: https://linkinghub.elsevier.com/retrieve/pii/S0378775312004284 (visited on 04/14/2026) (cited on p. 9).

[17] Anup Barai, Kotub Uddin, Matthieu Dubarry, Limhi Somerville, Andrew McGordon, Paul Jennings, and Ira Bloom. "A Comparison of Methodologies for the Non-Invasive Characterisation of Commercial Li-ion Cells". In: *Progress in Energy and Combustion Science* 72 (May 2019), pp. 1–31. DOI: 10.1016/j.pecs.2019.01.001. URL: https://linkinghub.elsevier.com/retrieve/pii/S0360128518300996 (visited on 04/14/2026) (cited on pp. 9, 45, 49).

[18] M. Berecibar, I. Gandiaga, I. Villarreal, N. Omar, J. Van Mierlo, and P. Van Den Bossche. "Critical Review of State of Health Estimation Methods of Li-ion Batteries for Real Applications". In: *Renewable and Sustainable Energy Reviews* 56 (Apr. 2016), pp. 572–587. DOI: 10.1016/j.rser.2015.11.042. URL: https://linkinghub.elsevier.com/retrieve/pii/S1364032115013076 (visited on 12/02/2025) (cited on pp. 9, 50, 105, 125, 129).

[19] *BQ79616 16-Series Battery Monitor, Balancer, and Integrated Hardware Protector*. 2023. URL: https://www.ti.com/lit/ds/symlink/bq79616.pdf?ts=1775312350051&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252FBQ79616 (cited on pp. 9, 79–81).

[20] *10s Battery Pack Monitoring, Balancing, and Comprehensive Protection, 50-A Discharge Reference Design*. URL: https://www.ti.com/lit/ug/tiduar8c/tiduar8c.pdf (cited on pp. 9, 80, 81).

[21] *Design Considerations of Current Sensing With BQ769x2 Family for Battery Management System*. URL: https://www.ti.com/lit/an/sluab10a/sluab10a.pdf?ts=1775276566199 (cited on pp. 9, 79–81).

[22] *TLE4972 High Precision Coreless Current Sensor for Automotive Applications*. URL: https://www.infineon.com/assets/row/public/documents/24/49/infineon-tle4972-ae35d5-datasheet-en.pdf?fileId=8ac78c8c7c72fb9a017c7e1109e20a17 (cited on pp. 9, 79, 81).

[23] *TMCS1108 3% Functional Isolation Hall-Effect Current Sensor With ±100-V Working Voltage*. URL: https://www.ti.com/lit/ds/symlink/tmcs1108.pdf (cited on pp. 9, 79, 81).

[24] Zeeshan Ahmad Khan, Prashant Shrivastava, Syed Muhammad Amrr, Saad Mekhilef, Abdullah A. Algethami, Mehdi Seyedmahmoudian, and Alex Stojcevski. "A Comparative Study on Different Online State of Charge Estimation Algorithms for Lithium-Ion Batteries". In: *Sustainability* 14.12 (June 17, 2022), p. 7412. DOI: 10.3390/su14127412. URL: https://www.mdpi.com/2071-1050/14/12/7412 (visited on 01/14/2025) (cited on p. 10).

[25] Yizhao Gao, Gregory L. Plett, Guodong Fan, and Xi Zhang. "Enhanced State-of-Charge Estimation of LiFePO4 Batteries Using an Augmented Physics-Based Model". In: *Journal of Power Sources* 544 (Oct. 2022), p. 231889. DOI: 10.1016/j.jpowsour.2022.231889. URL: https://linkinghub.elsevier.com/ret

rieve/pii/S0378775322008758 (visited on 01/14/2025) (cited on pp. 10, 11, 72, 81, 105, 130).

[26] Feng Guo, Luis D. Couto, Grietus Mulder, Khiem Trad, Guangdi Hu, Odile Capron, and Keivan Haghverdi. "A Systematic Review of Electrochemical Model-Based Lithium-Ion Battery State Estimation in Battery Management Systems". In: *Journal of Energy Storage* 101 (Nov. 2024), p. 113850. DOI: 10.1016/j.est.2024.113850. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X24034364 (visited on 01/14/2025) (cited on pp. 10, 11, 71, 72, 105).

[27] M.S. Hossain Lipu, M.A. Hannan, Aini Hussain, Afida Ayob, Mohamad H.M. Saad, Tahia F. Karim, and Dickson N.T. How. "Data-Driven State of Charge Estimation of Lithium-Ion Batteries: Algorithms, Implementation Factors, Limitations and Future Trends". In: *Journal of Cleaner Production* 277 (Dec. 2020), p. 124110. DOI: 10.1016/j.jclepro.2020.124110. URL: https://linkinghub.elsevier.com/retrieve/pii/S095965262034155X (visited on 04/14/2026) (cited on pp. 10, 12).

[28] Xiao Sun, Long Zuo, Mingkang Zhang, Yanzhi Su, Qiang Fu, and Jiahui Jiang. "Equivalent Circuit Models for Lithium-Ion Batteries: A Comprehensive Review". In: *Electronics* 15.9 (May 6, 2026), p. 1968. DOI: 10.3390/electronics15091968. URL: https://www.mdpi.com/2079-9292/15/9/1968 (visited on 06/24/2026) (cited on p. 10).

[29] Davide Pio Laudani, Davide Milillo, Michele Quercio, Francesco Riganti Fulginei, and Lorenzo Sabino. "A Comprehensive Review of Equivalent Circuit Models and Neural Network Models for Battery Management Systems". In: *Batteries* 12.1 (Jan. 22, 2026), p. 37. DOI: 10.3390/batteries12010037. URL: https://www.mdpi.com/2313-0105/12/1/37 (visited on 06/24/2026) (cited on p. 10).

[30] Jiabo Li, Min Ye, Kangping Gao, Xinxin Xu, Meng Wei, and Shengjie Jiao. "Joint Estimation of State of Charge and State of Health for Lithium-ion Battery Based on Dual Adaptive Extended Kalman Filter". In: *International Journal of Energy Research* 45.9 (July 2021), pp. 13307–13322. DOI: 10.1002/er.6658. URL: https://onlinelibrary.wiley.com/doi/10.1002/er.6658 (visited on 04/14/2026) (cited on pp. 11, 19).

[31] Feng Zhao, Yun Guo, and Baoming Chen. "A Review of Lithium-Ion Battery State of Charge Estimation Methods Based on Machine Learning". In: *World Electric Vehicle Journal* 15.4 (Mar. 25, 2024), p. 131. DOI: 10.3390/wevj15040131. URL: https://www.mdpi.com/2032-6653/15/4/131 (visited on 06/24/2026) (cited on p. 12).

[32] Zachary C. Lipton, John Berkowitz, and Charles Elkan. *A Critical Review of Recurrent Neural Networks for Sequence Learning*. Version 4. 2015. DOI: 10.48550/ARXIV.1506.00019. URL: https://arxiv.org/abs/1506.00019 (visited on 12/02/2025). Pre-published (cited on pp. 15, 109, 130).

[33] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. *On the Difficulty of Training Recurrent Neural Networks*. Version 2. 2012. DOI: 10.48550/ARXIV.1211.5063. URL: https://arxiv.org/abs/1211.5063 (visited on 12/02/2025). Pre-published (cited on pp. 15, 110).

[34] Yalan Bi, Yilin Yin, and Song-Yul Choe. "Online State of Health and Aging Parameter Estimation Using a Physics-Based Life Model with a Particle Filter". In: *Journal of Power Sources* 476 (Nov. 2020), p. 228655. DOI: 10.1016/j.jpowsour.2020.228655. URL: https://linkinghub.elsevier.com/retrieve/pii/S0378775320309599 (visited on 04/14/2026) (cited on p. 19).

[35] Jinpeng Tian, Rui Xiong, and Quanqing Yu. "Fractional-Order Model-Based Incremental Capacity Analysis for Degradation State Recognition of Lithium-Ion Batteries". In: *IEEE Transactions on Industrial Electronics* 66.2 (Feb. 2019), pp. 1576–1584. DOI: 10.1109/TIE.2018.2798606. URL: https://ieeexplore.ieee.org/document/8270363/ (visited on 04/14/2026) (cited on p. 19).

[36] Bin Gou, Yan Xu, and Xue Feng. "State-of-Health Estimation and Remaining-Useful-Life Prediction for Lithium-Ion Battery Using a Hybrid Data-Driven Method". In: *IEEE Transactions on Vehicular Technology* 69.10 (Oct. 2020), pp. 10854–10867. DOI: 10.1109/TVT.2020.3014932. URL: https://ieeexplore.ieee.org/document/9162518/ (visited on 04/14/2026) (cited on p. 19).

[37] Tsuyoshi Oji, Yanglin Zhou, Song Ci, Feiyu Kang, Xi Chen, and Xiulan Liu. "Data-Driven Methods for Battery SOH Estimation: Survey and a Critical Analysis". In: *IEEE Access* 9 (2021), pp. 126903–126916. DOI: 10.1109/ACCESS.2021.3111927. URL: https://ieeexplore.ieee.org/document/9535528/ (visited on 04/14/2026) (cited on p. 19).

[38] Yizhao Gao, Kailong Liu, Chong Zhu, Xi Zhang, and Dong Zhang. "Co-Estimation of State-of-Charge and State-of- Health for Lithium-Ion Batteries Using an Enhanced Electrochemical Model". In: *IEEE Transactions on Industrial Electronics* 69.3 (Mar. 2022), pp. 2684–2696. DOI: 10.1109/TIE.2021.3066946. URL: https://ieeexplore.ieee.org/document/9384220/ (visited on 01/14/2025) (cited on pp. 19, 105).

[39] Jiangwei Shen, Wensai Ma, Jian Xiong, Xing Shu, Yuanjian Zhang, Zheng Chen, and Yonggang Liu. "Alternative Combined Co-Estimation of State of Charge and Capacity for Lithium-Ion Batteries in Wide Temperature Scope". In: *Energy* 244 (Apr. 2022), p. 123236. DOI: 10.1016/j.energy.2022.123236. URL: https://linkinghub.elsevier.com/retrieve/pii/S0360544222001396 (visited on 01/14/2025) (cited on p. 19).

[40] Chunyu Wang, Naxin Cui, Zhongrui Cui, Haitao Yuan, and Chenghui Zhang. "Fusion Estimation of Lithium-Ion Battery State of Charge and State of Health Considering the Effect of Temperature". In: *Journal of Energy Storage* 53 (Sept. 2022), p. 105075. DOI: 10.1016/j.est.2022.105075. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X22010775 (visited on 01/14/2025) (cited on pp. 19, 81, 130).

[41] Le Xu, Zhongwei Deng, Yi Xie, Xianke Lin, and Xiaosong Hu. "A Novel Hybrid Physics-Based and Data-Driven Approach for Degradation Trajectory Prediction in Li-Ion Batteries". In: *IEEE Transactions on Transportation Electrification* 9.2 (June 2023), pp. 2628–2644. DOI: 10.1109/TTE.2022.3212024. URL: https://ieeexplore.ieee.org/document/9911693/ (visited on 04/14/2026) (cited on p. 19).

[42] Hui Pang, Longxing Wu, Jiahao Liu, Xiaofei Liu, and Kai Liu. "Physics-Informed Neural Network Approach for Heat Generation Rate Estimation of Lithium-Ion Battery under Various Driving Conditions". In: *Journal of Energy Chemistry* 78 (Mar. 2023), pp. 1–12. DOI: 10.1016/j.jechem.2022.11.036. URL: https://linkinghub.elsevier.com/retrieve/pii/S2095495622006349 (visited on 04/14/2026) (cited on p. 19).

[43] Ruth Cordova-Cardenas, Daniel Amor, and Álvaro Gutiérrez. "Edge AI in Practice: A Survey and Deployment Framework for Neural Networks on Embedded Systems". In: *Electronics* 14.24 (Dec. 11, 2025), p. 4877. DOI:

10.3390/electronics14244877. URL: https://www.mdpi.com/2079-9292/14/24/4877 (visited on 06/24/2026) (cited on p. 21).

[44] Davis Blalock, Jose Javier Gonzalez Ortiz, Jonathan Frankle, and John Guttag. *What Is the State of Neural Network Pruning?* Version 1. 2020. DOI: 10.48550/ARXIV.2003.03033. URL: https://arxiv.org/abs/2003.03033 (visited on 12/02/2025). Pre-published (cited on pp. 21, 22, 27, 30).

[45] Hongrong Cheng, Miao Zhang, and Javen Qinfeng Shi. "A Survey on Deep Neural Network Pruning: Taxonomy, Comparison, Analysis, and Recommendations". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.12 (Dec. 2024), pp. 10558–10578. DOI: 10.1109/TPAMI.2024.3447085. URL: https://doi.org/10.1109/TPAMI.2024.3447085 (visited on 07/25/2026) (cited on pp. 21, 22, 24, 26, 27).

[46] Yang He and Lingao Xiao. "Structured Pruning for Deep Convolutional Neural Networks: A Survey". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.5 (May 2024), pp. 2900–2919. DOI: 10.1109/TPAMI.2023.3334614. URL: https://doi.org/10.1109/TPAMI.2023.3334614 (visited on 07/25/2026) (cited on pp. 21, 23).

[47] Wei Wen, Chunpeng Wu, Yandan Wang, Yiran Chen, and Hai Li. *Learning Structured Sparsity in Deep Neural Networks*. Version 4. 2016. DOI: 10.48550/ARXIV.1608.03665. URL: https://arxiv.org/abs/1608.03665 (visited on 12/02/2025). Pre-published (cited on pp. 23, 27, 30, 111).

[48] Zhuang Liu, Jianguo Li, Zhiqiang Shen, Gao Huang, Shoumeng Yan, and Changshui Zhang. "Learning Efficient Convolutional Networks through Network Slimming". In: *Proceedings of the IEEE International Conference on Computer Vision*. Oct. 2017, pp. 2736–2744. DOI: 10.1109/ICCV.2017.298. URL: https://openaccess.thecvf.com/content_iccv_2017/html/Liu_Learning_Efficient_Convolutional_ICCV_2017_paper.html (visited on 07/27/2026) (cited on pp. 23, 27).

[49] TensorFlow Model Optimization. *Pruning Comprehensive Guide*. 2026. URL: https://www.tensorflow.org/model_optimization/guide/pruning/comprehensive_guide (visited on 07/27/2026) (cited on pp. 24, 30).

[50] Trevor Gale, Erich Elsen, and Sara Hooker. *The State of Sparsity in Deep Neural Networks*. Feb. 2019. DOI: 10.48550/arXiv.1902.09574. arXiv: 190

2.09574 [cs.LG]. URL: https://arxiv.org/abs/1902.09574 (visited on 07/27/2026) (cited on p. 26).

[51] Yann LeCun, John S. Denker, and Sara A. Solla. "Optimal Brain Damage". In: *Advances in Neural Information Processing Systems 2*. Morgan-Kaufmann, 1989, pp. 598–605. URL: https://proceedings.neurips.cc/paper/1989/hash/6c9882bbac1c7093bd25041881277658-Abstract.html (visited on 07/27/2026) (cited on p. 26).

[52] Pavlo Molchanov, Arun Mallya, Stephen Tyree, Iuri Frosio, and Jan Kautz. "Importance Estimation for Neural Network Pruning". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. June 2019, pp. 11264–11272. DOI: 10.1109/CVPR.2019.01152. URL: https://openaccess.thecvf.com/content_CVPR_2019/html/Molchanov_Importance_Estimation_for_Neural_Network_Pruning_CVPR_2019_paper.html (visited on 07/27/2026) (cited on p. 26).

[53] Christos Louizos, Max Welling, and Diederik P. Kingma. "Learning Sparse Neural Networks through L_0 Regularization". In: *International Conference on Learning Representations*. 2018. URL: https://openreview.net/forum?id=H1Y8hhg0b (visited on 07/27/2026) (cited on pp. 27, 30).

[54] Jonathan Frankle and Michael Carbin. "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks". In: *International Conference on Learning Representations*. 2019. URL: https://openreview.net/forum?id=rJl-b3RcF7 (visited on 07/27/2026) (cited on p. 28).

[55] Alex Renda, Jonathan Frankle, and Michael Carbin. "Comparing Rewinding and Fine-Tuning in Neural Network Pruning". In: *International Conference on Learning Representations*. 2020. URL: https://openreview.net/forum?id=S1gSj0NKvB (visited on 07/27/2026) (cited on p. 28).

[56] Namhoon Lee, Thalaiyasingam Ajanthan, and Philip H. S. Torr. "SNIP: Single-Shot Network Pruning Based on Connection Sensitivity". In: *International Conference on Learning Representations*. 2019. URL: https://openreview.net/forum?id=B1VZqjAcYX (visited on 07/27/2026) (cited on p. 28).

[57] Decebal Constantin Mocanu, Elena Mocanu, Peter Stone, Phuong H. Nguyen, Madeleine Gibescu, and Antonio Liotta. "Scalable Training of Artificial Neural Networks with Adaptive Sparse Connectivity Inspired by Network Sci-

ence". In: *Nature Communications* 9 (June 2018), p. 2383. DOI: 10.1038/s41467-018-04316-3. URL: https://www.nature.com/articles/s41467-018-04316-3 (visited on 07/27/2026) (cited on p. 28).

[58] Utku Evci, Trevor Gale, Jacob Menick, Pablo Samuel Castro, and Erich Elsen. "Rigging the Lottery: Making All Tickets Winners". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 2943–2952. URL: https://proceedings.mlr.press/v119/evci20a.html (visited on 07/27/2026) (cited on p. 28).

[59] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Dec. 3, 2019. DOI: 10.48550/arXiv.1912.01703. arXiv: 1912.01703 [cs]. URL: http://arxiv.org/abs/1912.01703 (visited on 12/02/2025). Pre-published (cited on pp. 30, 109).

[60] PyTorch Contributors. *Pruning Tutorial*. 2026. URL: https://docs.pytorch.org/tutorials/intermediate/pruning_tutorial.html (visited on 07/27/2026) (cited on p. 30).

[61] Microsoft. *Overview of NNI Model Pruning*. 2023. URL: https://nni.readthedocs.io/en/stable/compression/pruning.html (visited on 07/27/2026) (cited on p. 30).

[62] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Tijmen Blankevoort. *A White Paper on Neural Network Quantization*. June 15, 2021. DOI: 10.48550/arXiv.2106.08295. arXiv: 2106.08295 [cs.LG]. URL: https://arxiv.org/abs/2106.08295 (visited on 07/27/2026). Pre-published (cited on pp. 32, 34–36, 39).

[63] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. *A Survey of Quantization Methods for Efficient Neural Network Inference*. June 21, 2021. DOI: 10.48550/arXiv.2103.13630. arXiv: 2103.13630 [cs.CV]. URL: https://arxiv.org/abs/2103.13630 (visited on 07/27/2026). Pre-published (cited on pp. 32, 39).

[64] Ron Banner, Yury Nahshan, and Daniel Soudry. "Post Training 4-Bit Quantization of Convolutional Networks for Rapid-Deployment". In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc., 2019. URL: https://proceedings.neurips.cc/paper/2019/hash/c0a62e133894cdce435bcb4a5df1db2d-Abstract.html (visited on 07/27/2026) (cited on pp. 35, 39).

[65] Markus Nagel, Rana Ali Amjad, Mart van Baalen, Christos Louizos, and Tijmen Blankevoort. "Up or Down? Adaptive Rounding for Post-Training Quantization". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 7197–7206. URL: https://proceedings.mlr.press/v119/nagel20a.html (visited on 07/27/2026) (cited on pp. 35, 39).

[66] Raghuraman Krishnamoorthi. *Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper*. Version 1. 2018. DOI: 10.48550/ARXIV.1806.08342. URL: https://arxiv.org/abs/1806.08342 (visited on 12/02/2025). Prepublished (cited on pp. 36, 113, 130).

[67] Mark Horowitz. "1.1 Computing's Energy Problem and What We Can Do about It". In: *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers*. IEEE, 2014, pp. 10–14. DOI: 10.1109/ISSCC.2014.6757323. URL: https://doi.org/10.1109/ISSCC.2014.6757323 (visited on 07/27/2026) (cited on p. 37).

[68] Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. "Learned Step Size Quantization". In: *International Conference on Learning Representations*. 2020. URL: https://openreview.net/forum?id=rkgO66VKDS (visited on 07/27/2026) (cited on p. 39).

[69] PyTorch. *Quantization Overview*. PyTorch. Mar. 25, 2026. URL: https://docs.pytorch.org/ao/stable/contributing/quantization_overview.html (visited on 07/27/2026) (cited on p. 41).

[70] TensorFlow. *Post-Training Quantization*. Google. Aug. 3, 2022. URL: https://www.tensorflow.org/model_optimization/guide/quantization/post_training (visited on 07/27/2026) (cited on p. 41).

[71] Microsoft. *Quantize ONNX Models*. ONNX Runtime. URL: https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html (visited on 07/27/2026) (cited on p. 41).

[72] Spyridon Giazitzis, Maciej Sakwa, Sonia Leva, Emanuele Ogliari, Susheel Badha, and Filippo Rosetti. "A Case Study of a Tiny Machine Learning Application for Battery State-of-Charge Estimation". In: *Electronics* 13.10 (May 16, 2024), p. 1964. DOI: 10.3390/electronics13101964. URL: https://www.mdpi.com/2079-9292/13/10/1964 (visited on 05/19/2025) (cited on pp. 41, 105, 129).

[73] Giulia Crocioni, Danilo Pau, Jean-Michel Delorme, and Giambattista Gruosso. "Li-Ion Batteries Parameter Estimation With Tiny Neural Networks Embedded on Intelligent IoT Microcontrollers". In: *IEEE Access* 8 (2020), pp. 122135–122146. DOI: 10.1109/ACCESS.2020.3007046. URL: https://ieeexplore.ieee.org/document/9133084/ (visited on 05/19/2025) (cited on pp. 41, 105).

[74] Danilo Pietro Pau and Alberto Aniballi. "Tiny Machine Learning Battery State-of-Charge Estimation Hardware Accelerated". In: *Applied Sciences* 14.14 (July 18, 2024), p. 6240. DOI: 10.3390/app14146240. URL: https://www.mdpi.com/2076-3417/14/14/6240 (visited on 05/19/2025) (cited on pp. 41, 105).

[75] Muh-Tian Shiue, Yang-Chieh Ou, Chih-Feng Wu, Yi-Fong Wang, and Bing-Jun Liu. "Design and Implementation of a Decentralized Node-Level Battery Management System Chip Based on Deep Neural Network Algorithms". In: *Electronics* 15.2 (Jan. 9, 2026), p. 296. DOI: 10.3390/electronics15020296. URL: https://www.mdpi.com/2079-9292/15/2/296 (visited on 06/24/2026) (cited on p. 41).

[76] LEAP-RE. *MG FARM Smart Stand-Alone Micro-Grids as a Solution for Agriculture Farms Electrification*. URL: https://www.leap-re.eu/mg-farm/ (cited on pp. 43, 106).

[77] Microenergy. *MG-FARM Powering Sustainable Agriculture with Smart Micro-Grids*. URL: https://mg-farm.microenergy-consulting.com/ (cited on pp. 43, 106).

[78] Technische Universität Berlin. *MG Farm Smart Microgrids as a Solution for Agriculture Farms Electrification*. URL: https://www.tu.berlin/eet/forschung/projekte/mg-farm (cited on pp. 43, 106).

[79] LEAP-RE. *LEAP-RE Joint Call 2021 for AU–EU Collaborative Research and Innovation Projects on Renewable Energy*. Apr. 13, 2021. URL: https://leap-energy.eu/news-124-applications-received-during-1st-phase-of-the-leap-re-call-for-projects (visited on 07/23/2026) (cited on p. 43).

[80] European Commission. *Long-Term Joint EU–AU Research and Innovation Partnership on Renewable Energy*. CORDIS. DOI: 10.3030/963530. URL: https://cordis.europa.eu/project/id/963530 (visited on 07/23/2026) (cited on p. 43).

[81] EnArgus. *LEAP-RE: Verbundvorhaben MG-Farm, Teilvorhaben: Intelligente Microgrids als Lösung für die Elektrifizierung von landwirtschaftlichen Betrieben*. Project funding reference 03SF0684A. URL: https://enargus.de/ (visited on 07/23/2026) (cited on p. 43).

[82] Florian Rzepka. *Design of Experiments for Cyclic Aging of LFP 18650 Cells: Impact of Charge/Discharge C-Rates, Depth of Discharge, and Temperature on Battery Degradation*. Technische Universität Berlin, Nov. 13, 2025. DOI: 10.14279/DEPOSITONCE-24800. URL: https://depositonce.tu-berlin.de/handle/11303/25972 (visited on 12/02/2025) (cited on pp. 44, 46, 85, 106).

[83] Karl Siebertz, David Van Bebber, and Thomas Hochkirchen. *Statistische Versuchsplanung*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2017. DOI: 10.1007/978-3-662-55743-3. URL: http://link.springer.com/10.1007/978-3-662-55743-3 (visited on 12/03/2025) (cited on p. 47).

[84] Younes Sahri, Youcef Belkhier, Salah Tamalouzt, Nasim Ullah, Rabindra Nath Shaw, Md. Shahariar Chowdhury, and Kuaanan Techato. "Energy Management System for Hybrid PV/Wind/Battery/Fuel Cell in Microgrid-Based Hydrogen and Economical Hybrid Battery/Super Capacitor Energy Storage". In: *Energies* 14.18 (Sept. 11, 2021), p. 5722. DOI: 10.3390/en14185722. URL: https://www.mdpi.com/1996-1073/14/18/5722 (visited on 04/14/2026) (cited on p. 57).

[85] Mohamed S. Soliman, Youcef Belkhier, Nasim Ullah, Abdelyazid Achour, Yasser M. Alharbi, Ahmad Aziz Al Alahmadi, Habti Abeida, and Yahya Salameh Hassan Khraisat. "Supervisory Energy Management of a Hybrid

Battery/PV/Tidal/Wind Sources Integrated in DC-microgrid Energy Storage System". In: *Energy Reports* 7 (Nov. 2021), pp. 7728–7740. DOI: 10.1016/j.egyr.2021.11.05 URL: https://linkinghub.elsevier.com/retrieve/pii/S2352484721012014 (visited on 04/14/2026) (cited on p. 57).

[86] Fernando Puente León and HolgerBG Jäkel. *Signale Und Systeme:* DE GRUYTER, Sept. 25, 2015. DOI: 10.1515/9783110403862. URL: https://www.degruyterbrill.com/document/doi/10.1515/9783110403862/html (visited on 04/14/2026) (cited on p. 61).

[87] Tim Salimans and Diederik P. Kingma. *Weight Normalization: A Simple Reparameterization to Accelerate Training of Deep Neural Networks*. Version 3. 2016. DOI: 10.48550/ARXIV.1602.07868. URL: https://arxiv.org/abs/1602.07868 (visited on 04/14/2026). Pre-published (cited on p. 63).

[88] Hans Weytjens, Enrico Lohmann, and Martin Kleinsteuber. "Cash Flow Prediction: MLP and LSTM Compared to ARIMA and Prophet". In: *Electronic Commerce Research* 21.2 (June 2021), pp. 371–391. DOI: 10.1007/s10660-019-09362-7. URL: https://link.springer.com/10.1007/s10660-019-09362-7 (visited on 04/14/2026) (cited on p. 63).

[89] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization". Version 4. In: (2016). DOI: 10.48550/ARXIV.1603.06560. URL: https://arxiv.org/abs/1603.06560 (visited on 04/14/2026) (cited on pp. 63, 65).

[90] Jiaqi Yao and Julia Kowal. "Towards a Smarter Battery Management System: A Critical Review on Deep Learning-Based State of Charge Estimation of Lithium-Ion Batteries". In: *Energy and AI* 21 (Sept. 2025), p. 100585. DOI: 10.1016/j.egyai.2025.100585. URL: https://linkinghub.elsevier.com/retrieve/pii/S266654682500117X (visited on 03/23/2026) (cited on pp. 71, 72, 75).

[91] Pratik Mondal, Divyakumar Bhavsar, Kanupriya Mittal, and Mayank Mittal. "Estimating State-of-Charge in Lithium-Ion Batteries Through Deep Learning Techniques: A Comparative Evaluation". In: *IEEE Access* 12 (2024), pp. 78773–78786. DOI: 10.1109/ACCESS.2024.3408220. URL: https://ieeexplore.ieee.org/document/10546296/ (visited on 03/23/2026) (cited on pp. 72, 75).

[92] M. Adib Kamali, Angela C. Caliwag, and Wansu Lim. "Novel SOH Estimation of Lithium-Ion Batteries for Real-Time Embedded Applications". In: *IEEE Embedded Systems Letters* 13.4 (Dec. 2021), pp. 206–209. DOI: 10.1109/LES.2021.3078443. URL: https://ieeexplore.ieee.org/document/9426956/ (visited on 05/19/2025) (cited on pp. 76, 105).

[93] Bin Gou, Yan Xu, and Xue Feng. "An Ensemble Learning-Based Data-Driven Method for Online State-of-Health Estimation of Lithium-Ion Batteries". In: *IEEE Transactions on Transportation Electrification* 7.2 (June 2021), pp. 422–436. DOI: 10.1109/TTE.2020.3029295. URL: https://ieeexplore.ieee.org/document/9216038/ (visited on 01/14/2025) (cited on p. 76).

[94] Weihan Li, Neil Sengupta, Philipp Dechent, David Howey, Anuradha Annaswamy, and Dirk Uwe Sauer. "Online Capacity Estimation of Lithium-Ion Batteries with Deep Long Short-Term Memory Networks". In: *Journal of Power Sources* 482 (Jan. 2021), p. 228863. DOI: 10.1016/j.jpowsour.2020.228863. URL: https://linkinghub.elsevier.com/retrieve/pii/S0378775320311678 (visited on 01/14/2025) (cited on pp. 76, 129).

[95] Orazio Aiello. "Electromagnetic Susceptibility of Battery Management Systems' ICs for Electric Vehicles: Experimental Study". In: *Electronics* 9.3 (Mar. 19, 2020), p. 510. DOI: 10.3390/electronics9030510. URL: https://www.mdpi.com/2079-9292/9/3/510 (visited on 04/04/2026) (cited on pp. 80, 81).

[96] Mengxi Liu, Daniel Geißler, Sizhen Bian, Bo Zhou, and Paul Lukowicz. *Assessing the Impact of Sampling Irregularity in Time Series Data: Human Activity Recognition As A Case Study*. Jan. 25, 2025. DOI: 10.48550/arXiv.2501.15330. arXiv: 2501.15330 [eess]. URL: http://arxiv.org/abs/2501.15330 (visited on 04/04/2026). Pre-published (cited on pp. 80, 81).

[97] Jiaqi Yao and Julia Kowal. "Towards a Smarter Battery Management System: A Critical Review on Deep Learning-Based State of Charge Estimation of Lithium-Ion Batteries". In: *Energy and AI* 21 (Sept. 2025), p. 100585. DOI: 10.1016/j.egyai.2025.100585. URL: https://linkinghub.elsevier.com/retrieve/pii/S266654682500117X (visited on 12/04/2025) (cited on p. 105).

[98] A. M. S. M. H. S. Attanayaka, J. P. Karunadasa, K. T. M. U. Hemapala, and Department of Electrical Engineering, University of Moratuwa, Moratuwa, Sri Lanka. "Estimation of State of Charge for Lithium-Ion Batteries - A Re-

view". In: *AIMS Energy* 7.2 (2019), pp. 186–210. DOI: 10.3934/energy.2019.2.186. URL: http://www.aimspress.com/article/10.3934/energy.2019.2.186 (visited on 12/02/2025) (cited on p. 105).

[99] Wenjie Sun, Bangyu Zhou, Huan Xu, Wenjiao Yao, Yuanjun Guo, and Zhile Yang. "State of Charge Estimation of Sodium-Ion Batteries Based on N-BEATS Neural Network". In: *2023 7th Asian Conference on Artificial Intelligence Technology (ACAIT)*. 2023 7th Asian Conference on Artificial Intelligence Technology (ACAIT). Jiaxing, China: IEEE, Nov. 10, 2023, pp. 1395–1401. DOI: 10.1109/ACAIT60137.2023.10528411. URL: https://ieeexplore.ieee.org/document/10528411/ (visited on 01/13/2025) (cited on p. 105).

[100] Wenjie Sun, Huan Xu, Bangyu Zhou, Yuanjun Guo, Yongbing Tang, Wenjiao Yao, and Zhile Yang. "State-of-Charge Estimation of Sodium-Ion Batteries: A Fusion Deep Learning Approach". In: *Journal of Energy Storage* 91 (June 2024), p. 111527. DOI: 10.1016/j.est.2024.111527. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X24011125 (visited on 01/13/2025) (cited on p. 105).

[101] Shunli Wang, Chao Wang, Paul Takyi-Aninakwa, Siyu Jin, Carlos Fernandez, and Qi Huang. "An improved parameter identification and radial basis correction-differential support vector machine strategies for state-of-charge estimation of urban-transportation-electric-vehicle lithium-ion batteries". en. In: *Journal of Energy Storage* 80 (Mar. 2024), p. 110222. DOI: 10.1016/j.est.2023.110222. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X23036216 (cited on p. 105).

[102] Shunli Wang, Fan Wu, Paul Takyi-Aninakwa, Carlos Fernandez, Daniel-Ioan Stroe, and Qi Huang. "Improved singular filtering-Gaussian process regression-long short-term memory model for whole-life-cycle remaining capacity estimation of lithium-ion batteries adaptive to fast aging and multi-current variations". en. In: *Energy* 284 (Dec. 2023), p. 128677. DOI: 10.1016/j.energy.2023.128 URL: https://linkinghub.elsevier.com/retrieve/pii/S0360544223020716 (cited on p. 105).

[103] A. Aqachtoul, K. Karam, A. Elamrani, Y. Alidrissi, M. Najib, A. Rochd, and M. Bakhouya. "MGFARM: An IoT and Edge-Driven Framework for Smart and Sustainable Agricultural Practices". In: *Connected Objects, Artificial Intelligence, Telecommunications and Electronics Engineering*. Ed. by

Youssef Mejdoub, Abdelkebir Elamri, and Mustapha Kardouchi. Vol. 1584. Cham: Springer Nature Switzerland, 2026, pp. 419–425. DOI: 10.1007/978-3-032-01536-5_64. URL: https://link.springer.com/10.1007/978-3-032-01536-5_64 (visited on 12/03/2025) (cited on p. 106).

[104] JGNE. *JGCFR18650-1800mAh-3.2V Product Specification*. JGNE. URL: https://www.goldencellpower.com/product-item/18650-1800mah-3-2v-lifepo4-cells/ (cited on p. 106).

[105] Peter J. Huber and Elvezio M. Ronchetti. *Robust Statistics*. 1st ed. Wiley Series in Probability and Statistics. Wiley, Jan. 29, 2009. DOI: 10.1002/9780470434697. URL: https://onlinelibrary.wiley.com/doi/book/10.1002/9780470434697 (visited on 12/02/2025) (cited on p. 107).

[106] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Andreas Müller, Joel Nothman, Gilles Louppe, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. "Scikit-Learn: Machine Learning in Python". Version 4. In: (2012). DOI: 10.48550/ARXIV.1201.0490. URL: https://arxiv.org/abs/1201.0490 (visited on 12/02/2025) (cited on p. 107).

[107] Ilya Loshchilov and Frank Hutter. *Decoupled Weight Decay Regularization*. Version 3. 2017. DOI: 10.48550/ARXIV.1711.05101. URL: https://arxiv.org/abs/1711.05101 (visited on 12/02/2025). Pre-published (cited on p. 110).

[108] Ilya Loshchilov and Frank Hutter. *SGDR: Stochastic Gradient Descent with Warm Restarts*. Version 5. 2016. DOI: 10.48550/ARXIV.1608.03983. URL: https://arxiv.org/abs/1608.03983 (visited on 12/02/2025). Pre-published (cited on p. 110).

[109] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. *Mixed Precision Training*. Version 3. 2017. DOI: 10.48550/ARXIV.1710.03740. URL: https://arxiv.org/abs/1710.03740 (visited on 12/02/2025). Pre-published (cited on p. 110).

[110] Ekaterina Lobacheva, Nadezhda Chirkova, Alexander Markovich, and Dmitry Vetrov. "Structured Sparsification of Gated Recurrent Neural Networks". In: *Proceedings of the AAAI Conference on Artificial Intelligence* 34.04 (Apr. 3, 2020), pp. 4989–4996. DOI: 10.1609/aaai.v34i04.5938. URL: https://ojs.aaai

.org/index.php/AAAI/article/view/5938 (visited on 12/08/2025) (cited on p. 111).

[111] Sharan Narang, Erich Elsen, Gregory Diamos, and Shubho Sengupta. *Exploring Sparsity in Recurrent Neural Networks*. Version 2. 2017. DOI: 10.48550/ARXIV.1704.0 URL: https://arxiv.org/abs/1704.05119 (visited on 12/02/2025). Prepublished (cited on p. 111).

[112] Jonas Schulze, Nils Strassenburg, and Tilmann Rabl. "PQ Bench: Benchmarking Pruning and Quantization Techniques". In: *Proceedings of the Workshop on Data Management for End-to-End Machine Learning*. SIGMOD/PODS '25: International Conference on Management of Data. Berlin Germany: ACM, June 22, 2025, pp. 1–6. DOI: 10.1145/3735654.3735940. URL: https://dl.acm.org/doi/10.1145/3735654.3735940 (visited on 12/05/2025) (cited on p. 113).

[113] *ONNX Runtime: High-Performance Inference Engine*. URL: https://onnxruntime.ai/ (cited on p. 114).

[114] *NVIDIA TensorRT: High-Performance Deep Learning Inference*. URL: https://developer.nvidia.com/tensorrt (cited on p. 114).

[115] *X-CUBE-AI: STM32Cube Expansion Package for Artificial Intelligence*. URL: https://www.st.com/en/embedded-software/x-cube-ai.html (cited on p. 114).

[116] *STM32 Nucleo-144 Boards*. STMicroelectronics, Dec. 2, 2025. URL: https://www.st.com/resource/en/data_brief/nucleo-h753zi.pdf (visited on 12/02/2025) (cited on p. 115).

[117] Wei Wen, Yuxiong He, Samyam Rajbhandari, Minjia Zhang, Wenhan Wang, Fang Liu, Bin Hu, Yiran Chen, and Hai Li. *Learning Intrinsic Sparse Structures within Long Short-Term Memory*. Version 7. 2017. DOI: 10.48550/ARXIV.1709.05027. URL: https://arxiv.org/abs/1709.05027 (visited on 12/08/2025). Prepublished (cited on p. 130).

[118] S. Singh and P.R. Budarapu. "Deep Machine Learning Approaches for Battery Health Monitoring". In: *Energy* 300 (Aug. 2024), p. 131540. DOI: 10.1016/j.energy.2024.1 URL: https://linkinghub.elsevier.com/retrieve/pii/S0360544224013136 (visited on 01/13/2025) (cited on p. 130).

Appendix

Supplementary Material A

This appendix contains supplementary tables, additional hardware documentation, experiment matrices, and publication-specific material that would interrupt the flow of the main text if it were placed directly in the core chapters.

A.1 Supplementary Robustness-Benchmark Result Tables

The following tables collect the numerical robustness-benchmark values used in Chapter 5. Unless stated otherwise, SOC error metrics are reported on the normalized 0 to 1 SOC scale. Multiplying by 100 gives percentage points.

Table A.1: Baseline performance metrics for the selected robustness-benchmark trajectory. Values are reported on the normalized 0 to 1 SOC scale.

Class	MAE	RMSE	P95 error	Max error	Bias
DM	0.0311	0.0397	0.0800	1.0000	0.0311
HDM	0.0024	0.0061	0.0094	1.0000	0.0022
HECM	0.0090	0.0127	0.0241	1.0000	-0.0029
DD	0.0100	0.0143	0.0336	0.0686	-0.0062

Table A.2: Cross-scenario global performance metrics used in the Chapter 5 result discussion. Values are reported on the normalized 0 to 1 SOC scale.

Scenario	Class	MAE	ΔMAE	RMSE	ΔRMSE	P95 error
Current noise (high)	DM	0.0310	-0.0001	0.0394	-0.0002	0.0795
Current bias	DM	0.3143	0.2833	0.3683	0.3286	0.6670
Initial SOC error	DM	0.0322	0.0011	0.0413	0.0017	0.0833
Irregular sampling	DM	0.0309	-0.0001	0.0396	-0.0001	0.0799

Scenario	Class	MAE	ΔMAE	RMSE	ΔRMSE	P95 error
Burst dropout	DM	0.0332	0.0022	0.0487	0.0090	0.0819
Missing samples	DM	0.0386	0.0076	0.0476	0.0079	0.0926
Voltage spikes	DM	0.0312	0.0001	0.0398	0.0001	0.0801
Temperature noise	DM	0.0311	0.0000	0.0397	0.0000	0.0800
Voltage noise	DM	0.0524	0.0213	0.0648	0.0252	0.1228
Current noise (high)	HDM	0.0044	0.0019	0.0074	0.0013	0.0120
Current bias	HDM	0.3068	0.3043	0.3638	0.3577	0.6668
Initial SOC error	HDM	0.0036	0.0011	0.0136	0.0075	0.0101
Irregular sampling	HDM	0.0027	0.0003	0.0061	0.0001	0.0095
Burst dropout	HDM	0.0051	0.0027	0.0325	0.0265	0.0098
Missing samples	HDM	0.0103	0.0079	0.0129	0.0069	0.0221
Voltage spikes	HDM	0.0062	0.0038	0.0140	0.0079	0.0328
Temperature noise	HDM	0.0067	0.0043	0.0110	0.0049	0.0210
Voltage noise	HDM	0.0751	0.0727	0.0908	0.0847	0.1578
Current noise (high)	HECM	0.0165	0.0075	0.0280	0.0154	0.0701
Current bias	HECM	0.0932	0.0842	0.1303	0.1176	0.2954
Initial SOC error	HECM	0.0091	0.0001	0.0143	0.0016	0.0241
Irregular sampling	HECM	0.0095	0.0004	0.0131	0.0004	0.0248
Burst dropout	HECM	0.0104	0.0014	0.0228	0.0101	0.0246
Missing samples	HECM	0.0119	0.0029	0.0163	0.0036	0.0329
Voltage spikes	HECM	0.0090	0.0000	0.0127	0.0000	0.0241
Temperature noise	HECM	0.0090	0.0000	0.0127	0.0000	0.0241
Voltage noise	HECM	0.0090	0.0000	0.0127	0.0000	0.0241
Current noise (high)	DD	0.0109	0.0009	0.0151	0.0008	0.0345
Current bias	DD	0.2859	0.2759	0.3426	0.3283	0.6367
Initial SOC error	DD	0.0110	0.0010	0.0175	0.0032	0.0355
Irregular sampling	DD	0.0105	0.0005	0.0144	0.0001	0.0330
Burst dropout	DD	0.0122	0.0022	0.0316	0.0173	0.0348
Missing samples	DD	0.0117	0.0016	0.0149	0.0006	0.0302
Voltage spikes	DD	0.0144	0.0044	0.0213	0.0070	0.0474
Temperature noise	DD	0.0181	0.0081	0.0223	0.0080	0.0451
Voltage noise	DD	0.0698	0.0597	0.0839	0.0696	0.1501

Table A.3: Local behavior and recovery metrics used in the Chapter 5 result discussion. Thresholds are reported on the normalized 0 to 1 SOC scale where applicable.

Class	Scenario	Local metric	Value	Threshold
DM	Initial SOC error	Recovery time to strict band [h]	1.1658	0.0800
DM	Initial SOC error	Recovery time to fair band [h]	0.2500	0.1599

Class	Scenario	Local metric	Value	Threshold
HDM	Initial SOC error	Recovery time to strict band [h]	n.a.	0.0094
HDM	Initial SOC error	Recovery time to fair band [h]	n.a.	0.0200
HECM	Initial SOC error	Recovery time to strict band [h]	2.3693	0.0241
HECM	Initial SOC error	Recovery time to fair band [h]	1.1564	0.0482
DD	Initial SOC error	Recovery time to strict band [h]	1.0464	0.0336
DD	Initial SOC error	Recovery time to fair band [h]	1.0014	0.0672
DM	Burst dropout	Recovery time after burst dropout [h]	27.3511	0.0216
HDM	Burst dropout	Recovery time after burst dropout [h]	27.4093	0.0008
HECM	Burst dropout	Recovery time after burst dropout [h]	28.5611	0.0095
DD	Burst dropout	Recovery time after burst dropout [h]	27.4351	0.0070
DM	Voltage spikes	Median spike recovery time [s]	0.0000	0.0800
HDM	Voltage spikes	Median spike recovery time [s]	0.0000	0.0094
HECM	Voltage spikes	Median spike recovery time [s]	0.0000	0.0241
DD	Voltage spikes	Median spike recovery time [s]	0.0000	0.0336
DM	Current noise (high)	Late minus early excess rolling MAE	-0.0013	n.a.
DM	Current noise (high)	Mean excess rolling MAE	-0.0001	n.a.
HDM	Current noise (high)	Late minus early excess rolling MAE	-0.0011	n.a.
HDM	Current noise (high)	Mean excess rolling MAE	0.0019	n.a.
HECM	Current noise (high)	Late minus early excess rolling MAE	0.0038	n.a.
HECM	Current noise (high)	Mean excess rolling MAE	0.0073	n.a.
DD	Current noise (high)	Late minus early excess rolling MAE	-0.0002	n.a.
DD	Current noise (high)	Mean excess rolling MAE	0.0009	n.a.

Table A.4: Scenario overview linking disturbance configuration, primary evidence type, and the main figure in Chapter 5. This table clarifies which scenarios are interpreted predominantly through global metrics and which require additional local evidence.

Scenario	Configuration	Primary evidence	Main figure(s)
ADC quantization	Steps of 0,01 A, 0,005 V, and 0,5 °C	Global ΔMAE. Detailed values in Appendix Table A.5	Figs. 5.12, A.1

Scenario	Configuration	Primary evidence	Main figure(s)
Current bias	Constant current offset in the compact matrix, plus dedicated 0.5%/1.5%/3.0% sweep of $\lvert I\rvert_{\max}$	Global MAE degradation across the sweep	Fig. 5.5
Current noise	Compact matrix with $\sigma_I = 0{,}10\,\mathrm{A}$ and local detail sweep up to $\sigma_I = 0{,}20\,\mathrm{A}$	Global ΔMAE plus local output-variability and rolling-MAE evidence	Fig. 5.6
Voltage noise	Additive Gaussian noise with $\sigma_U = 0{,}01\,\mathrm{V}$	Global ΔMAE	Fig. 5.12
Temperature noise	Additive Gaussian noise with $\sigma_T = 1{,}0\,^{\circ}\mathrm{C}$	Global ΔMAE	Fig. 5.12
Initial SOC error	Additive initial mismatch of −0.10 SOC, corresponding to 10 percentage points. DD via online Q_c offset	Local recovery to strict and fair baseline bands. Global MAE is secondary	Fig. 5.7
Missing samples	Every 50th sample frozen on the nominal 1 s grid	Global ΔMAE	Figs. 5.8, 5.12
Irregular sampling	Uniform sampling-time jitter of ±0,1 s around the nominal 1 s grid	Global ΔMAE	Figs. 5.8, 5.12
Burst dropout	One central frozen gap of 3600 s	Global ΔMAE plus local transition and long-horizon recovery	Figs. 5.8, 5.9, 5.10

Scenario	Configuration	Primary evidence	Main figure(s)
Voltage spikes	One ±0,20 V single-sample spike every 1000 samples	Global ΔMAE plus event-wise P95 of post-spike peak excess error	Figs. 5.11, 5.12

Table A.5: ADC-quantization results for the four estimator classes. Values are reported on the normalized 0 to 1 SOC scale.

Class	Baseline MAE	ADC MAE	ΔMAE
DM	0.0311	0.0313	+0.0002
HDM	0.0024	0.0029	+0.0005
HECM	0.0090	0.0087	-0.0003
DD	0.0100	0.0098	-0.0002

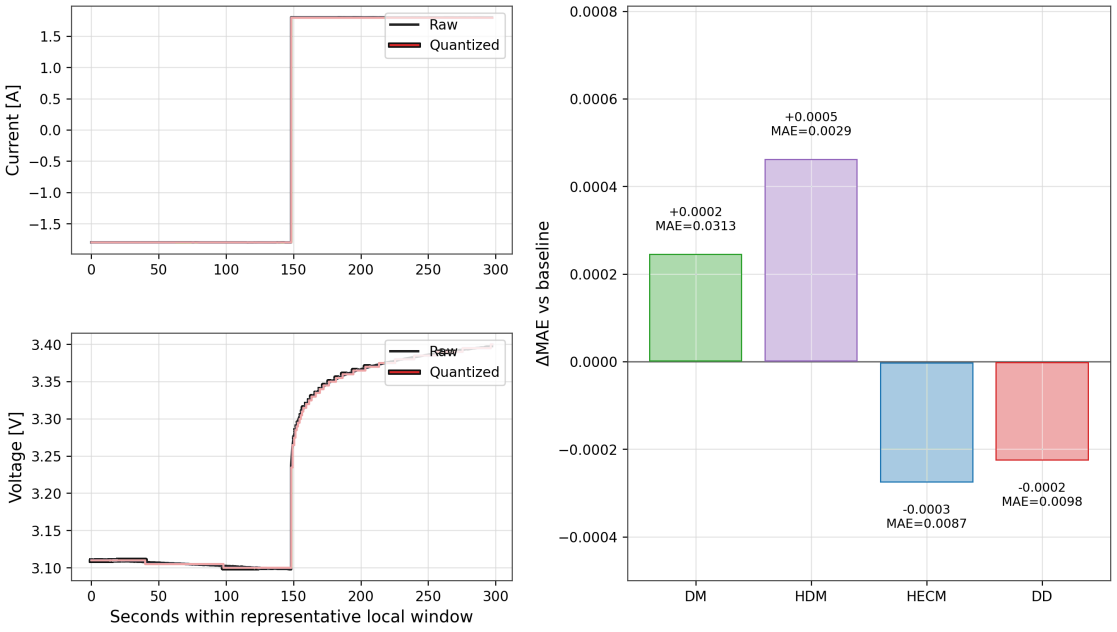


Figure A.1: ADC-quantization results for the robustness benchmark. The left panels illustrate representative current and voltage quantization in a local window, and the right panel summarizes the resulting global ΔMAE relative to the baseline case.

A.2 Supplementary Analyses for Embedded Model Compression

This appendix provides the detailed evidence behind the compression, temporal, filtering, utility, and software-fault analyses discussed in Chapter 6.

A.2.1 Diagnostics of the Structured-Pruning Step

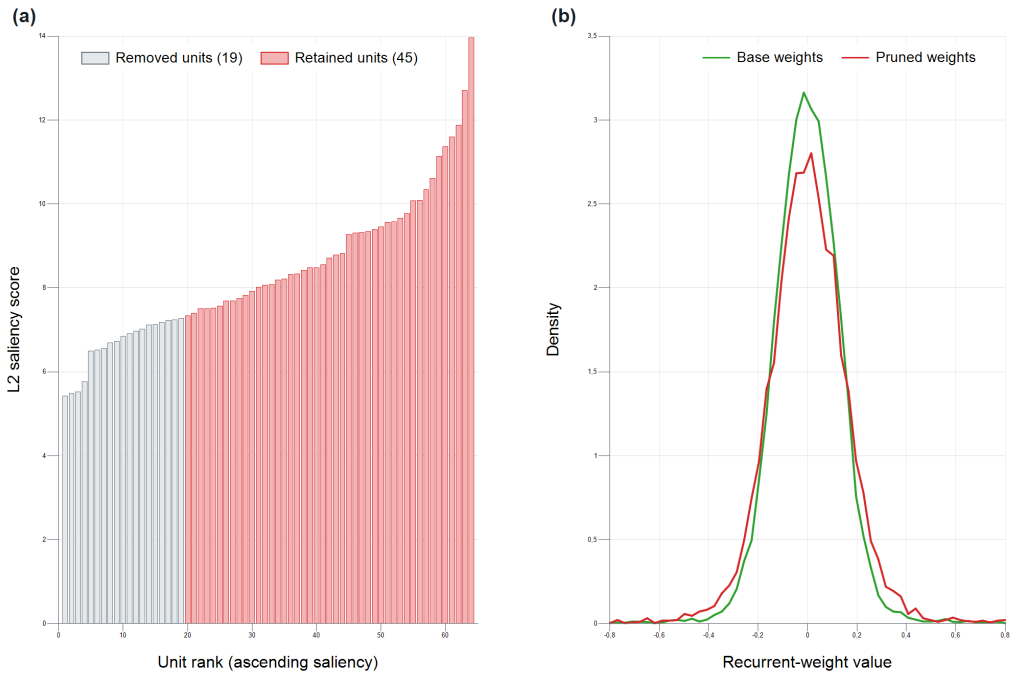


Figure A.2: Diagnostics of the one-shot SOC pruning step. Panel (a) orders hidden channels by the L2 gate-group saliency from Equation 6.11. The 19 lowest-ranked channels are removed, leaving 45 channels. Panel (b) compares the central range of the recurrent-weight distributions before and after this structural selection.

A.2.2 Long-Horizon Behaviour of the Compressed Models

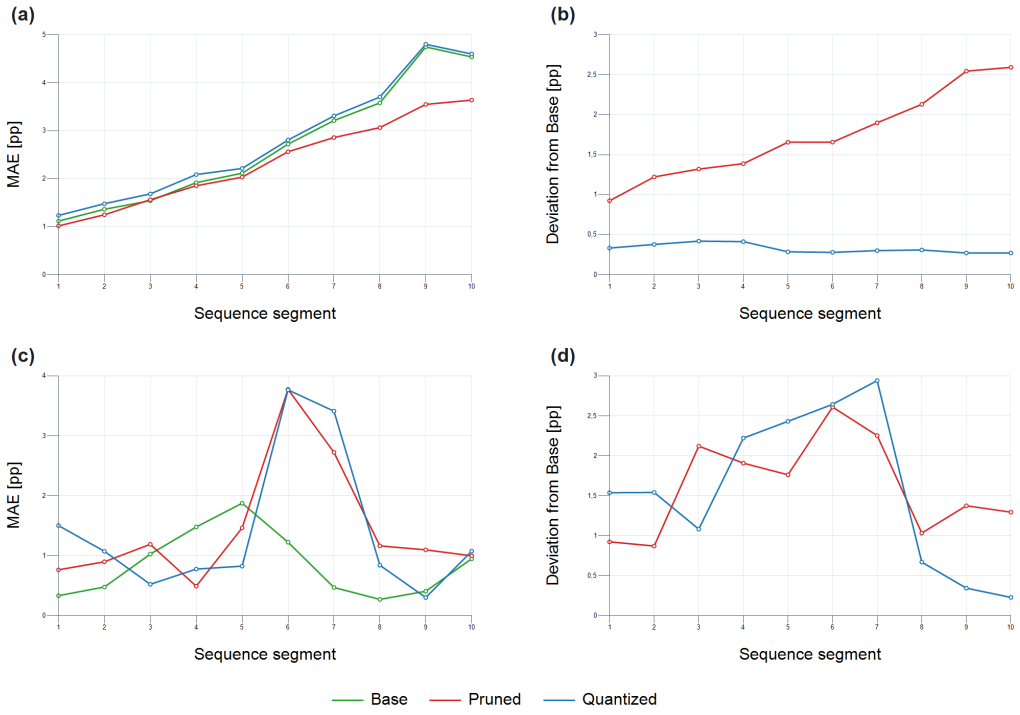


Figure A.3: Segment-wise errors over ten consecutive parts of the natural replay sequence. Recurrent states continue across all boundaries. Target MAE and P95 describe accuracy against the reference labels, while the model-to-Base deviation isolates the relative movement of each compressed output.

A.2.3 Interaction Between SOH Compression and Filtering

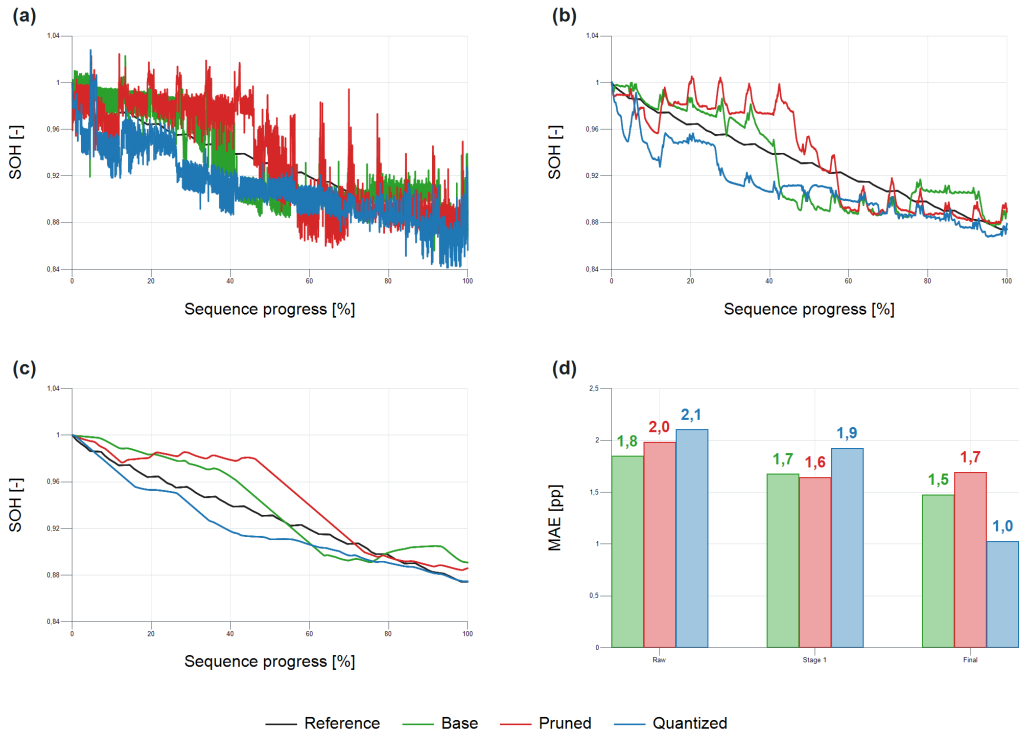


Figure A.4: SOH output and error at successive stages of the causal post-processing chain for Base, Pruned, and Quantized. Comparing raw, stage-1, and final trajectories shows that filtering affects absolute accuracy, ordering between variants, and temporal response.

A.2.4 Sensitivity of the Composite Utility Score

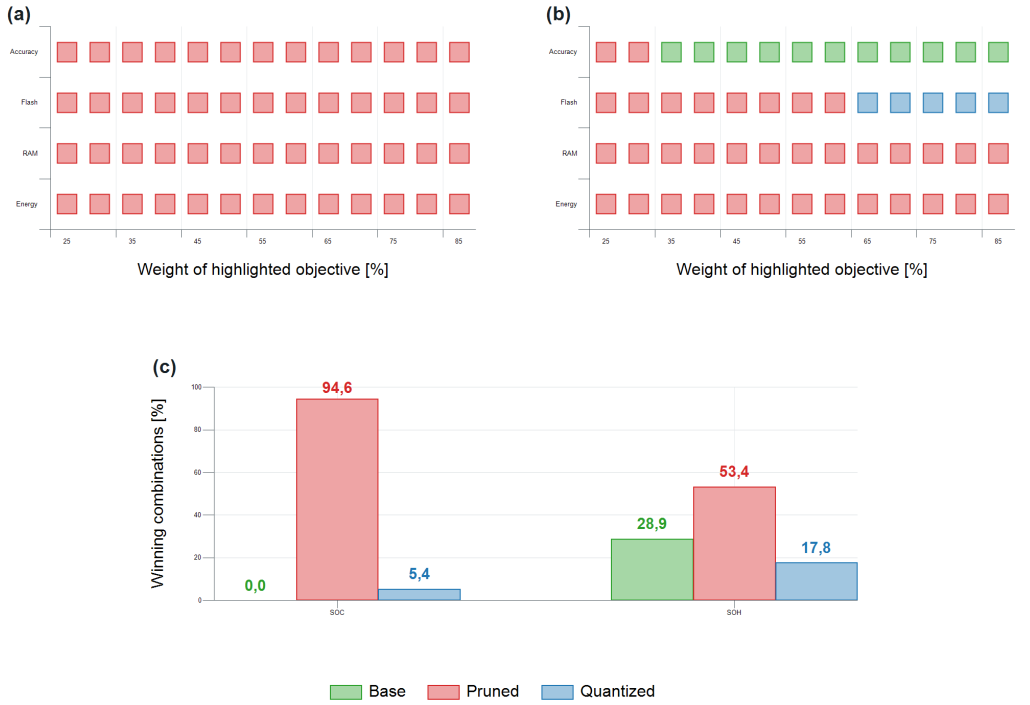


Figure A.5: Sensitivity of the composite utility ranking to different application priorities. The priority sweeps increase one weight while distributing the remaining importance equally. The ranking summary covers all 1771 weight combinations on the 0.05 simplex grid.

A.2.5 Transient Input-Buffer Fault Sensitivity

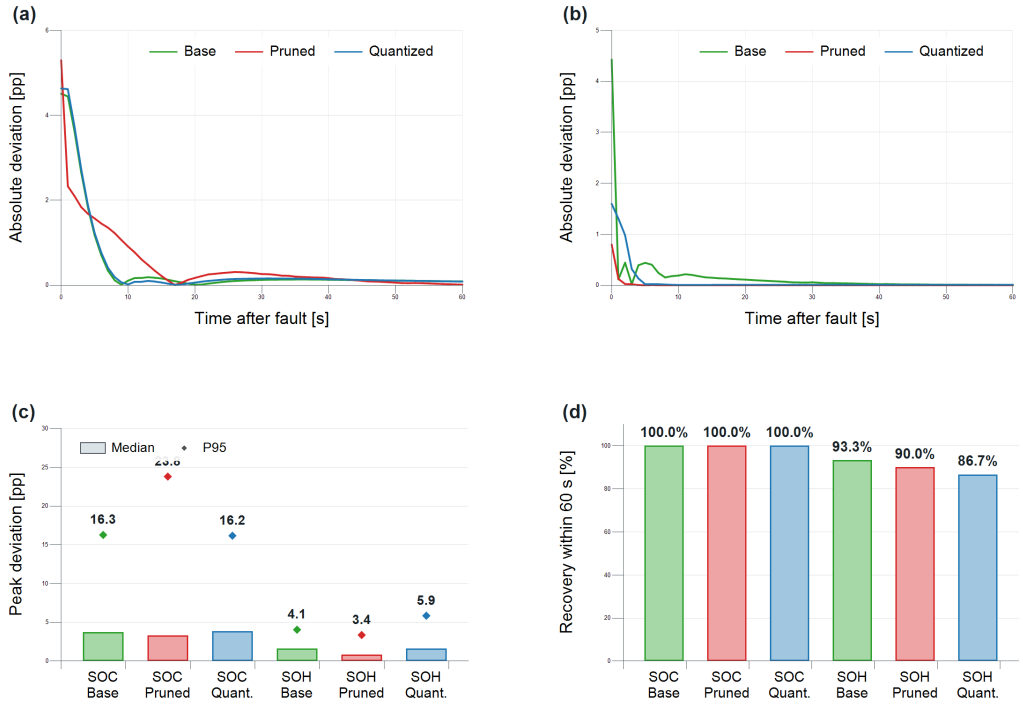


Figure A.6: Transient response of the clean and faulted model branches after a one-sample FP32 fraction-bit change in voltage, current, or temperature. The panels report peak disturbed-to-clean deviation, recovery within the following 60 s, and the target-error change over the complete 61-sample event window.

Table A.6: Median and P95 of the event-wise change in target MAE over each 61-sample fault window. All entries are percentage points.

Task	Model	Median ΔMAE_{61}	P95 ΔMAE_{61}
SOC	Base	0.0740	1.4235
SOC	Pruned	0.0464	1.5867
SOC	Quantized	0.0785	1.4314
SOH	Base	−0.0009	0.1224
SOH	Pruned	−0.00004	0.0533
SOH	Quantized	0.0012	0.1055

A.2.6 Model-Complexity Scaling Under Structured Pruning

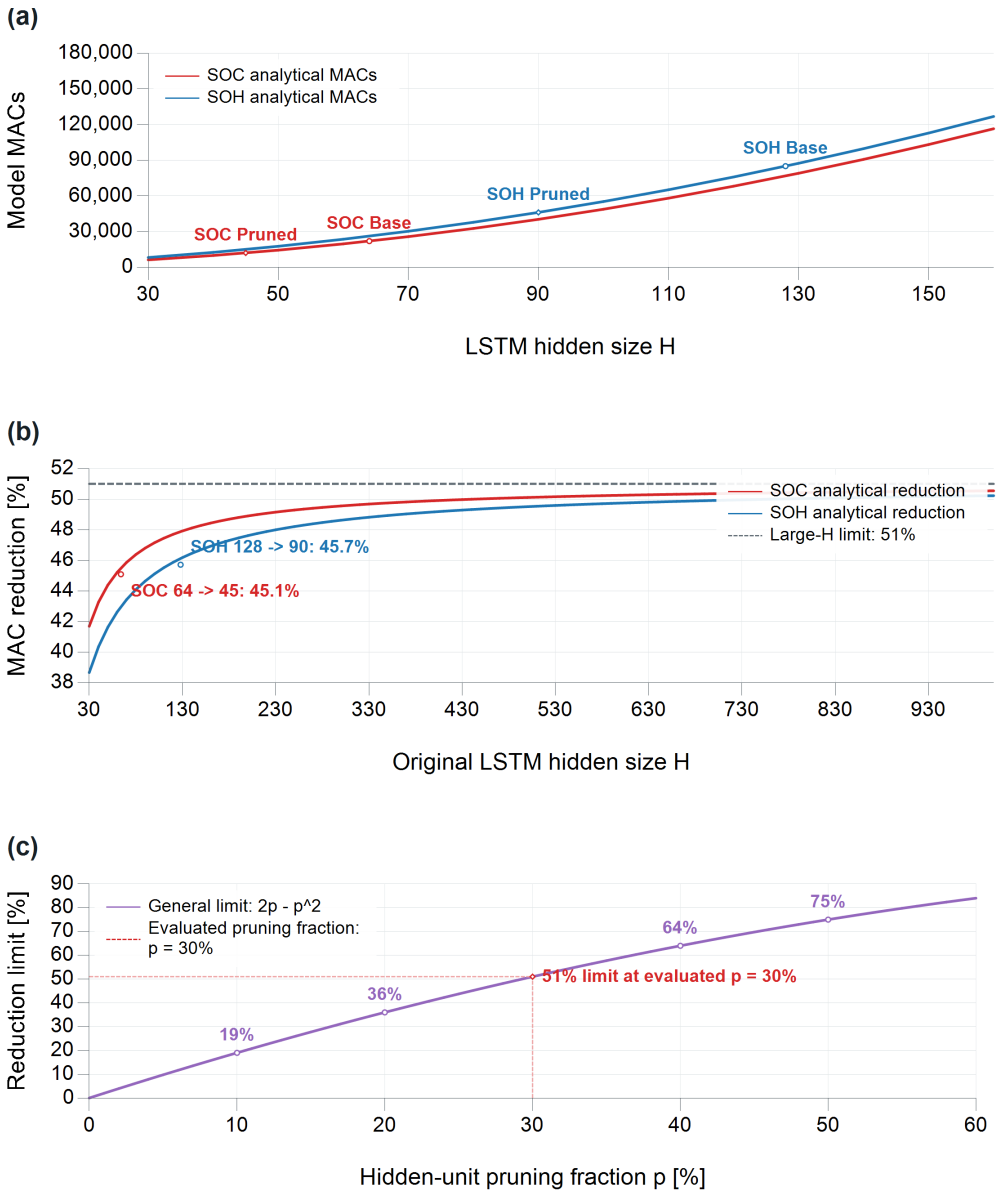
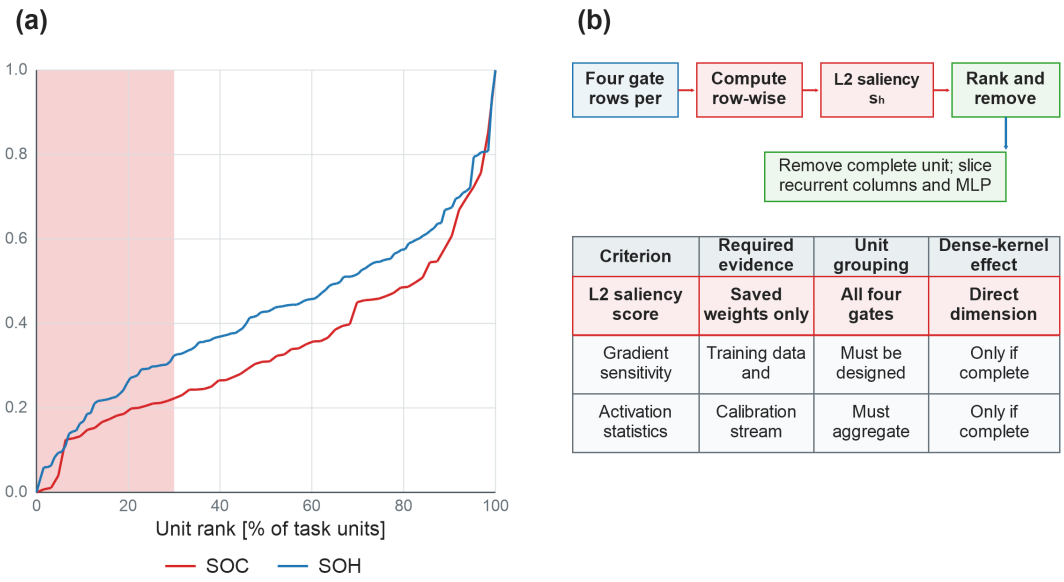


Figure A.7: Analytical MAC count and relative reduction for structured hidden-channel pruning. The finite-size curves follow Equation 6.10. Their large-model limits are $2p - p^2$, and the evaluated 30 % pruning configuration is marked for the SOC and SOH architectures.

A.2.7 Scope of the L2 Pruning Criterion



Criterion	Required evidence	Unit grouping	Dense-kernel effect
L2 saliency score	**Saved weights only**	**All four gates**	**Direct dimension**
Gradient sensitivity	Training data and	Must be designed	Only if complete
Activation statistics	Calibration stream	Must aggregate	Only if complete

Figure A.8: Definition and methodological scope of the L2 gate-group saliency used for hidden-channel selection. The diagram distinguishes the implemented deterministic criterion from gradient- and activation-based alternatives that would require additional data-dependent evaluation.

A.2.8 Precision Boundary of the Quantized Network

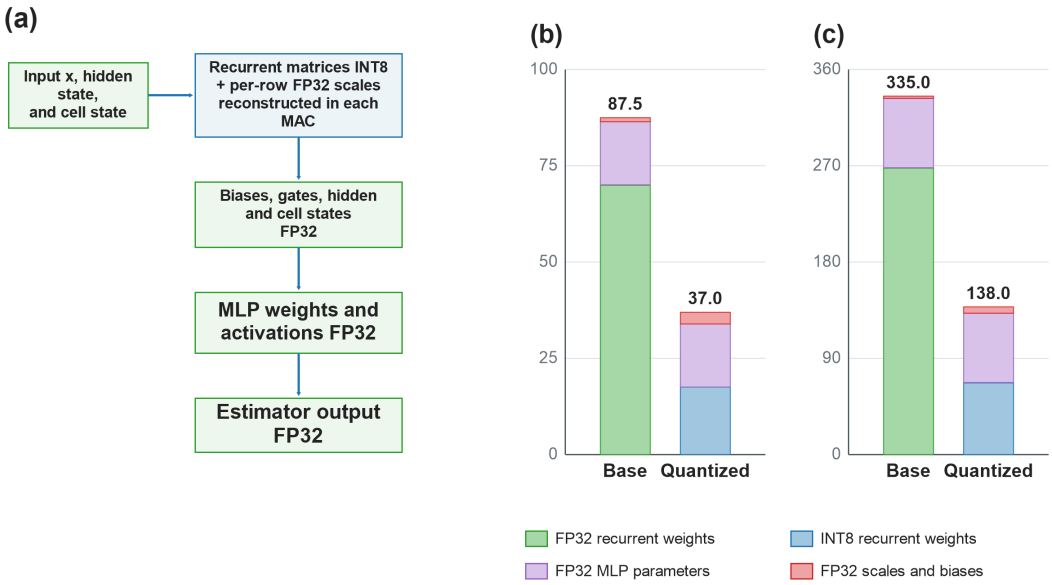


Figure A.9: Numerical path of the weight-only quantized implementation and composition of the stored model constants. Only the recurrent matrices use INT8 storage. Row scales, biases, recurrent states, activations, MLP parameters, and output processing remain in FP32.

A.2.9 Static Counts and Measured Quantized Runtime

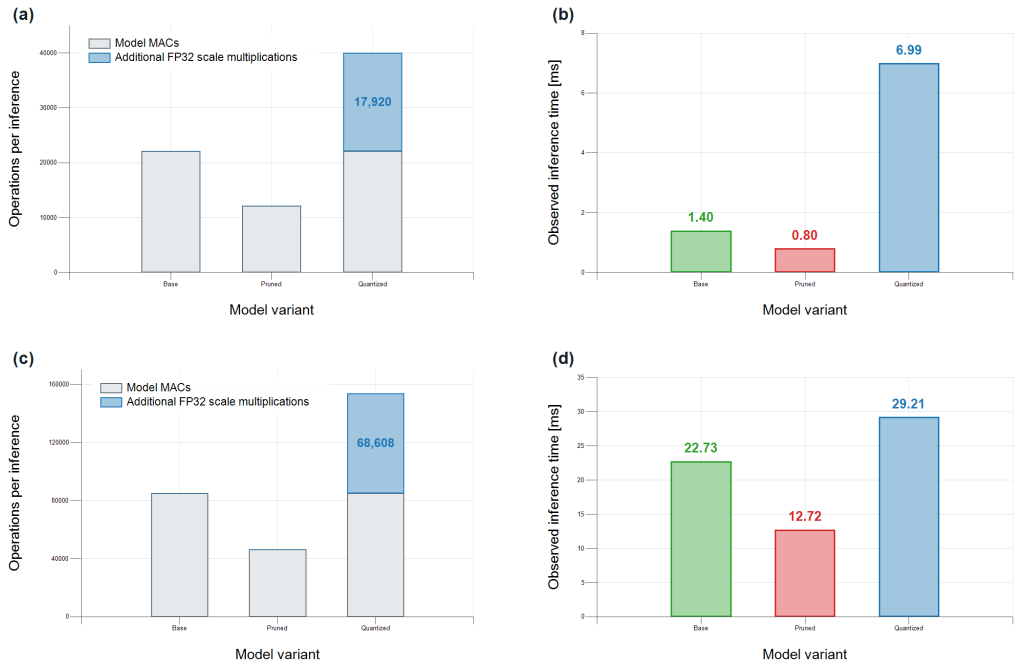


Figure A.10: Comparison of model MACs, additional source-level FP32 scale multiplications, and measured cycle-counter kernel times. The static count captures the explicit mixed-precision arithmetic but excludes conversion, indexing, loop, memory, activation-function, cache, and compiler effects.